Glow65 マニュアル

6502 を読んで学ぶ、動かして学ぶ

Glow65: a 6502 emulator for learning
https://github.com/Masakai/Glow65
2026-10-15 / e528ca9
MIT License

著作権表示とライセンス

Glow65 マニュアル(マニュアルとチュートリアル)
Copyright (c) 2026 masanorisakai

この文書は、Glow65 のリポジトリ(https://github.com/Masakai/Glow65)の一部です。リポジトリ直下の `LICENSE` に書かれた MIT License で公開しています。許諾文は、このページの最後にあります。

第三者の著作物

- Microsoft BASIC for 6502 v1.1: Copyright (c) Microsoft Corporation(MIT License)。Glow65 に同梱し、変換して使っています。https://github.com/microsoft/BASIC-M6502
- GAME 言語: 大西博氏が月刊アスキー(1978 年)で発表した言語です。GAME65G は、公開された仕様から書いた別の実装で、GAME65 のコードは使っていません。
- Klaus Dormann の 6502 機能テスト(GPLv3): 動作の検証に使っています。リポジトリにも、この文書にも含みません。

この文書に出てくる会社名、製品名は、各社の商標または登録商標です。

MIT License

```
MIT License

Copyright (c) 2026 masanorisakai

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

目次

第 1 部 マニュアル

このマニュアルについて

第1章 6502 の歴史

1.1 1975年: 25 ドルの CPU
1.2 6502 の仕様
1.3 Apple I と Apple II
1.4 Commodore PET 2001
1.5 ファミリーコンピュータ
1.6 そのほかの 6502 系
1.7 いま買える 6502
1.8 Glow65 にあるもの・ないもの
出典

第2章 コンピュータの基本

2.1 ビットとバイト
2.2 2進数
2.3 16進数
2.4 数の範囲
2.5 負の数と2の補数
2.6 文字コード(ASCII)
2.7 メモリとアドレス
2.8 ROM、RAM、I/O
2.9 練習問題
2.10 練習問題の答えと解き方

第3章 CPU の中身

3.1 CPU とメモリ
3.2 レジスタ
3.3 バス
3.4 フェッチと実行
3.5 命令の構成
3.6 命令の実行例
3.7 リセットと割り込みのベクタ
3.8 練習問題
3.9 練習問題の答えと解き方
出典

第4章 アドレッシングモード

4.1 アドレッシングモードとは
4.2 オペランドを持たないモード
4.3 イミディエイト(Immediate)
4.4 ゼロページ系のモード
4.5 アブソリュート系のモード
4.6 インダイレクト(Indirect)
4.7 インデックスト・インダイレクト(Indexed Indirect)
4.8 インダイレクト・インデックスト(Indirect Indexed)
4.9 リラティブ(Relative)

(c) 2026 masanorisakai / MIT License

4.10 全モードの一覧
4.11 65C02 で加わるモード
4.12 練習問題
4.13 練習問題の答えと解き方
出典

第5章 命令リファレンス

読み方
分類
全 56 命令の一覧
未定義のオペコード
出典

5.1 ロードとストア

5.2 レジスタ間の転送

5.3 算術演算と比較

5.4 論理演算とシフト

5.5 分岐とジャンプ

出典

5.6 スタックとフラグ操作

5.7 65C02 の追加命令

6502 との違い
Glow65 で 65C02 を動かす
新しい命令の一覧
新しい命令
6502 の命令に加わるモード
未定義だったオペコード
10進モードの違い
練習問題
練習問題の答えと解き方

第6章 フラグ、スタック、割り込み

6.1 フラグ(P レジスタ)
6.2 スタック
6.3 割り込み
6.4 リセット
6.5 練習問題
6.6 練習問題の答えと解き方
出典

第7章 メモリマップと I/O

7.1 メモリマップ
7.2 メモリマップド I/O
7.3 UART
7.4 モニタのルーチン
7.5 ベクタ領域
7.6 練習問題
7.7 練習問題の答えと解き方

(c) 2026 masanorisakai / MIT License

第8章 用語集と索引

8.3 チュートリアルとの対応

8.1 用語集

あ行
か行
さ行
た行
な行
は行
ま行
や行
ら行
わ行

8.2 索引

8.2.1 命令の索引
8.2.2 アドレッシングモードの索引
8.2.3 フラグとレジスタの索引
8.2.4 モニタのコマンドとルーチンの索引
8.2.5 主なアドレスの索引
8.2.6 図の一覧

第 2 部 チュートリアル

チュートリアル: はじめてのプログラム(LED カウンタ)

1. プログラム
2. コンピュータに入れる
3. 実行する
4. 遅くして観察する
5. 結果を見る
6. 試してみる

チュートリアル 2: 足し算とキャリーフラグ

1. プログラム
2. 読み込む
3. 実行する
4. キャリーを観察する
5. 試してみる

チュートリアル 3: UART と話す

1. UART は 2 つのアドレス
2. プログラム
3. 読み込んで実行する
4. パネルで見る
5. 近道: モニタのルーチンを使う
6. 試してみる

チュートリアル 4: キーボードを読む、エコーのプログラム

1. 画面にエコーが必要な理由
2. プログラム
3. 読み込んで実行する
4. パネルで見る

(c) 2026 masanorisakai / MIT License

5. 試してみる

チュートリアル 5: BASIC から機械語を呼ぶ(USR)

1. USR の仕組み
2. プログラム
3. 読み込む
4. 呼ぶ
5. ルーチンの限界
6. 機械語から文字を出す
7. 試してみる
8. 覚えておくこと

チュートリアル 6: 割り込み

1. 割り込みで CPU がすること
2. このボードのベクタ
3. NMI: ボタンを押した回数を数える
4. パネルで割り込みを観察する
5. IRQ: 割り込みでエコーする
6. BRK も割り込み
7. 試してみよう
8. 覚えておくこと

チュートリアル 7: 65C02 に切り替える

1. 65C02 で加わったもの
2. 同じプログラムを2通り
3. アセンブルする
4. 65C02 で動かす
5. 6502 で動かす
6. パネルの CPU スイッチ
7. 試してみよう
8. 覚えておくこと

(c) 2026 masanorisakai / MIT License

このマニュアルについて

このマニュアルは、読んで理解するための資料です。6502 の歴史、CPU の仕組み、命令、メモリの使い方を順に説明します。

手を動かして試す資料は、別の場所にあるチュートリアルです。チュートリアルは、短いプログラムを Glow65 に入れて動かし、LED とモニタで結果を確かめます。マニュアルは、その背景にある考え方と仕様を扱います。どちらを先に読んでもかまいません。

(c) 2026 masanorisakai / MIT License

第1章 6502 の歴史

6502 は、1975年に発表された 8 ビットの CPU(中央処理装置、プログラムの命令を実行する部品)です。Apple II、Commodore PET 2001、ファミリーコンピュータの CPU は、6502 かその派生品です。

[1] のような番号は、章末の出典です。

1.1 1975年: 25 ドルの CPU

6502 は、アメリカの MOS Technology 社が 1975年9月に、サンフランシスコの見本市 WESCON で発表しました[3]。価格は 1 個 25 ドルです[3]。

設計したのは、Chuck Peddle と Bill Mensch らのチームです[2][3]。Peddle がチームを率い、Mensch がチップを設計したとされます[3]。

2人は、もとは Motorola の技術者で、会社は 6800 という CPU を売っていました。Peddle らが出した、もっと安い CPU の案は受け入れられませんでした。1974年8月、Peddle と数人の技術者は Motorola を離れ、MOS Technology に移りました[3][4]。

Motorola は 1975年11月に MOS Technology を訴えました。元社員が Motorola の技術を使ったという主張です。1976年の春に和解し、MOS Technology は 6800 とピン配置が同じ 6501 の販売をやめました[4]。6502 の販売は続きました。

1.2 6502 の仕様

6502 の仕様は、次のとおりです[1]。

項目	内容
データの幅	8 ビット。1 回に 1 バイトを扱う
アドレス(メモリの番地)の幅	16 ビット。2 の 16 乗で、65,536 バイト(64KB)を指定できる
レジスタ	A(アキュムレータ)、X、Y、S(スタックポインタ)、P(ステータス)は 8 ビット。PC(プログラムカウンタ)は 16 ビット
ゼロページ	アドレスの上位が 00 の、先頭 256 バイト
クロック	1 MHz(マニュアルのタイミング表の前提)

1 MHz は、1 秒に 100 万回のクロックを刻むことです。のちに 2 MHz や 3 MHz で動く品種も出ました[3]。トランジスタの数は、約 3,500 個です[3][5]。

(c) 2026 masanorisakai / MIT License

コラム: トランジスタの数え方

6502 のトランジスタは、「約 3,500 個」と書かれることが多い数です。正確には、論理を作るトランジスタが 3,510 個です。このほかに、負荷として使われるトランジスタが 1,018 個あります。両方を合わせると 4,528 個です。数字の違いは、どこまでをトランジスタに数えるかの違いです[5]。

Glow65 で確かめられること

パネルの LED に、16 ビットのアドレスバス(A0 から A15)と 8 ビットのデータバス(D0 から D7)があります。PC、SP、A、X、Y、ステータスフラグの LED もあります。モニタの `D 0000 00FF` で、ゼロページ全体の内容を表示できます。`L 0300 0308` で、メモリの内容を命令として読めます。手順はチュートリアル第1章にあります。

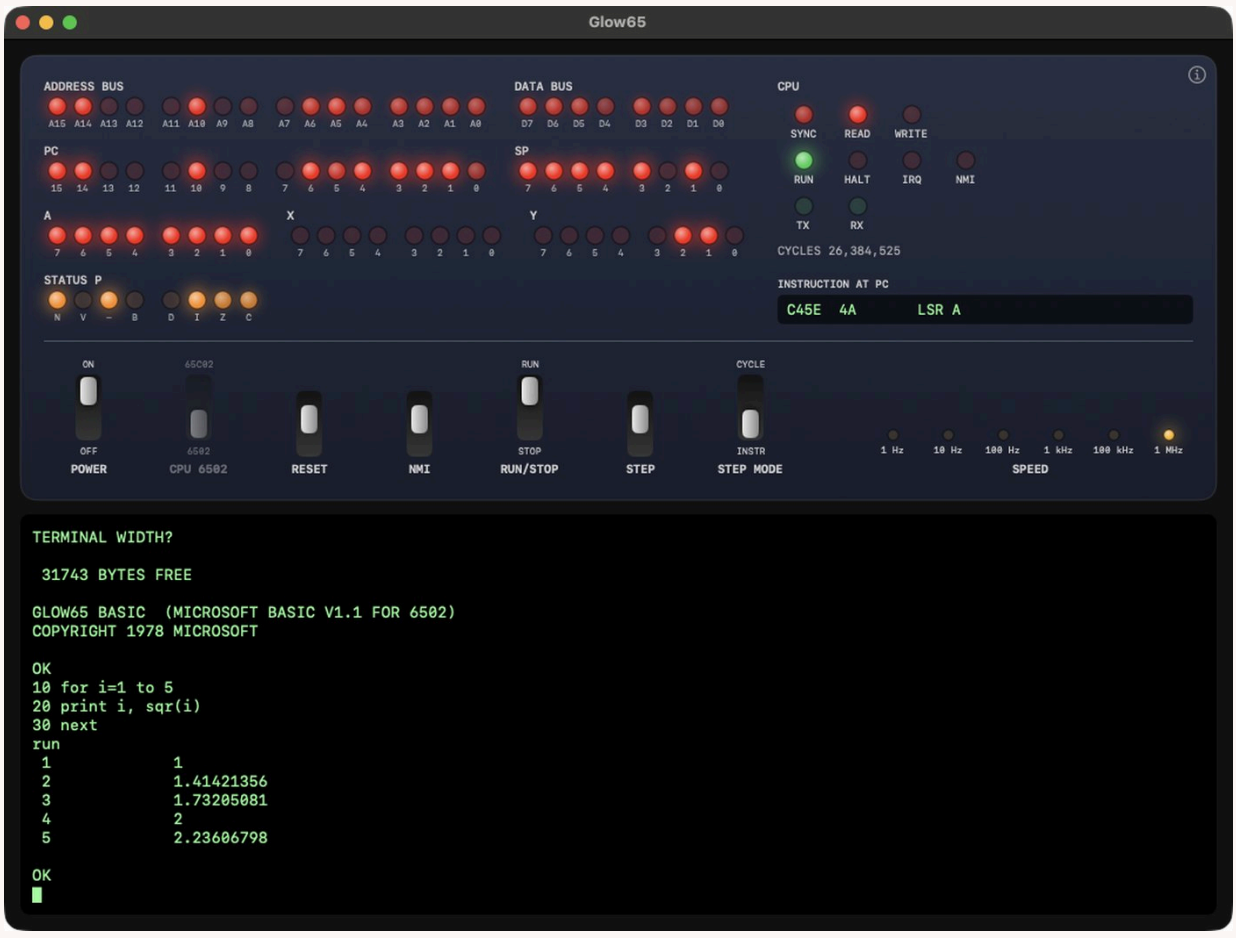


図 1-1 Glow65 のフロントパネル(BASIC の実行中)。上の段に、アドレスバスとデータバスの LED、PC、SP、A、X、Y、ステータスの LED が並ぶ

1.3 Apple I と Apple II

Apple I は 1976年7月に発売されました。CPU は 6502 で、クロックは約 1 MHz、価格は 666.66 ドルです[6]。

Apple II は 1977年6月10日に出荷が始まりました[7]。CPU は 6502 で、クロックは 1.023 MHz です[7]。発売時の価格は、RAM 4KB の構成で 1,298 ドル、最大 48KB の構成で 2,638 ドルでした[7]。

Apple II は、映像を NTSC のコンポジット信号で出力します。文字は 40 桁 24 行です。キーボードは大文字だけで、音は 1 ビットのスピーカーを切り替えて出します。カセットテープに保存できます。ROM(電源を切っても消えない記憶)には、Integer BASIC と、機械語モニタが入っています[8]。

> **コラム: 1.023 MHz という半端なクロック**
>
> Apple II のクロックは、1 MHz ちょうどではありません。NTSC のテレビ信号には、色を表す約 3.579545 MHz の信号があります。Apple II のクロックは、その 2/7 にあたる値です。正確には 1.022727 MHz で、これを丸めて 1.023 MHz と書きます。映像回路と共通の基準から作られた値です[6]。

1.4 Commodore PET 2001

PET 2001 は、Commodore 社のコンピュータです。CPU は 6502、クロックは 1 MHz、価格は 795 ドルでした[9]。

「いつ発売されたか」は、基準によって答えが変わります。

基準	時期
展示	1977年1月、Winter CES(家電見本市)で試作機を公開
出荷	1977年10月に最初の約 100 台。一般の購入者へは 1977年12月

Apple II の出荷(1977年6月)と比べると、展示は PET が先、出荷は Apple II が先です[7][9]。

PET は、画面とキーボード、カセットテープ装置が本体に一体になっています。画面は 40 桁 25 行の白黒表示です。音は、小さなブザーが付くモデルと、付かないモデルがありました。ROM には、BASIC と、基本的な入出力を担当する KERNAL、文字の形が入っています[9]。

1.5 ファミリーコンピュータ

任天堂のファミリーコンピュータ(ファミコン)は、1983年に発売されました[12]。発売日は 7月15日、価格は 14,800 円です[10]。CPU はリコー製の RP2A03 です。6502 と互換のコアに、サウンドを出す回路などを一緒に入れたチップです[10][11]。

リコーの技術者(高橋豊文氏)のインタビューでは、2A03 は「6502 の CPU コアとサウンドが入っているチップ」と説明されています[14]。CPU コアは別のグループが作ったもので、高橋氏はサウンドの回路を担当しました。リコーは、画像処理用のチップ 2C02 と合わせて、CPU を作って任天堂に納めていました[14]。

ファミコンの CPU には、6502 にある 10 進モード(数を 10 進数のまま計算する機能)がありません[15]。

(c) 2026 masanorisakai / MIT License

CPU のクロックは、日本向けの NTSC 版で約 1.79 MHz です。PAL 方式の地域向けは、チップが RP2A07 になり、約 1.66 MHz です[15]。

> **Glow65 で確かめられること**
>
> Glow65 の CPU は、6502 の命令に加えて 10 進モードも動きます。Klaus Dormann の機能テストを、10 進モードを含めて通過しています(README の「テスト」の節)。ファミコンのチップにはない機能です。

> **コラム: ファミコンが 6502 を選んだ理由**
>
> ファミコンの開発に関わった上村雅之氏は、任天堂の公式インタビューで経緯を語っています。6502 を勧めたのは、半導体を作っていたリコーでした。当時の日本では、6502 はほとんど使われていない CPU でした。上村氏は、中身を理解できる人が少なく、他社に解析されにくいことが採用の理由だったと話しています。社内の技術者からは反対されたそうです[13]。

1.6 そのほかの 6502 系

6502 とその派生品を使った例です。

機種・チップ	年	内容
Atari 2600	1977年ごろ	6507 を使用。約 1.19 MHz[16]
VIC-20	1980年(日本)/ 1981年(世界)	6502、約 1.02 MHz(NTSC)[17]
BBC Micro	1981年12月	6502A、2 MHz[18]
Commodore 64	1982年8月	6510、約 1.023 MHz(NTSC)[19]
65C02	1983年	消費電力を下げた CMOS 版。WDC が開発[20]
65816	1985年ごろ	内部が 16 ビット、アドレスは 24 ビット(16MB)[21]
PC エンジン	1987年10月30日	Hudson Soft の HuC6280(65C02 を拡張)[22][23]
スーパーファミコン	1990年11月21日	Ricoh 5A22(65816 の派生)[24]

CMOS は、消費電力が少ないトランジスタの作り方です。

> **コラム: アドレス端子を減らした 6507**
>
> Atari 2600 の CPU は、6502 の端子の一部を減らした 6507 です。6502 は 40 ピンですが、6507 は 28 ピンです。アドレス端子は 16 本から 13 本に減り、指定できるのは、2 の 13 乗で 8,192 バイト(8KB)までです。割り込みの端子もありません。端子を減らして、安く作るための CPU でした。チップの内部は 6502 と同じです[16][27]。

(c) 2026 masanorisakai / MIT License

1.7 いま買える 6502

6502 の系列は、現在も販売されています。WDC(Western Design Center)は、公式ページで W65C02S を案内しています。パッケージは 40 ピン DIP、44 ピン PLCC、44 ピン QFP の 3 種類で、公式ページは Mouser や Jameco などの販売店を案内しています[25]。

6502 は、25 ドルで発表された安価な CPU です。トランジスタは約 3,500 個で[3][5]、MOS Technology はハードウェアマニュアルを出していました[1]。6507、6510、2A03、65C02、65816、HuC6280 など、用途に合わせた派生品も作られました。

1.8 Glow65 にあるもの・ないもの

Glow65 は、6502 の学習用に作った、架空のワンボードコンピュータのエミュレータです。3 つの実機との違いは、次の表のとおりです。

項目	Apple II(1977年)	PET 2001(1977年)	ファミコン(1983年)	Glow65
CPU	6502、1.023 MHz[7]	6502、1 MHz[9]	RP2A03(6502 互換、10 進モードなし)、約 1.79 MHz[15]	NMOS 6502(既定)。パネルの SPEED で 1 Hz から 1 MHz まで切り替え。CPU スイッチで 65C02 にも切り替え可
メモリ	RAM 4KB 〜 48KB[7]	RAM 4KB または 8KB[9]	CPU 用 RAM 2KB、映像用 RAM 2KB[26]	RAM 32KB(`$0000–$7FFF`)
画面表示	NTSC コンポジット出力、40 桁 24 行[8]	本体に一体の白黒画面、40 桁 25 行[9]	画像処理チップ 2C02、256 x 240 ドット[26]	UART につないだ端末だけ。専用のグラフィック画面はなし。CPU の状態を見るフロントパネル(LED)あり
入力	キーボード(大文字のみ)[8]	本体のキーボード[9]	ゲームパッド(本体に固定された 2 つ)[26]	端末からのキーボード入力(UART 経由)
サウンド	1 ビットのスピーカー[8]	小さなブザーのみ(なしのモデルもあり)[9]	5 チャンネル[26]	なし
外部記憶	カセットテープ[8]	本体のカセット装置[9]	本体にはなし[26]	なし。BASIC の `SAVE` と `LOAD` は使えない。ファイルの読み込み(File → Load Program)は、エミュレータが RAM へ直接書き込む機能
ROM のソフト	Integer BASIC、機械語モニタ(8KB)[8]	BASIC、KERNAL、文字の形[9]	ゲームのカートリッジ(プログラム用チップは最大 32KB)[26]	モニタ(`$F000–$FFFF`)、Microsoft BASIC(`$C000–$EFFF`)、GAME65G(`$8000–$9FFF`)

Glow65 の BASIC は、Microsoft の 6502 用 BASIC(1978年の v1.1)[28]を、このボード向けに変換したものです。

(c) 2026 masanorisakai / MIT License

Glow65 の目的は、実機の再現ではなく、CPU の動きを見ることです。実機の余分なダミーリードとダミーライトは再現していないため、バスの表示は実際の 6502 と少し異なります。

出典

1. MOS Technology, "MCS6500 Microcomputer Family Hardware Manual", 1976年1月版: https://www.commodore.ca/wp-content/uploads/2018/11/kim1_hardware.txt
2. IEEE Spectrum, "Chip Hall of Fame: MOS Technology 6502 Microprocessor": https://spectrum.ieee.org/chip-hall-of-fame-mos-technology-6502-microprocessor
3. Wikipedia, "MOS Technology 6502": https://en.wikipedia.org/wiki/MOS_Technology_6502
4. Wikipedia, "Motorola 6800": https://en.wikipedia.org/wiki/Motorola_6800
5. Ken Shirriff, "Intel x86 documentation has more pages than the 6502 has transistors": https://www.righto.com/2013/09/intel-x86-documentation-has-more-pages.html
6. Wikipedia, "Apple I": https://en.wikipedia.org/wiki/Apple_I
7. Wikipedia, "Apple II": https://en.wikipedia.org/wiki/Apple_II
8. Wikipedia, "Apple II (original)": https://en.wikipedia.org/wiki/Apple_II_(original)
9. Wikipedia, "Commodore PET": https://en.wikipedia.org/wiki/Commodore_PET
10. Wikipedia 日本語版, "ファミリーコンピュータ": https://ja.wikipedia.org/wiki/ファミリーコンピュータ
11. Wikipedia, "Ricoh 2A03": https://en.wikipedia.org/wiki/Ricoh_2A03
12. 任天堂 沿革: https://www.nintendo.co.jp/corporate/history/index.html
13. 任天堂「社長が訊く マリオ25周年 Vol.2」: https://www.nintendo.co.jp/n10/interview/mario25th/vol2/index2.html
14. 高橋豊文氏オーラルヒストリー(第2回前半)、一橋大学 IIR Working Paper WP#19-17、2019年: https://pubs.iir.hit-u.ac.jp/admin/ja/pdfs/file/2267
15. NESdev Wiki, "CPU": https://www.nesdev.org/wiki/CPU
16. Wikipedia, "Atari 2600": https://en.wikipedia.org/wiki/Atari_2600
17. Wikipedia, "VIC-20": https://en.wikipedia.org/wiki/VIC-20
18. Wikipedia, "BBC Micro": https://en.wikipedia.org/wiki/BBC_Micro
19. Wikipedia, "Commodore 64": https://en.wikipedia.org/wiki/Commodore_64
20. Wikipedia, "WDC 65C02": https://en.wikipedia.org/wiki/WDC_65C02
21. Wikipedia, "WDC 65C816": https://en.wikipedia.org/wiki/WDC_65C816
22. Wikipedia, "TurboGrafx-16": https://en.wikipedia.org/wiki/TurboGrafx-16
23. Wikipedia, "Hudson Soft HuC6280": https://en.wikipedia.org/wiki/Hudson_Soft_HuC6280
24. Wikipedia, "Super Nintendo Entertainment System": https://en.wikipedia.org/wiki/Super_Nintendo_Entertainment_System
25. Western Design Center, "W65C02S": https://www.westerndesigncenter.com/wdc/w65c02s-chip.php
26. Wikipedia, "Nintendo Entertainment System": https://en.wikipedia.org/wiki/Nintendo_Entertainment_System
27. Wikipedia, "MOS Technology 6507": https://en.wikipedia.org/wiki/MOS_Technology_6507

(c) 2026 masanorisakai / MIT License

28. Microsoft, "BASIC-M6502"(Microsoft BASIC for 6502 のソース):
https://github.com/microsoft/BASIC-M6502

(c) 2026 masanorisakai / MIT License

第2章 コンピュータの基本

CPU が扱うのは、0 と 1 の並びだけです。この章では、その並びを人間が読み書きする方法を説明します。ビットとバイト、2進数と16進数、負の数、文字、メモリとアドレスを順に扱います。数学の前提は、2 のべき乗(2、4、8、16 ...)と四則演算までです。

各節の末尾に、Glow65 で確かめられる操作を載せています。モニタの使い方は、README の「モニタのコマンド」にあります。練習問題と答えは、章末にあります。

2.1 ビットとバイト

ビットは、0 か 1 のどちらかを持つ、最小の単位です。スイッチの ON と OFF にあたります。

バイトは、8 ビットをひとまとめにした単位です。6502 は、1 バイトずつデータを扱います。ビットを 8 個並べると、0 と 1 の組み合わせは 2 の 8 乗で 256 通りになります。

8 個のビットには、右端を 0 番として、左へ 1 番、2 番、...、7 番と番号を付けます。0 番を**最下位ビット**、7 番を**最上位ビット**と呼びます。

2.2 2進数

ふだん使う 10 進数は、1 桁ごとに 10 倍になります。345 は、3 x 100 + 4 x 10 + 5 x 1 です。

2進数は、1 桁ごとに 2 倍になる数の書き方です。使う数字は 0 と 1 だけです。8 ビットの各桁の重みは、左から次のとおりです。

ビット番号	7	6	5	4	3	2	1	0
重み	128	64	32	16	8	4	2	1

この本では、2進数の先頭に `%` を付けます。`%01101001` は、重み 64、32、8、1 のビットが 1 なので、64 + 32 + 8 + 1 で 105 です。

2進数から10進数への変換は、1 のビットの重みを足します。

10進数から2進数への変換は、大きい重みから順に、引けるかどうかを調べます。200 の場合は次のとおりです。

1. 200 から 128 を引ける。残り 72。ビット 7 は 1。
2. 72 から 64 を引ける。残り 8。ビット 6 は 1。
3. 8 は 32 より小さい。ビット 5 は 0。16 より小さい。ビット 4 は 0。
4. 8 から 8 を引ける。残り 0。ビット 3 は 1。
5. 残りはないので、ビット 2、1、0 は 0。

結果は `%11001000` です。

(c) 2026 masanorisakai / MIT License

2.3 16進数

2進数は、桁数が多くて読みにくくなります。8 ビットで 8 桁、16 ビットでは 16 桁です。そこで、4 ビットずつまとめて 1 桁で書く**16進数**を使います。

4 ビットは 0 から 15 までの 16 通りです。16進数は、1 桁で 0 から 15 を表します。10 から 15 には、アルファベットの A から F を使います。

10進	2進	16進		10進	2進	16進
0	%0000	0		8	%1000	8
1	%0001	1		9	%1001	9
2	%0010	2		10	%1010	A
3	%0011	3		11	%1011	B
4	%0100	4		12	%1100	C
5	%0101	5		13	%1101	D
6	%0110	6		14	%1110	E
7	%0111	7		15	%1111	F

6502 の世界では、16進数の先頭に `$` を付けます。`$A0` は 16進数の A0 です。この本も、これに従います。

2進数と16進数の変換は、4 ビットずつ区切って、表を引くだけです。

- `%10100000` を 4 ビットずつに分けると、`1010` と `0000` です。表から、A と 0 です。結果は `$A0` です。
- `$3C` は、3 が `0011`、C が `1100` です。結果は `%00111100` です。

16進数から10進数への変換は、上の桁に 16 を掛けて、下の桁を足します。`$A0` は、A が 10 なので 10 x 16 + 0 で 160 です。

10進数から16進数への変換は、16 で割ります。商が上の桁、余りが下の桁です。200 を 16 で割ると、商が 12、余りが 8 です。12 は C なので、`$C8` です。

16 ビットの値は、16進数 4 桁で書きます。`$1234` は、上位の 2 桁 `$12` と、下位の 2 桁 `$34` に分けて考えます。

> **コラム: 16進数の書き方いろいろ**
>
> 16進数の書き方は、言語や道具によって違います。6502 のアセンブラでは、`$` を付けるのが一般的です。C 言語などでは `0x` を付けて、`0xA0` と書きます。Intel 系のアセンブラでは、末尾に `h` を付けて `0A0h` と書きます。どれも同じ数を表します。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

パネルのデータバスの LED は、D7 から D0 までの 8 個です。バスにある値の各ビットが 1 なら、その LED が点きます。値が `$A0`(`%10100000`)のとき、点くのは D7 と D5 の 2 個です。アドレスバスは A15 から A0 までの 16 個です。

`$A0` を LED で見るには、`LDA #$A0` を実行して、A レジスタの LED を読みます。

1. モニタで `E 0300 A9 A0 8D F0 03 4C 00 03` を入力し、`G 0300` を実行する。プログラムは、`LDA #$A0`、`STA $03F0`、`JMP $0300` を繰り返す。
2. **RUN/STOP** を STOP にして、CPU を止める。**STEP MODE** を INSTR にする。
3. **STEP** を押して、INSTRUCTION AT PC が `LDA #$A0` になるまで進め、もう 1 回押す。

`LDA #$A0` を実行した直後は、A の LED で A7 と A5 が点灯します(図 2-1)。DATA BUS も、D7 と D5 が点灯します。直前に読んだオペランドが `$A0` だからです。ADDRESS BUS は、オペランドを読んだ番地 `$0301` です。最上位ビットが 1 なので、N フラグの LED も点灯します。

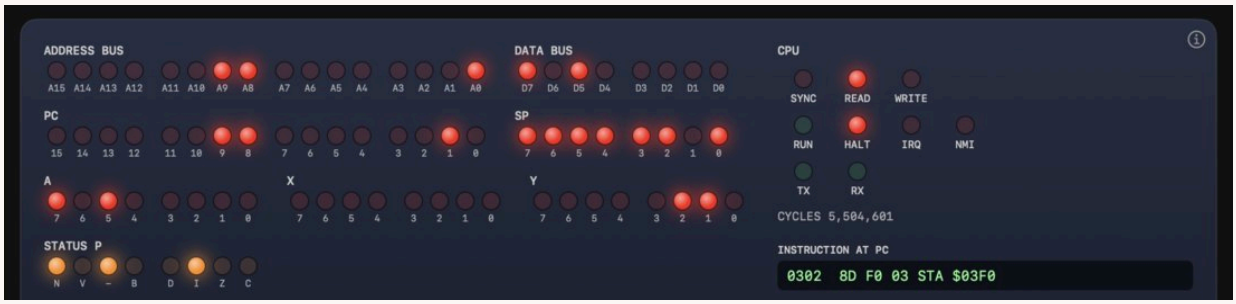


図 2-1 `LDA #$A0` のあと。A の LED は A7 と A5、DATA BUS は D7 と D5、ADDRESS BUS は `$0301`

2.4 数の範囲

8 ビットで表せる数は、すべてのビットが 0 のときの 0 から、すべてが 1 のときの 255 までです。16進数では `$00` から `$FF` です。

16 ビットなら、0 から 65535 まで、`$0000` から `$FFFF` です。65535 は、2 の 16 乗から 1 を引いた数です。

ビット数	範囲(10進)	範囲(16進)
8	0 〜 255	`$00` 〜 `$FF`
16	0 〜 65535	`$0000` 〜 `$FFFF`

範囲を超える計算をすると、入りきらなかった桁が捨てられます。8 ビットで 255 + 1 を計算すると、結果は 256 ではなく 0 になります。桁があふれたことは、**キャリー**という印(フラグ)に記録されます。キャリーは第6章で説明します。

(c) 2026 masanorisakai / MIT License

2.5 負の数と2の補数

CPU が持っているのは、ビットの並びだけです。負の数も、ビットの並びで表します。6502 では、**2の補数**という方式を使います。

2の補数では、最上位ビット(ビット 7)が 1 なら負の数、0 なら 0 以上の数と読みます。このビットを**符号ビット**と呼びます。8 ビットで表せる範囲は、-128 から 127 です。

2進	16進	符号なしで読むと	符号付きで読むと
%00000000	`$00`	0	0
%00000001	`$01`	1	1
%01111111	`$7F`	127	127
%10000000	`$80`	128	-128
%11111111	`$FF`	255	-1

同じ `$FF` が、読み方によって 255 にも -1 にもなります。CPU は、どちらで読むかを知りません。プログラムを書く人が決めます。

負の数の作り方は、2 つの手順です。-5 の場合は次のとおりです。

1. 5 の 2進数 `%00000101` の、0 と 1 をすべて反転する。`%11111010` になる。
2. 1 を足す。`%11111011`、つまり `$FB` になる。

確かめ方は、256 を足して見ることです。`$FB` は 251 で、251 - 256 は -5 です。また、`$FB` に 5 を足すと 256 になり、8 ビットでは 0 に戻ります。

2の補数には、符号付きの数も、符号なしの数と同じ足し算の回路で計算できるという利点があります。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

分岐命令 `BNE` は、飛び先を、次の命令からの距離(符号付き 1 バイト)で持ちます。モニタの `L` で、距離のバイトが飛び先の番地に変わる様子を見られます。

```
*E 0300 D0 FB
*L 0300 0301
0300  D0 FB     BNE   $02FD
```

`BNE` は 2 バイトの命令なので、次の命令は `$0302` です。距離 `$FB` は -5 です。0302 から 5 戻って `$02FD` になります。`D0 7F`(距離 +127)なら `BNE $0381`、`D0 F0`(距離 -16)なら `BNE $02F2` と表示されます。

2 の補数は、A の LED でも確かめられます。上の 2.3 と同じ手順で、`E 0300 A9 FB 8D F0 03 4C 00 03` を入力して実行し、`LDA #$FB` のあとで止めます。A の LED は、A2 だけが消灯して、`%11111011`(`$FB`、-5)になります(図 2-2)。最上位ビットが 1 なので、N フラグも点灯します。CPU は、この値を符号なしの 251 とも、符号付きの -5 とも読めます。

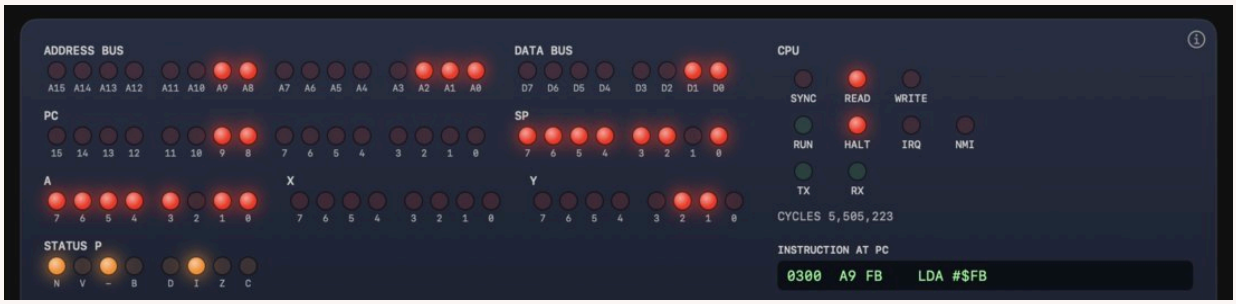


図 2-2 `LDA #$FB` のあと。A の LED は A2 以外が点灯

2.6 文字コード(ASCII)

コンピュータは、文字も数で表します。どの文字にどの数を割り当てるかを決めた表が**文字コード**です。英数字と記号では、ASCII(アスキー)という割り当てが広く使われています。Glow65 の UART が送受信するのは、ASCII です。

ASCII の主な割り当ては、次のとおりです。

文字	コード	備考
`0` 〜 `9`	`$30` 〜 `$39`	数字の文字。`0` が `$30`、`9` が `$39`
`A` 〜 `Z`	`$41` 〜 `$5A`	大文字。`A` が `$41`
`a` 〜 `z`	`$61` 〜 `$7A`	小文字。`a` が `$61`
CR	`$0D`	改行(キャリッジリターン)。Return キーで送られる

(c) 2026 masanorisakai / MIT License

数字の文字 `7` は、数の 7 ではなく、コード `$37` です。文字 `7` から数の 7 を得るには、`$30` を引きます。

大文字と小文字は、コードの差がちょうど `$20` です。`A`(`$41`)と `a`(`$61`)がその例です。

ASCII は 7 ビットで足ります。8 ビットのバイトの最上位ビットは、使われません。Glow65 のモニタの `CHROUT` ルーチンは、出力するバイトの最上位ビットを捨てます(README のジャンプテーブル)。

> **Glow65 で確かめられること**
>
> UART のデータは `$A001` に書きます。`LDA #$48`、`STA $A001` を並べたプログラムを作ると、`H`(`$48`)が端末に出ます。次は、`H`、`I`、CR を順に送る 16 バイトのプログラムです。`G` で実行し、最後の `60`(`RTS`)でモニタに戻ります。
>
> ```
> *E 0300 A9 48 8D 01 A0 A9 49 8D 01 A0 A9 0D 8D 01 A0 60
> *G 0300
> HI
> ```
>
> `$0D` を送ると、端末の表示が改行されます。

2.7 メモリとアドレス

プログラムとデータは、**メモリ**に置きます。メモリは、1 バイトずつ入る箱が一列に並んだ棚です。箱の一つ一つに、先頭から順に番号を付けます。この番号が**アドレス**(番地)です。

6502 のアドレスは 16 ビットです。表せる番号は、`$0000` から `$FFFF` までの 65,536 個です。1 つの箱が 1 バイトなので、全体で 65,536 バイトになります。1 KB(キロバイト)を 1,024 バイトとすると、これは 64 KB です。この全体を**アドレス空間**と呼びます。

16 ビットのアドレスをメモリに置くには、2 バイト使います。6502 は、**下位のバイトを先**に、上位のバイトを次に置きます。この並べ方を**リトルエンディアン**と呼びます。

たとえば、アドレス `$1234` を `$0300` から置くと、次のようになります。

アドレス	`$0300`	`$0301`
内容	`$34` (下位)	`$12` (上位)

見た目の順序と逆なので、読むときは注意してください。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

次の 3 つのコマンドで、メモリの読み書きを確かめられます。

- `E` は、指定したアドレスから、バイトを順に書き込みます。
- `D` は、メモリの内容を 16 進数で表示します。表示は 16 バイト単位で、開始アドレスから 16 個を出します。
- `L` は、メモリの内容を命令として読みます。

`$1234` を下位から置く例です。

```
*E 0300 34 12
*D 0300 0301
0300: 34 12 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

`$0300` に `34`、`$0301` に `12` が入っています。先頭の 2 バイトが、下位、上位の順です。

BASIC でも、同じことを確認できます。モニタで `B` を入力して BASIC を起動し、Return を 2 回押して、次を入力します。

```
POKE 768,52 : POKE 769,18
PRINT PEEK(769)*256+PEEK(768)
 4660
```

768 は `$0300`、52 は `$34`、18 は `$12` です。上位バイトに 256 を掛けて下位バイトを足すと、4660 になります。4660 は `$1234` の 10進数です。

16 ビットの値を 2 バイトで置く例は、6502 自身の中にもあります。CPU は、電源が入ったときや割り込みのときに、アドレス `$FFFA` から `$FFFF` までの 6 バイトを読みます。そこには、3 つの飛び先のアドレスが、リトルエンディアンで並んでいます。詳しくは第6章で説明します。

Glow65 で確かめられること

モニタの `D FFF0 FFFF` で、末尾の 6 バイトを見られます。

```
*D FFF0 FFFF
FFF0: FF FF FF FF FF FF FF FF FF FF 1E F4 2A F1 21 F4
```

末尾の 6 バイトは、`1E F4`、`2A F1`、`21 F4` の 3 組です。リトルエンディアンなので、`$F41E`、`$F12A`、`$F421` と読みます。`$FFFC` と `$FFFD` の `$F12A` が、電源投入時に CPU が最初に飛ぶアドレスです。

(c) 2026 masanorisakai / MIT License

2.8 ROM、RAM、I/O

アドレス空間は、1 種類の部品だけでできているとは限りません。同じアドレスの付け方で、別の役割の部品をつなぎます。

- **RAM**は、読み書きできるメモリです。電源を切ると内容が消えます。プログラムの変数や、作業用のデータを置きます。
- **ROM**は、読み出し専用のメモリです。電源を切っても内容が残ります。起動時に動くプログラムを置きます。
- **I/O**(入出力)は、外の装置との窓口です。メモリのアドレスの一部に、装置が割り当てられます。そのアドレスを読み書きすると、装置とのやり取りになります。この方式を、メモリマップド I/O と呼びます。

Glow65 のアドレス空間は、次のように使います(README の「メモリマップ」と同じ内容です)。

アドレス	種類	内容
`$0000–$7FFF`	RAM	32KB
`$0200–$027F`	RAM の中	モニタの行入力バッファ
`$0300–$03F7`	RAM の中	自作プログラム用に空けてある領域
`$03F8–$03F9`	RAM の中	NMI ハンドラのアドレス
`$03FA–$03FB`	RAM の中	IRQ と BRK のハンドラのアドレス
`$0400–$7FFF`	RAM の中	BASIC か GAME65G のプログラムと変数
`$8000–$9FFF`	ROM	GAME65G
`$A000`	I/O	UART の状態と制御
`$A001`	I/O	UART のデータ
`$C000–$EFFF`	ROM	Microsoft BASIC(約 8.6KB を使用)
`$F000–$FFFF`	ROM	モニタとベクタ(`$FFFA–$FFFF`)

`$A000–$A0FF` は UART に割り当てられています。偶数番地が状態と制御、奇数番地がデータとして働き、通常使うのは `$A000` と `$A001` です。上の表に載っていないアドレスには、何もつながっていません。そこを読むと `$FF` が返ります。ROM に書き込んでも、内容は変わりません。

UART は、端末とやり取りするための装置です。`$A001` に書いたバイトは端末へ出て、端末から入力されたバイトは `$A001` から読めます。`$A000` は、受信できるか、送信できるかを知らせる状態です。詳しい使い方は、第7章で説明します。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

何もつながっていないアドレスは、`$FF` になります。

```
*D B000 B00F
B000: FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
```

ROM への書き込みは、無視されます。`$F000` は ROM で、モニタの先頭の `JMP ($4C)`が入っています。

```
*E F000 00
*D F000 F00F
F000: 4C 0F F0 4C 28 F0 4C 37 F0 4C 4B F1 4C 1A F0 AD
```

`E F000 00` のあとも、`$F000` は `4C` のままです。RAM の `$0300` で同じ操作をすると、書いた値が残ります(2.7 の例)。

2.9 練習問題

答えと解き方は、次の節にあります。

問1 次の 2進数を 10進数にしてください。(1) `%10110101` (2) `%01100110`

問2 次の 10進数を 8 ビットの 2進数にしてください。(1) 200 (2) 77

問3 次の 16進数を 10進数にしてください。(1) `$2F` (2) `$C8`

問4 次の 10進数を 16進数にしてください。(1) 100 (2) 173

問5 次の 16進数を 2進数にしてください。(1) `$5A` (2) `$E3`

問6 `%10010110` を 16進数にしてください。

問7 次の数を、8 ビットの 2の補数で 16進数にしてください。(1) -3 (2) -100

問8 次の 8 ビットの値を、符号付きの数として 10進数で読んでください。(1) `$F0` (2) `$80`

問9 8 ビットで `$7F` に `$01` を足すと `$80` になります。(1) 符号なしの数として読むと、いくつからいくつへの変化ですか。(2) 符号付きの数として読むと、いくつからいくつへの変化ですか。

問10 文字列 `HI` の ASCII コードを、16進数で答えてください。文字 `7` のコードは `$37` です。この文字から、数の 7 を得る計算を答えてください。

問11 `$0300` に `5A`、`$0301` に `02` が入っています。この 2 バイトが 16 ビットのアドレスなら、いくつですか。答えを、Glow65 のモニタで確かめる方法も書いてください。

(c) 2026 masanorisakai / MIT License

問12 次のアドレスの範囲は、何バイトですか。(1) `$0400` から `$7FFF` まで (2) `$0300` から `$03F7` まで

2.10 練習問題の答えと解き方

問1 (1) 181 (2) 102

- (1) 128 + 32 + 16 + 4 + 1 = 181
- (2) 64 + 32 + 4 + 2 = 102

問2 (1) `%11001000` (2) `%01001101`

- (1) 本文の 2.2 を参照。128 + 64 + 8 = 200。
- (2) 77 から 64 を引くと 13。13 から 8、4、1 を引くと 0。ビット 6、3、2、0 が 1。

問3 (1) 47 (2) 200

- (1) 2 x 16 + 15 = 47
- (2) 12 x 16 + 8 = 200

問4 (1) `$64` (2) `$AD`

- (1) 100 を 16 で割ると、商 6、余り 4。
- (2) 173 を 16 で割ると、商 10、余り 13。10 は A、13 は D。

問5 (1) `%01011010` (2) `%11100011`

- (1) 5 は `0101`、A は `1010`。
- (2) E は `1110`、3 は `0011`。

問6 `$96`

- `1001` は 9、`0110` は 6。

問7 (1) `$FD` (2) `$9C`

- (1) 3 は `%00000011`。反転して `%11111100`、1 を足して `%11111101` で `$FD`。
- (2) 100 は `$64`、`%01100100`。反転して `%10011011`、1 を足して `%10011100` で `$9C`。確かめ: `$9C` は 156 で、156 - 256 は -100。

問8 (1) -16 (2) -128

- (1) `$F0` は 240。240 - 256 = -16。
- (2) `$80` は 128。128 - 256 = -128。最上位ビットだけが 1 の値が、8 ビットで一番小さい数です。

問9 (1) 127 から 128 (2) 127 から -128

(c) 2026 masanorisakai / MIT License

- 同じビットの変化が、符号なしでは 1 つ増えただけです。符号付きでは、一番大きい数から一番小さい数に変わります。

問10 `HI` は `$48 $49`。数の 7 は、`$37 - $30 = 7`。

- `H` は `$48`、`I` は `$49`。数字の文字から `$30` を引くと、数になる。

問11 `$025A` (10進数で 602)

- 下位バイトが先なので、上位は `$02`、下位は `$5A`。
- モニタで確かめる方法は、`E 0300 5A 02` を入力して `D 0300 0301` で内容を表示することです。表示の先頭が `5A 02` になります。
- BASIC なら、`POKE 768,90 : POKE 769,2` のあと `PRINT PEEK(769)*256+PEEK(768)` で、602 になります。

問12 (1) 31,744 バイト (2) 248 バイト

- (1) `$7FFF - $0400 + 1`。`$7FFF` は 32767、`$0400` は 1024。32767 - 1024 + 1 = 31744。
- (2) `$03F7 - $0300 + 1`。1015 - 768 + 1 = 248。

(c) 2026 masanorisakai / MIT License

第3章 CPU の中身

CPU は、メモリにある命令を 1 つずつ取り出して実行する部品です。この章では、6502 の内部にある記憶領域(レジスタ)と、メモリとの間で信号をやり取りする線(バス)、命令を実行する手順(フェッチと実行)を説明します。

3.1 CPU とメモリ

プログラムは、メモリに置いたバイトの列です。CPU は、メモリの先頭から順に命令のバイトを読み、その内容に従って動きます。足し算をする、メモリに値を書く、別の場所へ飛ぶ、といった動きです。

CPU が次に読む場所は、**PC**(プログラムカウンタ)というレジスタが覚えています。命令を 1 つ読むたびに、PC はその命令の長さだけ進みます。

3.2 レジスタ

レジスタは、CPU の内部にある、数バイト分の記憶領域です。メモリより速く読み書きできます。6502 のレジスタは、次の 6 つです[1]。

レジスタ	幅	役割
A(アキュムレータ)	8 ビット	計算の中心。足し算や比較の対象になる
X	8 ビット	インデックスレジスタ。アドレスにずらしを加えるときなどに使う
Y	8 ビット	X と同じ役割のレジスタ
SP(スタックポインタ)	8 ビット	スタックの現在位置。スタックは `$0100–$01FF` を使う
PC(プログラムカウンタ)	16 ビット	次に実行する命令のアドレス
P(ステータス)	8 ビット	直前の命令の結果を表す、フラグの集まり

A、X、Y は、プログラムが自由に使えます。SP は第6章で扱います。X と Y の使い方は第4章で扱います。

フラグ

P は、1 ビットずつに意味があります。1 ビットを**フラグ**と呼びます。上位のビットから、次のように並んでいます。

ビット	7	6	5	4	3	2	1	0
名前	N	V	-	B	D	I	Z	C

(c) 2026 masanorisakai / MIT License

ビット	7	6	5	4	3	2	1	0
意味	ネガティブ。結果の最上位ビットが 1	オーバーフロー。符号付きの結果が範囲を超えた	未使用	ブレーク	10進モード	割り込み禁止	ゼロ。結果が 0	キャリー。桁あふれ

各フラグの働きは、第6章で説明します。ここでは、**N と Z** を覚えてください。A や X に値が入る命令の多くは、その値が 0 なら Z を 1 に、最上位ビットが 1 なら N を 1 にします。

リセット後の値

CPU は、リセット(電源投入や RESET スイッチ)のあとに、決まった状態から動き出します。Glow65 では、リセット後の値は次のとおりです。

レジスタ	リセット後の値
A、X、Y	0
SP	`$FD`
P	`$24`(`%00100100`)。I フラグが 1、ビット 5 が 1
PC	アドレス `$FFFC` と `$FFFD` にある 2 バイト(下位が先)

PC が `$FFFC` と `$FFFD` から決まる仕組みは、3.7 で説明します。実機の 6502 では、リセット後のレジスタの内部状態は決まっておらず、プログラムが初期化する必要があります[1]。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

パネルには、PC、SP、A、X、Y と、ステータスの 8 個のフラグの LED があります。ステータスの LED は、左から `N`、`V`、`-`、`B`、`D`、`I`、`Z`、`C`(ビット 7 から 0)の順に並んでいます。LED は、値の 1 のビットが点灯し、0 のビットは消灯します。

リセット直後の状態は、次の手順で見られます。

1. **POWER** を ON にする。
2. **RUN/STOP** を STOP にして、CPU を止める。
3. **RESET** を押す。

リセットした CPU は、止まったまま、次の状態になります。

表示	LED	読み取れる値
PC	15、14、13、12、8、5、3、1 が点灯	`$F12A`
SP	7、6、5、4、3、2、0 が点灯	`$FD`
A、X、Y	すべて消灯	0
STATUS P	`-` と `I` が点灯	`$24`
ADDRESS BUS	A1 以外が点灯	`$FFFD`
DATA BUS	D7、D6、D5、D4、D0 が点灯	`$F1`
CYCLES	7	
INSTRUCTION AT PC	`F12A 78 SEI`	

最後に読んだのは `$FFFD` で、その値 `$F1` が PC の上位バイトになりました。READ と HALT の LED が点灯し、RUN の LED は消灯しています。PC の `$F12A` は、`$FFFC` の `2A` と `$FFFD` の `F1` から決まった値です(3.7)。



図 3-1 リセット直後のパネル

3.3 バス

CPU とメモリの間は、複数の線でつながっています。この線の束を**バス**と呼びます。

(c) 2026 masanorisakai / MIT License

バス・信号	本数	役割
アドレスバス	16 本(A0 〜 A15)	CPU が、どの番地を読み書きするかを伝える
データバス	8 本(D0 〜 D7)	読み書きするバイトが流れる
R/W	1 本	読み出し(Read)なら 1、書き込み(Write)なら 0
SYNC	1 本	命令の最初の 1 バイト(オペコード)を読んでいるとき 1

アドレスバスの 16 本は、第2章の 16 ビットのアドレスに対応します。データバスの 8 本は、1 バイトの 8 ビットに対応します[1]。

R/W は、CPU がメモリを**読む**のか**書く**のかを示します。メモリを読むときは、CPU がアドレスを出し、メモリがデータバスにバイトを出します。書くときは、CPU がアドレスとバイトの両方を出します。

SYNC は、いま読んでいるバイトが、命令の最初のバイトであることを示します。命令の途中のバイトを読むとき、また書き込みのときは、SYNC は出ません[1]。

CPU は、1 回のクロック(サイクル)で、バスに 1 回アクセスします。読むか書くかのどちらかです[1]。

> **Glow65 で確かめられること**
>
> パネルには、ADDRESS BUS の LED が A0 から A15 の 16 個、DATA BUS の LED が D0 から D7 の 8 個あります。READ、WRITE、SYNC の LED もあります。READ はメモリを読んでいるとき、WRITE は書いているとき、SYNC はオペコードを読んでいるときに光ります。1 kHz 以下の SPEED では、バスアクセスが 1 つずつ表示されます。

3.4 フェッチと実行

CPU は、同じ手順を繰り返して、プログラムを実行します。

1. **フェッチ(取り出し)**: PC の番地から、命令の最初の 1 バイト(**オペコード**)を読む。PC を 1 進める。
2. **オペランドの取り出し**: オペコードが、さらにバイトを必要とするなら、続けて読む。読むたびに PC を 1 進める。オペランドとは、命令が使う値や、番地を表すバイトのことです。
3. **実行**: 命令の動作を行う。レジスタを変える、メモリに書く、PC を別の場所にする、などです。

オペコードを読んだ時点で、CPU はその命令に必要なバイト数と動作を知ります。次は、チュートリアル第1章の 5 行のプログラムを実行したときの、バスアクセスの表です。

```
0300  A9 00        LDA #$00
0302  8D F0 03     STA $03F0
0305  18           CLC
0306  69 01        ADC #$01
0308  4C 02 03     JMP $0302
```

(c) 2026 masanorisakai / MIT License

#	アドレス	データ	読み/書き	SYNC	命令
1	`$0300`	`A9`	読み	○	`LDA #$00`
2	`$0301`	`00`	読み		
3	`$0302`	`8D`	読み	○	`STA $03F0`
4	`$0303`	`F0`	読み		
5	`$0304`	`03`	読み		
6	`$03F0`	`00`	書き		
7	`$0305`	`18`	読み	○	`CLC`
8	`$0306`	`69`	読み	○	`ADC #$01`
9	`$0307`	`01`	読み		
10	`$0308`	`4C`	読み	○	`JMP $0302`
11	`$0309`	`02`	読み		
12	`$030A`	`03`	読み		

SYNC が付くのは、命令の最初のバイトを読むときだけです。`STA $03F0` は、オペコードと 2 バイトのアドレスで 3 回読み、そのあとの 1 回で書きます。

サイクルとクロック

CPU の動きは、**クロック**という一定間隔の信号で進みます。クロックの 1 周期が 1 **サイクル**です。クロックが 1 MHz なら、1 秒に 100 万サイクルです。1 サイクルは 1 マイクロ秒です。

命令の実行にかかるサイクル数は、命令によって違います。MOS 社の資料では、2 サイクルから 7 サイクルの命令があります[1]。`LDA #$00` は 2 サイクル、`STA $03F0` は 4 サイクルです。

実機の 6502 は、`CLC` のような 1 バイトの命令でも 2 サイクルかかり、次のバイトをもう 1 回読みます。これは結果に使われません[1]。Glow65 は、この余分な読み出しを再現しません。表で `CLC` のアクセスが 1 回なのはそのためです。サイクルのカウンタは、実機と同じ 2 だけ進みます。

> **コラム: SYNC を使ったシングルステップ**
>
> MOS 社の資料には、SYNC 信号の使い方が書かれています。SYNC が 1 になるのは、オペコードを読むサイクルです。そのとき、CPU の RDY という端子を 0 にすると、CPU はその状態で止まります。RDY を 1 に戻すと再開します。この仕組みで、命令を 1 つずつ実行するシングルステップを作れます[1]。Glow65 の STEP スイッチも、命令やバスアクセスを 1 つずつ進めるための機能です。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

パネルで、サイクル単位の動きを追えます。3.2 の手順でリセットしたあと、**STEP MODE** を CYCLE にして、**STEP** を 1 回ずつ押します。CPU は、モニタの先頭のプログラム(`SEI`、`CLD`、`LDX #$FF`、`TXS`)を実行します。

STEP	ADDRESS BUS	DATA BUS	SYNC	その STEP で起きたこと
押す前	`$FFFD`	`$F1`		リセットの読み出し(3.2)
1 回目	`$F12A`	`$78`	点灯	`SEI` のオペコードを読んだ。1 回の読み出しで終わる命令なので、PC は `$F12B` に進み、INSTRUCTION AT PC は `CLD` になる
2 回目	`$F12B`	`$D8`	点灯	`CLD` のオペコードを読んだ。PC は `$F12C`
3 回目	`$F12C`	`$A2`	点灯	`LDX #$FF` のオペコードを読んだ
4 回目	`$F12D`	`$FF`	消灯	`LDX` のオペランドを読んだ
5 回目	`$F12D`	`$FF`	消灯	バスは変わらず、`LDX` の結果が表示に反映される

3 回目と 4 回目の間、PC は `$F12C` のままで、X は 0 のままです。2 バイトの命令では、**命令を読み終えるまで、PC、レジスタ、フラグは、実行前の値を示します。**5 回目の STEP で、X が `$FF`、N フラグが点灯、PC が `$F12E`(INSTRUCTION AT PC は `TXS`)になります。

CYCLES は、命令の最初のバスアクセスで、その命令のサイクル数だけ進みます。`SEI` の STEP で 7 から 9 になり、`CLD` で 11、`LDX #$FF` の 3 回目で 13 になります。

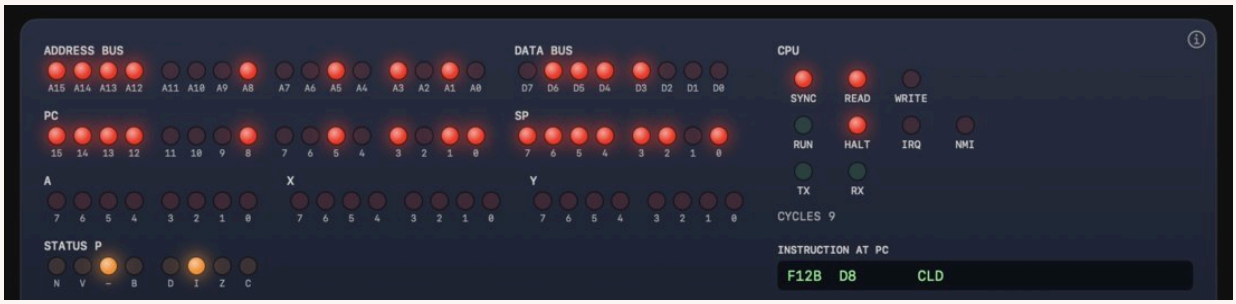


図 3-2 STEP 1 回目。`SEI` のオペコードを読んだところ



図 3-3 STEP 3 回目。`LDX #$FF` のオペコードを読んだところ。PC は `$F12C`、X は 0 のまま

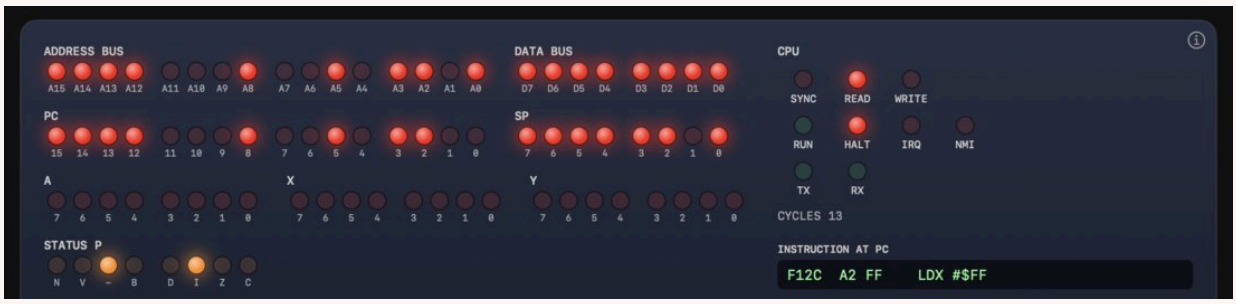


図 3-4 STEP 4 回目。`LDX` のオペランドを読んだところ。SYNC は消灯し、X はまだ 0



図 3-5 STEP 5 回目。バスは変わらず、X が `$FF`、N が点灯、PC が `$F12E` になる

SPEED を 10 Hz にして RUN にすると、同じ動きが連続して流れます。アドレスバスの下位の LED は、PC が進むたびに変わります。

STEP MODE を INSTR にすると、STEP を 1 回押すごとに 1 命令進みます。バスには、その命令の最後のバスアクセスが残ります。手順の詳細は、チュートリアル第1章にあります。

3.5 命令の構成

命令は、1 バイトのオペコードと、0 から 2 バイトのオペランドでできています。命令の長さは、1 バイトから 3 バイトです。

長さ	構成	例
1 バイト	オペコードだけ	`AA`（TAX）
2 バイト	オペコード + 1 バイト	`A9 00`（LDA #$00）

(c) 2026 masanorisakai / MIT License

長さ	構成	例
3 バイト	オペコード + 2 バイト(下位が先)	`8D F0 03`(`STA $03F0`)

長さは、オペコードで決まります。同じ `LDA` でも、`A9` は 2 バイトで、`AD` は 3 バイトです。オペランドの意味(値そのものか、番地か)は、**アドレッシングモード**で決まります。第4章で詳しく扱います。

3 バイトの命令で、番地は、第2章のリトルエンディアンで置きます。`8D F0 03` の `F0 03` は、番地 `$03F0` です。

> **Glow65 で確かめられること**
>
> モニタの `E` でバイトを書き、`L` で命令として読みます。
>
> ```
> *E 0300 A9 00 8D F0 03
> *L 0300 0304
> 0300  A9 00     LDA   #$00
> 0302  8D F0 03  STA   $03F0
> ```
>
> `L 0300 0304` は、`$0300` から `$0304` までの範囲を逆アセンブルします。`LDA` が 2 バイト、`STA` が 3 バイトであることが、アドレスの増え方でわかります。
>
> 同じバイト列でも、先頭の位置を変えると、別の命令になります。
>
> ```
> *E 0300 AD 34 12 A9 10 8D 00 04 AA
> *L 0300 0308
> 0300  AD 34 12  LDA   $1234
> 0303  A9 10     LDA   #$10
> 0305  8D 00 04  STA   $0400
> 0308  AA        TAX
> ```
>
> `AD 34 12` は 3 バイトの `LDA $1234` です。`34 12` は、リトルエンディアンで `$1234` になります。

3.6 命令の実行例

次のプログラムを、バスの動きとレジスタ・フラグの変化で追います。

```
0300  A9 80        LDA #$80
0302  AA           TAX
0303  8D F0 03     STA $03F0
0306  8E F1 03     STX $03F1
0309  60           RTS
```

(c) 2026 masanorisakai / MIT License

命令	バスアクセス	A	X	N	Z	メモリの変化
`LDA #$80`	読み 2 回(オペコード、値)	`$80`	変わらない	1	0	なし
`TAX`	読み 1 回(オペコード)	`$80`	`$80`	1	0	なし
`STA $03F0`	読み 3 回、書き 1 回	`$80`	`$80`	1	0	`$03F0` が `$80`
`STX $03F1`	読み 3 回、書き 1 回	`$80`	`$80`	1	0	`$03F1` が `$80`

- `LDA #$80` は、A に `$80` を入れます。`$80` は最上位ビットが 1 なので、N が 1 になります。値は 0 ではないので、Z は 0 です。
- `TAX` は、A の値を X にコピーします。A はそのままです。X に入った `$80` に従って、N と Z が決まります。
- `STA` と `STX` は、レジスタの値をメモリに書きます。レジスタとフラグは変わりません。
- `RTS` は、`G` で呼んだモニタに戻る命令です。第6章で扱います。

値が 0 の場合は、Z が 1 に、N が 0 になります。`LDA #$00` のあとは、Z だけが 1 です。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること(パネル)

3.2 の手順でリセットし、STEP MODE を INSTR にして STEP を押すと、1 命令ずつ進みます。モニタの先頭のプログラムで、レジスタとフラグの変化を見られます。

STEP	実行した命令	レジスタ・フラグの変化	ADDRESS BUS / DATA BUS	次の命令
1 回目	`SEI`	変化なし(I フラグはもともと 1)	`$F12A` / `$78` (SYNC 点灯)	`CLD` (`$F12B`)
2 回目	`CLD`	変化なし(D フラグはもともと 0)	`$F12B` / `$D8` (SYNC 点灯)	`LDX #$FF` (`$F12C`)
3 回目	`LDX #$FF`	X が `$FF` になり、N が点灯する	`$F12D` / `$FF` (`LDX` のオペランド)	`TXS` (`$F12E`)
4 回目	`TXS`	SP が `$FF` になる(SP の LED がすべて点灯)。N は点灯のまま	`$F12E` / `$9A` (SYNC 点灯)	`LDA #$03` (`$F12F`)
5 回目	`LDA #$03`	A が `$03` になる(ビット 1、0 が点灯)。N が消灯	`$F130` / `$03`	`STA $A000` (`$F131`)

`LDX #$FF` の `$FF` は、最上位ビットが 1 なので、N が点灯します。`TXS` は、X の値を SP にコピーするだけで、フラグは変えません。`LDA #$03` は、最上位ビットが 0 で、0 でもない値なので、N も Z も点灯しません。

STEP の回数は、3.2 の手順でリセットした直後から数えています。



図 3-6 3 回目の STEP のあと。`LDX #$FF` が実行され、X が `$FF`、N が点灯

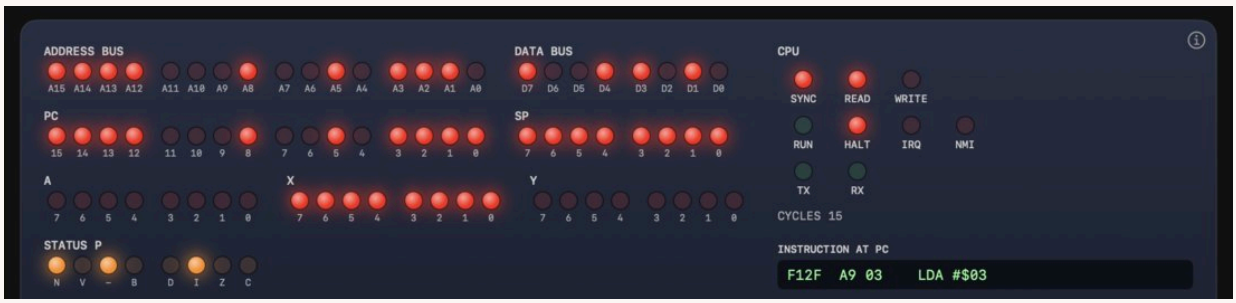


図 3-7 4 回目の STEP のあと。`TXS` が実行され、SP の LED がすべて点灯

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること(モニタ)

モニタには、レジスタを表示するコマンドがありません。プログラムが結果をメモリに書き、`D` で読みます。

まず、上の 5 命令を実行します。

```
*E 0300 A9 80 AA 8D F0 03 8E F1 03 60
*L 0300 0309
0300  A9 80     LDA   #$80
0302  AA        TAX
0303  8D F0 03  STA   $03F0
0306  8E F1 03  STX   $03F1
0309  60        RTS
*G 0300
*D 03F0 03F1
03F0: 80 80 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`$03F0` と `$03F1` が、どちらも `80` です。`D` は 16 バイト単位で表示するため、続く `24 F4 24 F4` は、モニタの割り込みベクタです。第6章で説明します。

次に、フラグを調べます。`PHP` は P をスタックに積み、`PLA` は積んだ値を A に取り出す命令です。`PHP` は、ビット 4(B)とビット 5 を 1 にして積みます。この 2 つを除いた、N と Z を見ます。

```
*E 0300 A9 80 08 68 8D F0 03 A9 00 08 68 8D F1 03 60
*G 0300
*D 03F0 03F1
03F0: B4 36 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`LDA #$80` のあとの P は `$B4`(`%10110100`)で、N が 1 です。`LDA #$00` のあとの P は `$36`(`%00110110`)で、Z が 1 です。どちらも、I フラグ(ビット 2)は 1 です。

3.7 リセットと割り込みのベクタ

CPU は、リセットや割り込みのとき、メモリ上の決まった場所から、飛び先のアドレスを読みます。この場所が**ベクタ**です。アドレス空間の最後の 6 バイトにあります[1]。

アドレス	用途
`$FFFA` `$FFFB`	NMI(マスクできない割り込み)
`$FFFC` `$FFFD`	リセット
`$FFFE` `$FFFF`	IRQ と BRK

(c) 2026 masanorisakai / MIT License

各ベクタは、2 バイトで、下位が先です。リセットのあと、CPU は `$FFFC` と `$FFFD` を読み、その 16 ビットのアドレスを PC に入れます。プログラムは、そのアドレスから始まります。割り込みの詳細は、第6章で説明します。

Glow65 では、ベクタはモニタの ROM にあります(第2章の `D FFF0 FFFF` の例を参照)。

3.8 練習問題

答えと解き方は、次の節にあります。

問1 PC と A のレジスタの幅は、それぞれ何ビットですか。

問2 Glow65 でリセットした直後の SP と P の値を、16 進数で答えてください。P で 1 になっているフラグは何ですか。

問3 リセットのあと、CPU が最初に実行する命令のアドレスは、どこから決まりますか。Glow65 で `$FFFC` と `$FFFD` に `2A F1` が入っているとき、PC はいくつになりますか。

問4 `STA $03F0`(`8D F0 03`)を 1 回実行するときの、バスアクセスの回数を、読みと書きに分けて答えてください。SYNC が 1 になるのは何回ですか。

問5 バイト列 `A9 10 8D 00 04 AA` を、命令に分けてください。それぞれの命令の長さと、全体のバイト数も答えてください。

問6 `LDA #$00` を実行したあとに 1 になるのは、N と Z のどちらですか。`LDA #$FF` のあとは、どちらですか。

問7 Glow65 で `E 0300 A2 05 8E F0 03 60` を入力し、`G 0300` で実行して、`D 03F0 03F0` を入力すると、`$03F0` の値はいくつになりますか。

問8 3.6 の方法で P を読んだところ、`$B4` でした。1 になっているフラグを、すべて答えてください。

問9 `A9 7F 08 68 8D F0 03 60` を実行したあとの `$03F0` は、いくつですか。

3.9 練習問題の答えと解き方

問1 PC は 16 ビット、A は 8 ビット。

- PC は、16 ビットのアドレスを持ちます。A は、1 バイトのデータを扱います。

問2 SP は `$FD`、P は `$24`。1 になっているのは I フラグ(ビット 2)とビット 5。

- `$24` は `%00100100` です。ビット 5 は未使用で常に 1 です。ビット 2 が I です。

問3 `$FFFC` と `$FFFD` から決まる。PC は `$F12A`。

- 下位バイトが先なので、`$FFFC` の `2A` が下位、`$FFFD` の `F1` が上位です。

(c) 2026 masanorisakai / MIT License

問4 読み 3 回、書き 1 回。SYNC は 1 回。

- 読み: オペコード `8D`、下位 `F0`、上位 `03`。書き: `$03F0` への 1 回。SYNC はオペコードを読むときだけです。

問5 `A9 10` = `LDA #$10`(2 バイト)、`8D 00 04` = `STA $0400`(3 バイト)、`AA` = `TAX`(1 バイト)。合計 6 バイト。

- `A9` の後ろに値 1 バイト、`8D` の後ろにアドレス 2 バイト、`AA` はオペランドなし。

問6 `LDA #$00` のあとは Z、`LDA #$FF` のあとは N。

- `$00` は 0 なので Z が 1。`$FF` は最上位ビットが 1 なので N が 1。

問7 `$05`。

- `A2 05` は `LDX #$05`、`8E F0 03` は `STX $03F0`、`60` は `RTS`。X の 5 が `$03F0` に書かれます。

問8 N、I。このほかに、PHP が積むときに 1 にするビット 4(B)とビット 5。

- `$B4` は `%10110100`。ビット 7(N)、5、4(B)、2(I)が 1 です。

問9 `$34`。

- `LDA #$7F` は、最上位ビットが 0 で、0 でもない値なので、N と Z はどちらも 0 です。I(ビット 2)は 1、ビット 4 と 5 は PHP が 1 にします。`%00110100` で `$34` です。

出典

1. MOS Technology, "MCS6500 Microcomputer Family Hardware Manual", 1976年1月版: https://www.commodore.ca/wp-content/uploads/2018/11/kim1_hardware.txt

(c) 2026 masanorisakai / MIT License

第4章 アドレッシングモード

命令のオペランドは、「どのメモリの値を対象にするか」を決めます。その決め方を**アドレッシングモード**と呼びます。6502 には 13 のモードがあります。

この章では、モードごとに、書き方、バイト数、実効アドレスの求め方を、図で説明します。

4.1 アドレッシングモードとは

同じ `LDA` (A レジスタに値を読み込む命令)でも、モードが違えば、動作もバイト数も違います。第3章の 3.5 で見たように、命令の長さはオペコードで決まります。モードは、オペコードに含まれています。

書き方	モード	オペコード	バイト数
`LDA #$10`	イミディエイト	`A9`	2
`LDA $10`	ゼロページ	`A5`	2
`LDA $10,X`	ゼロページ X	`B5`	2
`LDA $0400`	アブソリュート	`AD`	3
`LDA $0410,X`	アブソリュート X	`BD`	3
`LDA $0420,Y`	アブソリュート Y	`B9`	3
`LDA ($20,X)`	インデックスト・インダイレクト	`A1`	2
`LDA ($30),Y`	インダイレクト・インデックスト	`B1`	2

オペランドが指すメモリの番地を、**実効アドレス**と呼びます。CPU は、実効アドレスを求めてから、その番地を読み書きします。アドレッシングモードの違いは、実効アドレスの求め方の違いです。

図の見方

この章の図は、すべて同じ部品でできています。

- 上の段: 命令のバイト列。左端がオペコード、青い枠がオペランドです。
- 左: レジスタ。橙の枠が、結果を受け取るレジスタです。
- 中央の白い枠: 実効アドレスを求める計算です。
- 右: メモリ。アドレスと中身が並び、青い枠が対象のバイトです。
- 矢印の番号: CPU が行う手順の順番です。橙の矢印は、結果がレジスタに入る動きです。

実行例の形

この章の実行例は、同じ形です。命令の結果を `STA $03F0` でメモリに書き、`RTS` でモニタに戻ります。`G 0300` で実行し、`D 03F0 03F0` で `$03F0` の値を読みます。`D` は 16 バイト単位で表示するので、行の先頭のバイトが結果です。続く `24 F4 24 F4` は、モニタの割り込みベクタです。

(c) 2026 masanorisakai / MIT License

プログラムは `$0300` に置きます。データは、`E` で RAM に書きます。

Glow65 で確かめられること

次のバイト列を `$0300` に書き、`L` で読むと、モードの書き方がそのまま表示されます。

```
*E 0300 E8 0A A9 10 A5 10 B5 10 B6 10 AD 00 04 BD 10 04 B9 20 04 6C 30 04 A1 20 B1
30 D0 FE
*L 0300 0319
0300  E8        INX
0301  0A        ASL   A
0302  A9 10     LDA   #$10
0304  A5 10     LDA   $10
0306  B5 10     LDA   $10,X
0308  B6 10     LDX   $10,Y
030A  AD 00 04  LDA   $0400
030D  BD 10 04  LDA   $0410,X
0310  B9 20 04  LDA   $0420,Y
0313  6C 30 04  JMP   ($0430)
0316  A1 20     LDA   ($20,X)
0318  B1 30     LDA   ($30),Y
```

4.2 オペランドを持たないモード

インプライド(Implied)

オペランドがなく、オペコードだけの命令です。命令の名前が、対象(レジスタやフラグ)を含んでいます。`INX`(X に 1 を足す)、`TAX`、`CLC` などです。長さは 1 バイトです。



① オペコード $E8(INX)を読み、X の値に 1 を足す。X は $05 から $06 になる。

(c) 2026 masanorisakai / MIT License

図 4-1 インプライド(`INX`)。メモリの内容は使わない。

Glow65 で確かめられること

```
*E 0300 A2 05 E8 8E F0 03 60
*G 0300
*D 03F0 03F0
03F0: 06 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`LDX #$05` で X が 5 になり、`INX` で 6 になります。

アキュムレータ(Accumulator)

操作の対象が、A レジスタそのものです。`ASL A`(左に 1 ビットずらす)、`LSR A`、`ROL A`、`ROR A` の 4 命令がこのモードを持ちます(65C02 では `INC A` と `DEC A` が加わります)。長さは 1 バイトです。

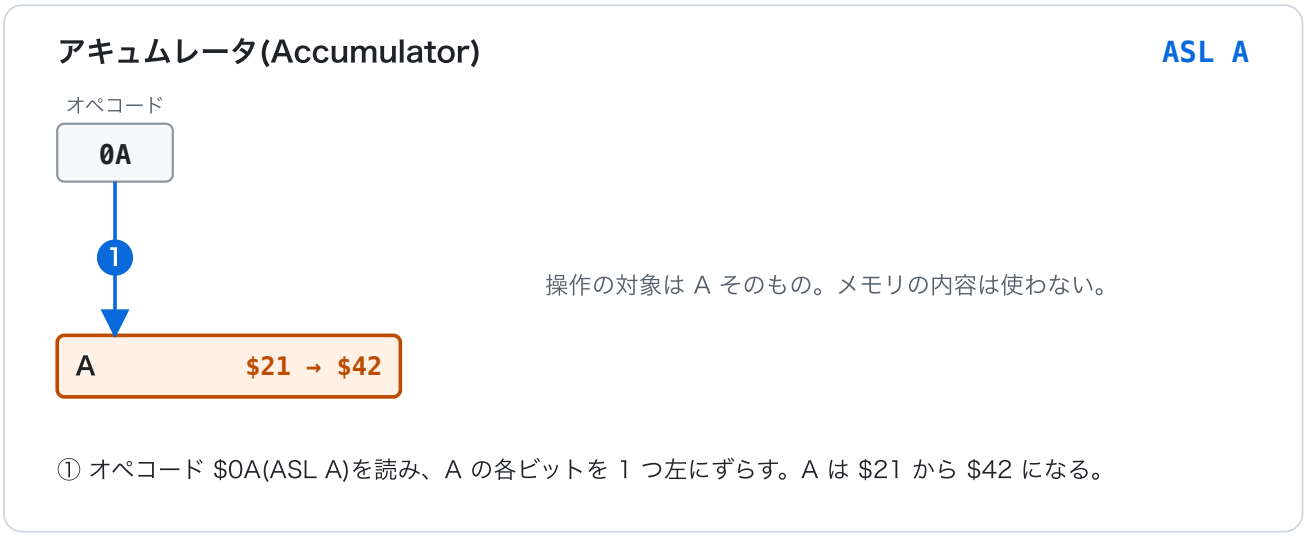


図 4-2 アキュムレータ(`ASL A`)。

Glow65 で確かめられること

```
*E 0300 A9 21 0A 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 42 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`$21`(`%00100001`)を左に 1 ビットずらすと、`$42`(`%01000010`)になります。

(c) 2026 masanorisakai / MIT License

4.3 イミディエイト(Immediate)

オペランドの値が、そのまま使われます。メモリの番地ではありません。書き方は、値の前に `#` を付けます。

項目	内容
書き方	`LDA #$10`
機械語	`A9 10`
バイト数	2
実効アドレス	なし。オペランドが値そのもの



図 4-3 イミディエイト(`LDA #$10`)。

`LDA #$10` は「A に値 `$10` を入れる」、`LDA $10` は「アドレス `$0010` の内容を A に入れる」です。`#` の有無で、意味が変わります。

Glow65 で確かめられること

```
*E 0300 A9 10 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 10 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

4.4 ゼロページ系のモード

ゼロページ(先頭の 256 バイト、`$0000-$00FF`)は、1 バイトのオペランドで指定できます。命令が短くなり、実行も速くなります。

(c) 2026 masanorisakai / MIT License

ゼロページ(Zero Page)

項目	内容
書き方	`LDA $10`
機械語	`A5 10`
バイト数	2
実効アドレス	`$0010`(上位バイトは `$00`)

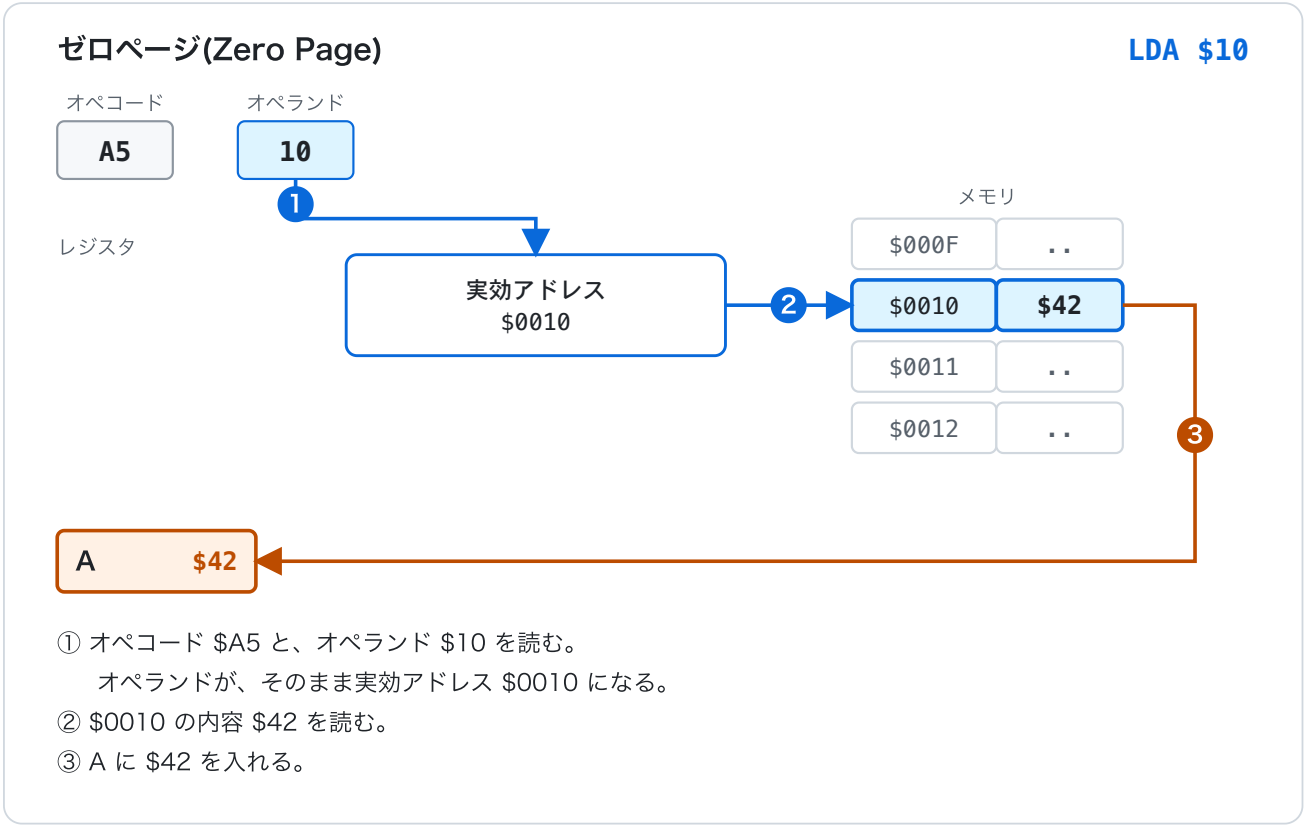


図 4-4 ゼロページ(`LDA $10`)。

Glow65 で確かめられること

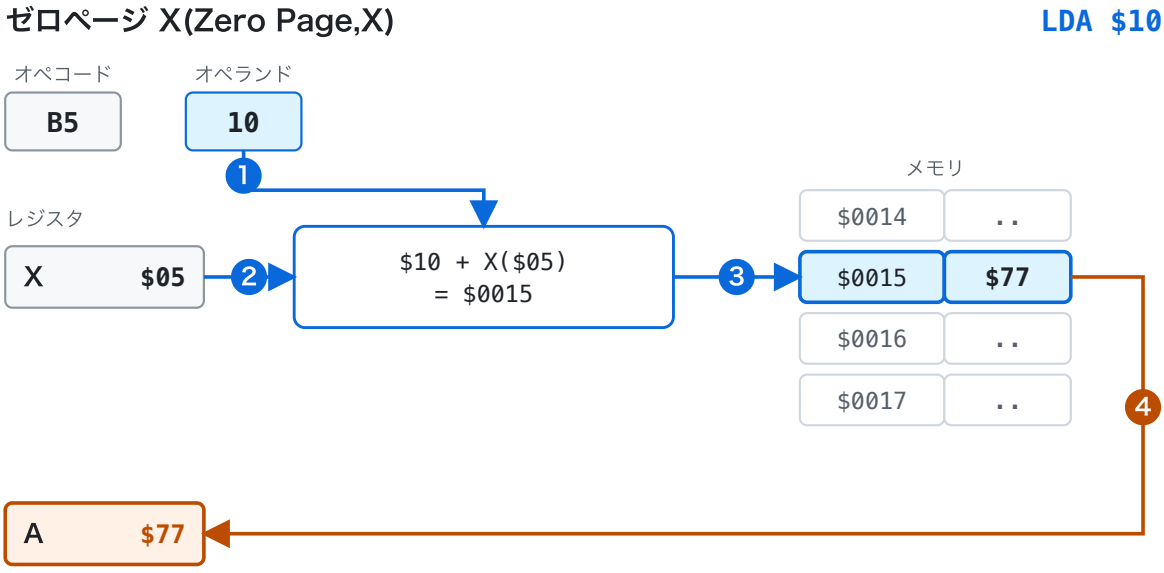
```
*E 0010 42
*E 0300 A5 10 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 42 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

ゼロページ X(Zero Page,X)

オペランドに X の値を足した番地を使います。配列のように、並んだデータを順に読むときに使います。

(c) 2026 masanorisakai / MIT License

項目	内容
書き方	`LDA $10,X`
機械語	`B5 10`
バイト数	2
実効アドレス	オペランド + X。結果は 256 で割った余り(`$FF` を超えたら `$00` に戻る)



① オペコード $B5 と、オペランド $10 を読む。
② X の値 $05 を足して、実効アドレス $0015 を得る。
　ゼロページの計算は 256 バイトで折り返す($FF の次は $00)。
③ $0015 の内容 $77 を読む。
④ A に $77 を入れる。

図 4-5 ゼロページ X(`LDA $10,X`)。X が `$05` のとき、`$0015` を読む。

計算の結果は、ゼロページの中に収まります。`LDA $FF,X` で X が `$02` なら、実効アドレスは `$0101` ではなく `$0001` です。足した結果が `$FF` を超えると、`$00` から数え直します[1]。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

```
*E 0015 77
*E 0300 A2 05 B5 10 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 77 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

折り返しは、次のように確かめられます。`LDA $FF,X` は `$0001` を、2 バイト目が `$00` のアブソリュート X の `LDA $00FF,X` は `$0101` を読みます。

```
*E 0001 7B
*E 0101 3D
*E 0300 A2 02 B5 FF 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 7B 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
*E 0300 A2 02 BD FF 00 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 3D 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

ゼロページ Y(Zero Page,Y)

ゼロページ X と同じで、足すレジスタが Y です。このモードを持つ命令は、`LDX` と `STX` だけです。

項目	内容
書き方	`LDX $10,Y`
機械語	`B6 10`
バイト数	2
実効アドレス	オペランド + Y。256 で折り返す

(c) 2026 masanorisakai / MIT License

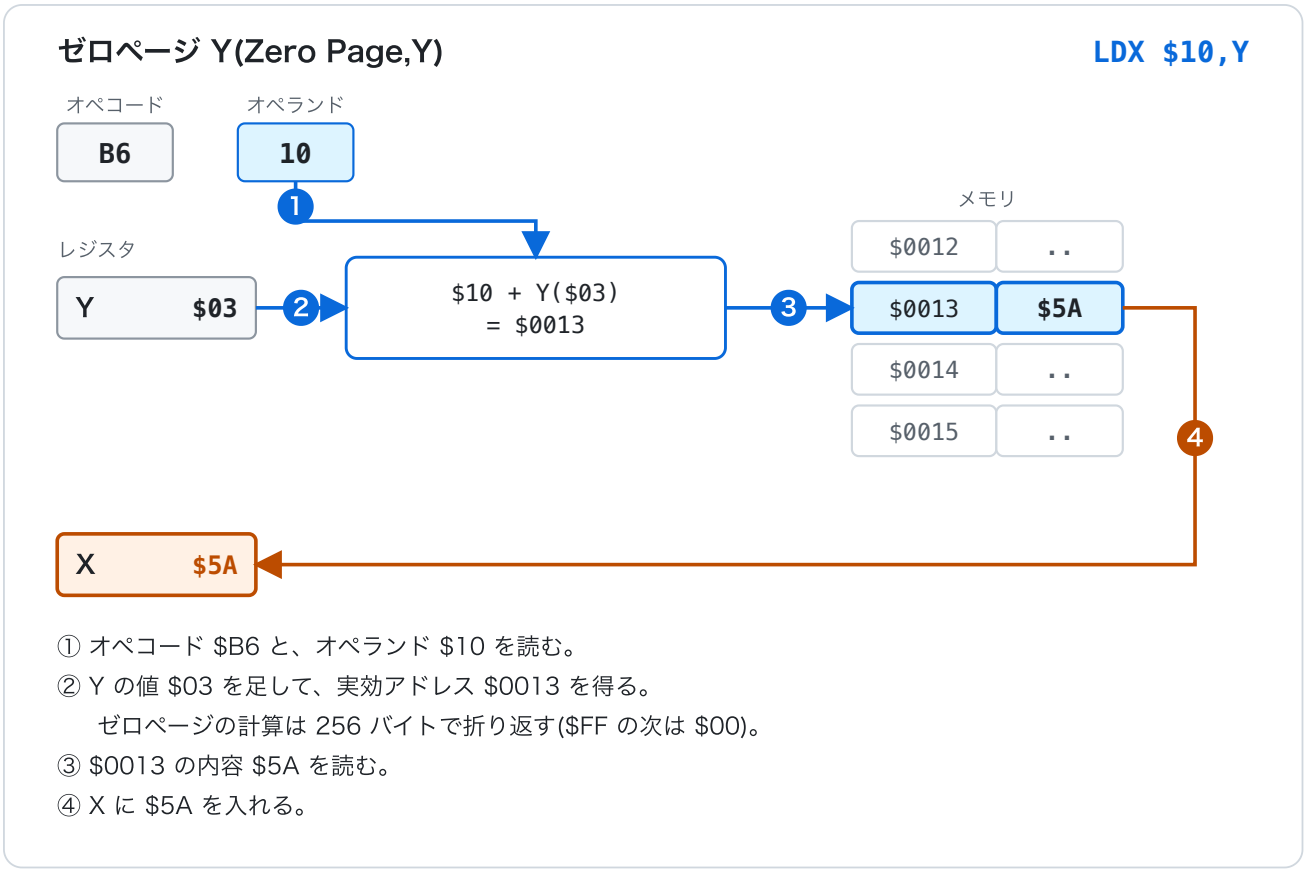


図 4-6 ゼロページ Y(`LDX $10,Y`)。結果は X に入る。

Glow65 で確かめられること

```
*E 0013 5A
*E 0300 A0 03 B6 10 8E F0 03 60
*G 0300
*D 03F0 03F0
03F0: 5A 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

4.5 アブソリュート系のモード

64KB のどの番地でも、2 バイトのオペランドで指定できます。オペランドは、リトルエンディアン(下位が先)です。

アブソリュート(Absolute)

項目	内容
書き方	`LDA $0400`
機械語	`AD 00 04`

(c) 2026 masanorisakai / MIT License

項目	内容
バイト数	3
実効アドレス	オペランドの 2 バイト(`$0400`)

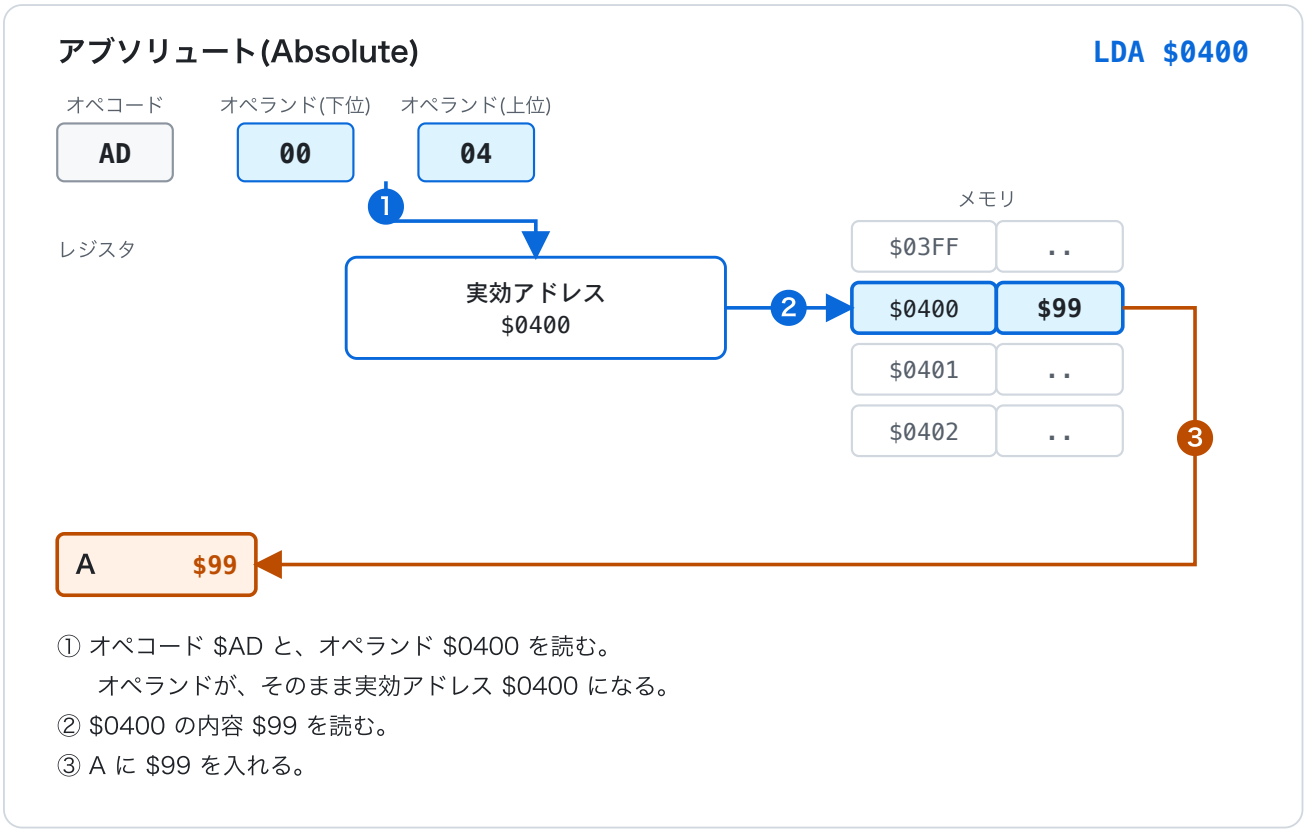


図 4-7 アブソリュート(`LDA $0400`)。

Glow65 で確かめられること

```
*E 0400 99
*E 0300 AD 00 04 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 99 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

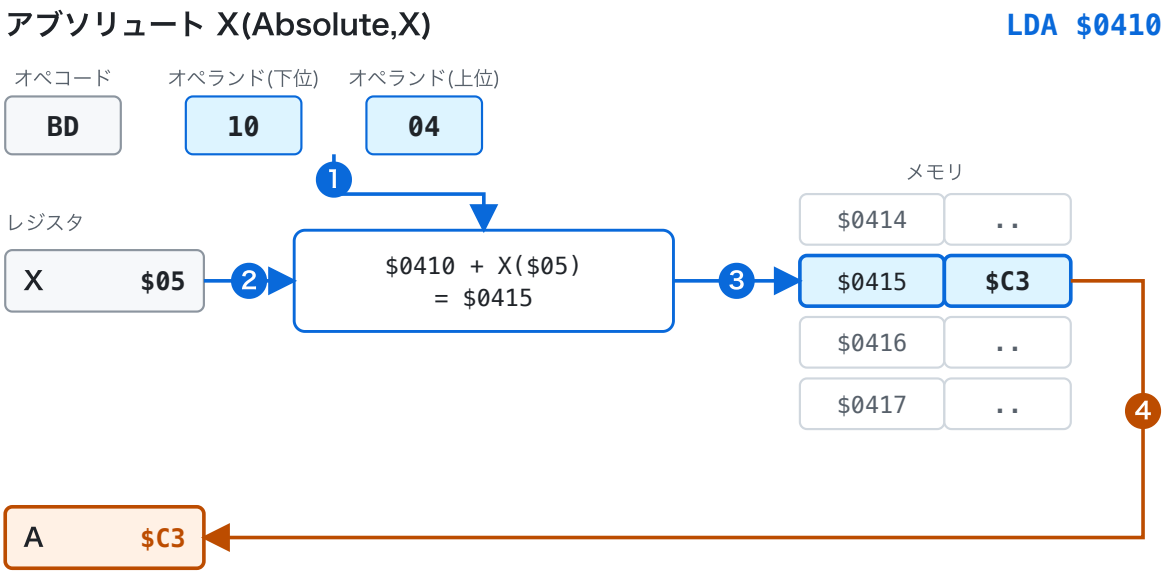
アブソリュート X(Absolute,X)

2 バイトのオペランドに、X の値を足した番地を使います。

項目	内容
書き方	`LDA $0410,X`
機械語	`BD 10 04`

(c) 2026 masanorisakai / MIT License

項目	内容
バイト数	3
実効アドレス	オペランド + X(16 ビットの足し算)



① オペコード $BD と、オペランド $0410 を読む。
② X の値 $05 を足して、実効アドレス $0415 を得る。
③ $0415 の内容 $C3 を読む。
④ A に $C3 を入れる。

図 4-8 アブソリュート X(`LDA $0410,X`)。

足した結果が、元の番地と別のページ(上位バイトが違う番地)になることを、**ページをまたぐ**と呼びます。`LDA $04FF,X` で X が `$01` なら、実効アドレスは `$0500` で、ページをまたぎます。実機の 6502 は、`LDA` などの読み出し命令でページをまたぐと、1 サイクル余分にかかります[1]。`STA` などの書き込み命令は、またがなくても、いつも長いほうのサイクル数です。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

```
*E 0415 C3
*E 0300 A2 05 BD 10 04 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: C3 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

ページをまたぐ場合も、読み出す値は同じように求まります。

```
*E 0500 5E
*E 0300 A2 01 BD FF 04 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 5E 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

アブソリュート Y(Absolute,Y)

アブソリュート X と同じで、足すレジスタが Y です。

項目	内容
書き方	`LDA $0420,Y`
機械語	`B9 20 04`
バイト数	3
実効アドレス	オペランド + Y

(c) 2026 masanorisakai / MIT License

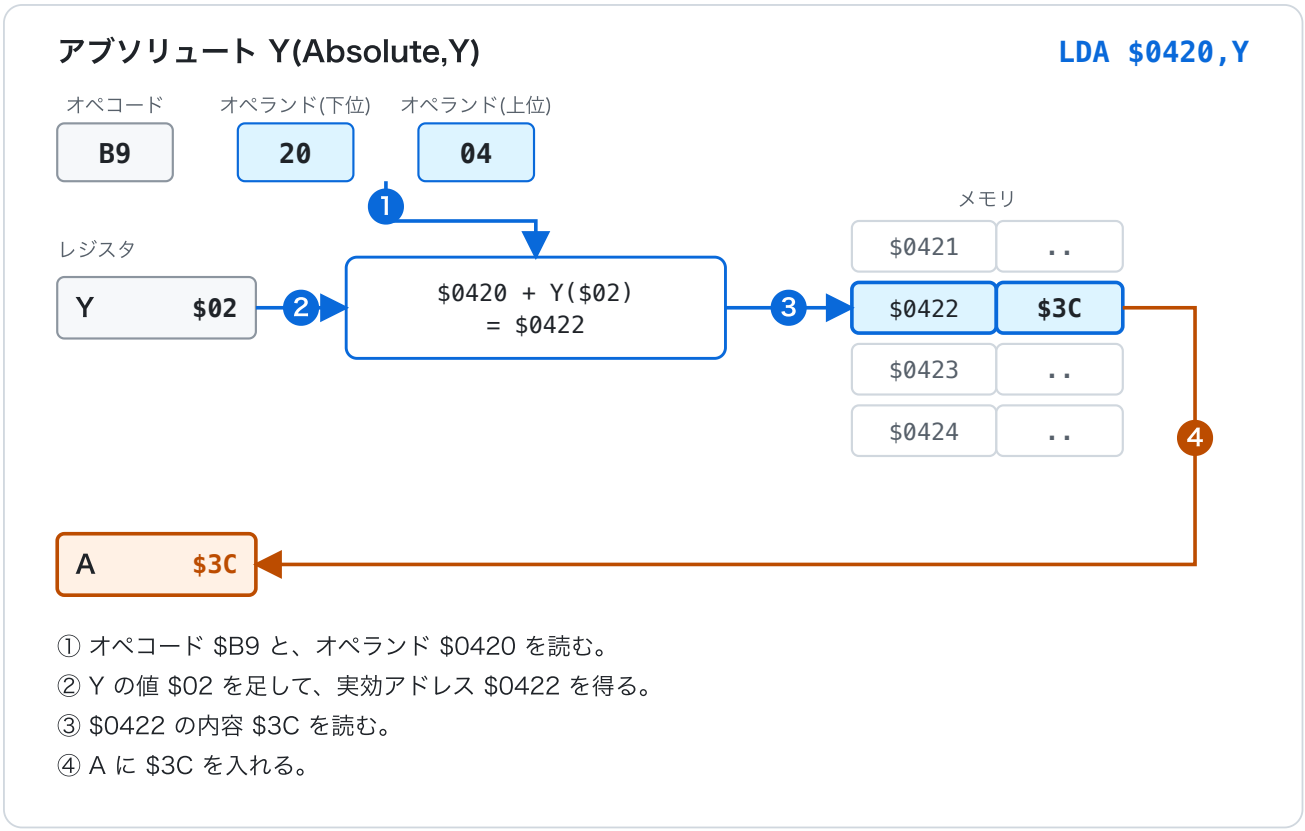


図 4-9 アブソリュート Y(`LDA $0420,Y`)。

Glow65 で確かめられること

```
*E 0422 3C
*E 0300 A0 02 B9 20 04 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 3C 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

実効アドレスは、バスに出る番地として、パネルで見られます。次のプログラムを使います。

```
*E 0410 00 00 00 00 00 C3
*E 0010 42
*E 0300 A2 05 BD 10 04 A5 10 A9 10 4C 00 03
*G 0300
```

`LDX #$05`、`LDA $0410,X`、`LDA $10`、`LDA #$10`、`JMP $0300` を繰り返します。**RUN/STOP** を STOP、**STEP MODE** を INSTR にして、STEP を 1 回ずつ押すと、命令ごとに、最後のバスアクセスが ADDRESS BUS と DATA BUS に残ります。

実行した命令	ADDRESS BUS	DATA BUS	A
`LDA $0410,X`（X = `$05`）	`$0415`	`$C3`	`$C3`
`LDA $10`	`$0010`	`$42`	`$42`
`LDA #$10`	`$0308`	`$10`	`$10`

アブソリュート X の `LDA $0410,X` は、オペランド `$0410` に X の `$05` を足した `$0415` を読みます(図 4-14)。ゼロページの `LDA $10` は、`$0010` を読みます(図 4-15)。イミディエイトの `LDA #$10` は、命令のすぐ後ろ(`$0308`)のバイトを読みます(図 4-16)。

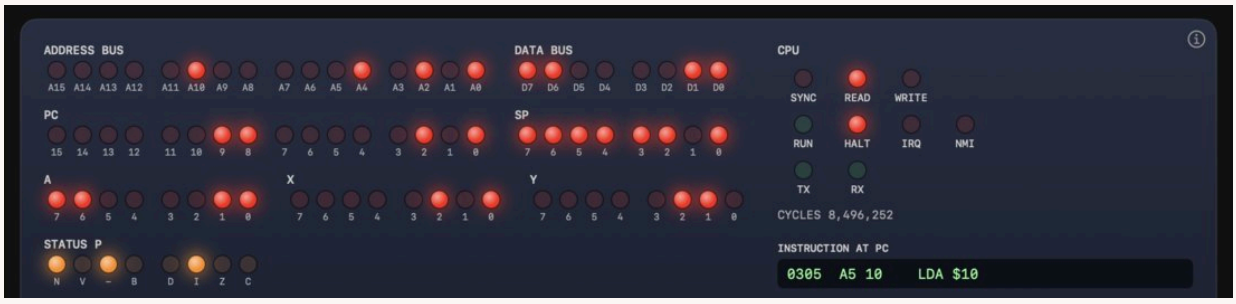


図 4-14 `LDA $0410,X` のあと。ADDRESS BUS に実効アドレス `$0415`、DATA BUS と A に `$C3`、X は `$05`



図 4-15 `LDA $10` のあと。ADDRESS BUS は `$0010`、DATA BUS と A は `$42`

(c) 2026 masanorisakai / MIT License



図 4-16 `LDA #$10` のあと。ADDRESS BUS は命令の直後の `$0308`、DATA BUS と A は `$10`

4.6 インダイレクト(Indirect)

オペランドの番地に収められた 2 バイトを、飛び先として使います。このモードを持つ命令は、`JMP` だけです。

項目	内容
書き方	`JMP ($0430)`
機械語	`6C 30 04`
バイト数	3
飛び先	`$0430` の 1 バイトが下位、`$0431` の 1 バイトが上位

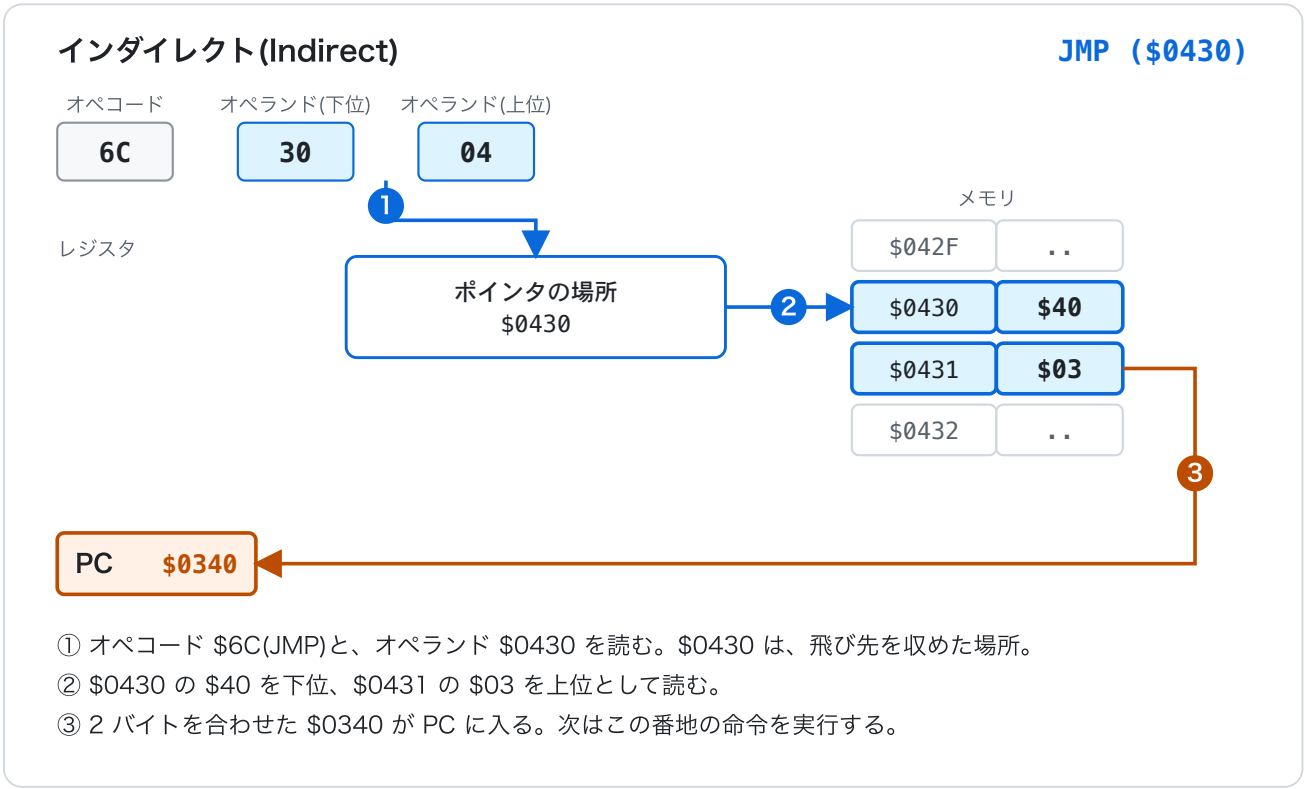


図 4-10 インダイレクト(`JMP ($0430)`)。結果は PC に入る。

(c) 2026 masanorisakai / MIT License

NMOS 6502 には、このモードの癖があります。ポインタが、ページの最後のバイト(`$xxFF`)にあるときです。上位バイトは、次のページの先頭ではなく、**同じページの先頭**から読まれます。`JMP ($04FF)` なら、下位を `$04FF` から、上位を `$0500` ではなく `$0400` から読みます。65C02 では、この癖が直っています。

Glow65 で確かめられること

```
*E 0430 40 03
*E 0340 A9 AB 8D F0 03 60
*E 0300 6C 30 04
*G 0300
*D 03F0 03F0
03F0: AB 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`$0340` から始まるプログラムが `$AB` を書きます。

ページの最後のバイトの癖は、次のように確かめられます。`$04FF` に `40`、`$0500` に `05`、`$0400` に `03` を置きます。上位バイトを `$0500` から読めば `$0540` に、`$0400` から読めば `$0340` に飛びます。

```
*E 04FF 40
*E 0500 05
*E 0400 03
*E 0340 A9 AB 8D F0 03 60
*E 0540 A9 CD 8D F0 03 60
*E 0300 6C FF 04
*G 0300
*D 03F0 03F0
03F0: AB 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

既定の NMOS 6502 では、`$0340` に飛んで `AB` が書かれます。`glow65-cli --cpu 65c02` で同じ操作をすると、`$0540` に飛んで `CD` が書かれます。

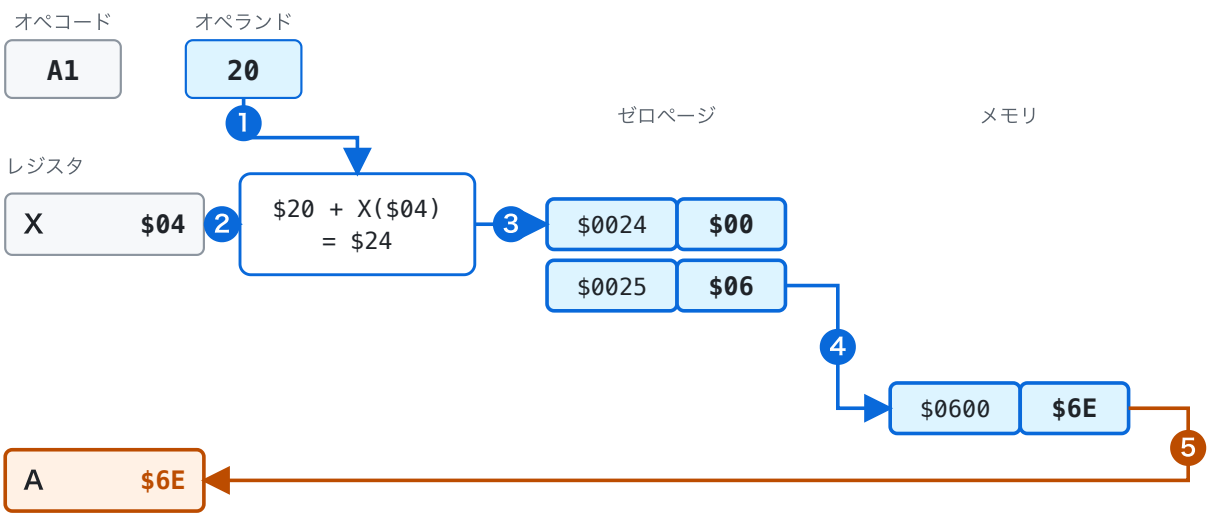
4.7 インデックスト・インダイレクト(Indexed Indirect)

ゼロページに、番地(2 バイト)を並べた表を置きます。命令は、表の中から X 番目の番地を取り出し、その番地の内容を使います。`(zp,X)` と書きます。

項目	内容
書き方	`LDA ($20,X)`
機械語	`A1 20`

(c) 2026 masanorisakai / MIT License

項目	内容
バイト数	2
実効アドレス	ゼロページの `$20 + X` にある 2 バイト(下位が先)。足した結果は 256 で折り返す



① オペコード $A1(LDA)と、オペランド $20 を読む。
② オペランドに X の値 $04 を足して、ゼロページの場所 $24 を得る(256 バイトで折り返す)。
③ $0024 の $00 を下位、$0025 の $06 を上位として読む。
④ 2 バイトを合わせた $0600 が実効アドレスになる。
⑤ $0600 の内容 $6E を読んで、A に入れる。

図 4-11 インデックスト・インダイレクト(`LDA ($20,X)`)。ゼロページの `$0024` と `$0025` に、番地 `$0600` が収められている。

X は、表の何番目かを選びます。番地は 2 バイトなので、X を 2 ずつ増やして使います。

Glow65 で確かめられること

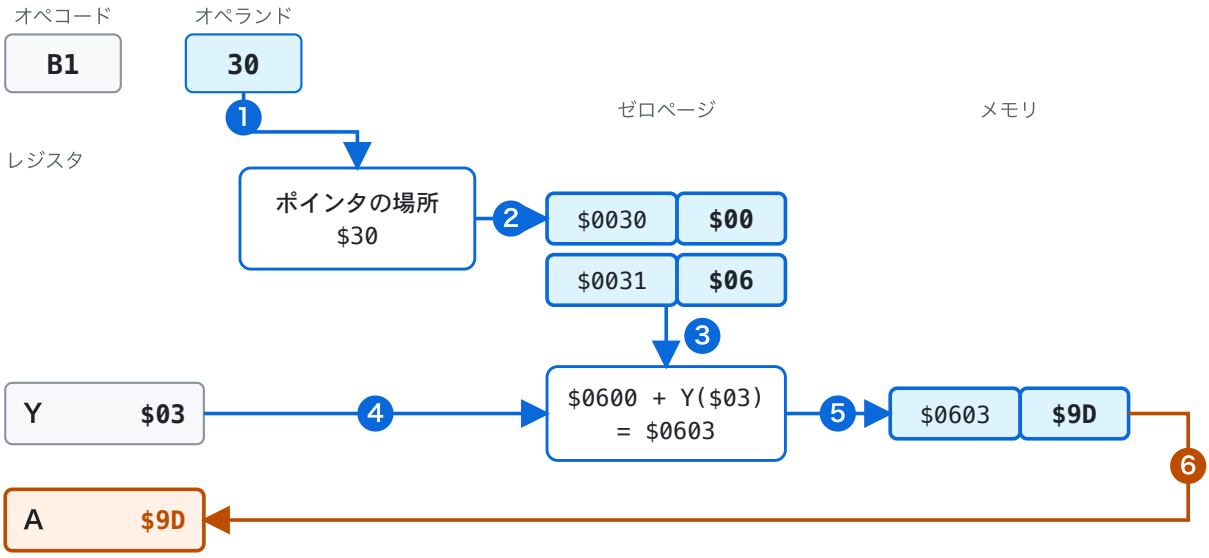
```
*E 0024 00 06
*E 0600 6E
*E 0300 A2 04 A1 20 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 6E 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

(c) 2026 masanorisakai / MIT License

4.8 インダイレクト・インデックスト(Indirect Indexed)

ゼロページにある 2 バイトの番地を基準にして、Y の値を足します。`(zp),Y` と書きます。メモリ上の配列の先頭番地をゼロページに置き、Y で要素を選ぶ使い方ができます。

項目	内容
書き方	`LDA ($30),Y`
機械語	`B1 30`
バイト数	2
実効アドレス	ゼロページの `$30` にある 2 バイト(下位が先)を基準の番地として、Y を足す



① オペコード $B1(LDA)と、オペランド $30 を読む。
② $0030 の $00 を下位、$0031 の $06 を上位として読み、基準の番地 $0600 を得る。
③④ 基準の番地に Y の値 $03 を足して、実効アドレス $0603 を得る。
⑤⑥ $0603 の内容 $9D を読んで、A に入れる。

図 4-12 インダイレクト・インデックスト(`LDA ($30),Y`)。基準の番地 `$0600` に Y の `$03` を足す。

基準の番地に Y を足した結果が別のページになると、実機の 6502 は 1 サイクル余分にかかります[1]。

(c) 2026 masanorisakai / MIT License

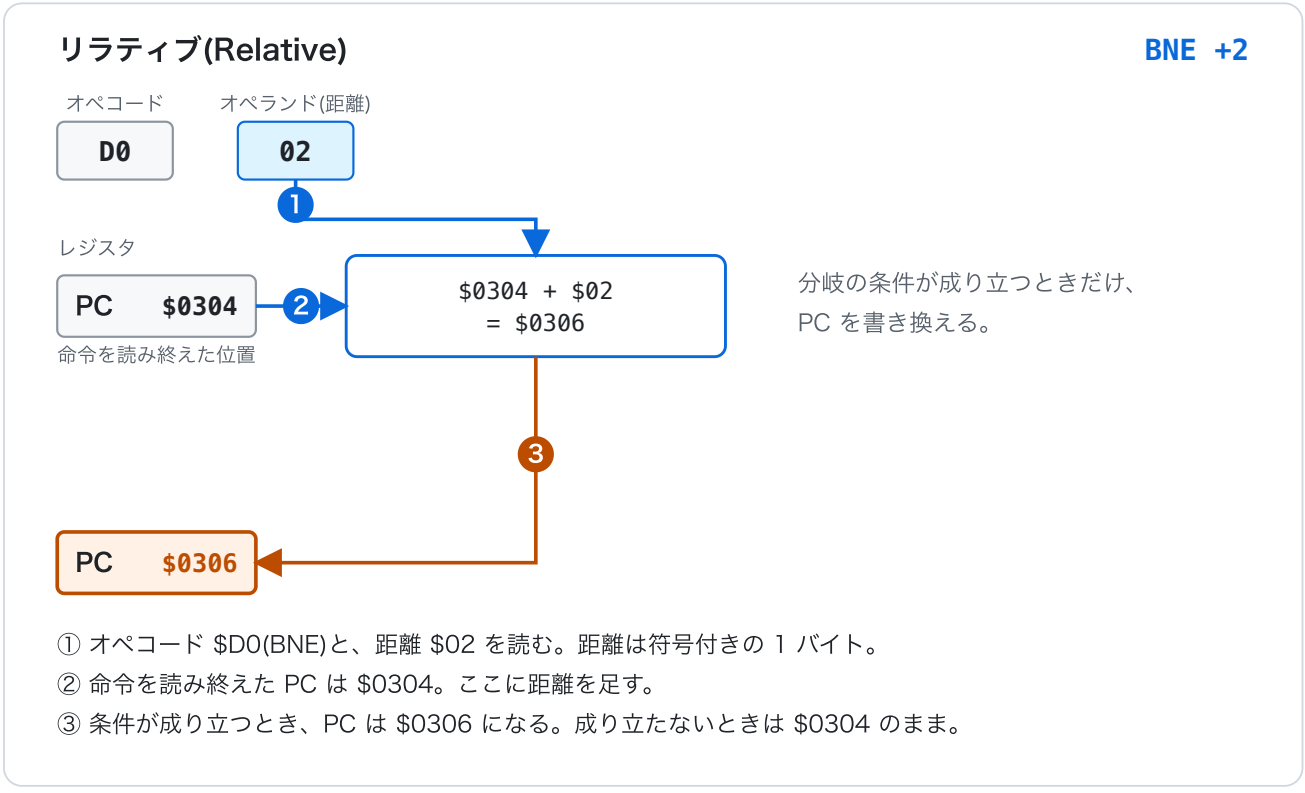
Glow65 で確かめられること

```
*E 0030 00 06
*E 0603 9D
*E 0300 A0 03 B1 30 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 9D 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

4.9 リラティブ(Relative)

分岐命令(`BNE`、`BEQ`、`BCC` など)が使うモードです。オペランドは、符号付きの 1 バイトで、**次の命令の番地からの距離**を表します。範囲は -128 から +127 です。負の数は、第2章の 2 の補数で表します。

項目	内容
書き方	`BNE $0306` (アセンブラが距離を計算する)
機械語	`D0 02`
バイト数	2
飛び先	命令を読み終えた PC + 距離



① オペコード $D0(BNE)と、距離 $02 を読む。距離は符号付きの 1 バイト。
② 命令を読み終えた PC は $0304。ここに距離を足す。
③ 条件が成り立つとき、PC は $0306 になる。成り立たないときは $0304 のまま。

図 4-13 リラティブ(`BNE`)。距離 `$02` のとき、飛び先は `$0306`。

(c) 2026 masanorisakai / MIT License

距離は、命令の先頭ではなく、命令を読み終えた位置から測ります。`D0 02` は 2 バイトの命令なので、`$0302` にあれば、読み終えた PC は `$0304` です。距離が `$FB` (-5)なら、`$0304` から 5 戻って `$02FF` です。

条件が成り立たなければ、分岐しないで次の命令に進みます。実行に必要なサイクル数は、条件が成り立たないとき 2、成り立つと 3、成り立って別のページに飛ぶと 4 です。

Glow65 で確かめられること

```
*E 0300 A9 01 D0 02 A9 EE 8D F0 03 60
*L 0300 0309
0300  A9 01     LDA   #$01
0302  D0 02     BNE   $0306
0304  A9 EE     LDA   #$EE
0306  8D F0 03  STA   $03F0
0309  60        RTS
*G 0300
*D 03F0 03F0
03F0: 01 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`LDA #$01` のあと、Z フラグは 0 です。`BNE` は分岐し、`LDA #$EE` を飛ばして `$0306` へ進みます。`$03F0` には `01` が書かれます。

後ろへ戻る分岐もあります。`BNE $0302` は、距離が `$FD` (-3)です。

```
*E 0300 A2 03 CA D0 FD 8E F0 03 60
*L 0300 0308
0300  A2 03     LDX   #$03
0302  CA        DEX
0303  D0 FD     BNE   $0302
0305  8E F0 03  STX   $03F0
0308  60        RTS
```

`DEX` で X を 1 ずつ減らし、0 になるまで `$0302` に戻ります。

4.10 全モードの一覧

サイクル数は、`LDA` など、そのモードを持つ代表的な命令の値です。`+` は、読み出し命令でページをまたぐと 1 サイクル増えることを表します。

(c) 2026 masanorisakai / MIT License

モード	書き方	バイト数	サイクル数	例
インプライド	(なし)	1	2	`INX`
アキュムレータ	`A`	1	2	`ASL A`
イミディエイト	`#$nn`	2	2	`LDA #$10`
ゼロページ	`$nn`	2	3	`LDA $10`
ゼロページ X	`$nn,X`	2	4	`LDA $10,X`
ゼロページ Y	`$nn,Y`	2	4	`LDX $10,Y`
アブソリュート	`$nnnn`	3	4	`LDA $0400`
アブソリュート X	`$nnnn,X`	3	4 +	`LDA $0410,X`
アブソリュート Y	`$nnnn,Y`	3	4 +	`LDA $0420,Y`
インダイレクト	`($nnnn)`	3	5	`JMP ($0430)`
インデックスト・インダイレクト	`($nn,X)`	2	6	`LDA ($20,X)`
インダイレクト・インデックスト	`($nn),Y`	2	5 +	`LDA ($30),Y`
リラティブ	`$nnnn`	2	2(成立 3、別ページ 4)	`BNE $0306`

`LDA` 以外の命令では、サイクル数が変わることがあります。たとえば `STA $0410,X` は 5 サイクルで、ページをまたがなくても同じです。命令ごとのサイクル数は、第5章の命令リファレンスにまとめます。

4.11 65C02 で加わるモード

65C02 では、次のモードと命令が加わります。`BIT` や `INC` など、6502 にあった命令が新しいモードを持つ場合も含みます。

モード	書き方	バイト数	例
ゼロページ・インダイレクト	`($nn)`	2	`LDA ($12)`
アブソリュート X・インダイレクト	`($nnnn,X)`	3	`JMP ($1234,X)`
ゼロページ X(新しい命令)	`$nn,X`	2	`BIT $10,X`
イミディエイト(新しい命令)	`#$nn`	2	`BIT #$10`
アキュムレータ(新しい命令)	`A`	1	`INC A`
リラティブ(新しい命令)	`$nnnn`	2	`BRA $0309`
ゼロページ・リラティブ	`$nn,$nnnn`	3	`BBR0 $10,$030C`

`BRA` は、いつも分岐する命令です。`BBR0` は、ゼロページの値のビット 0 が 0 のときに分岐します。Glow65 は、パネルの CPU スイッチ(または起動オプション `--cpu 65c02`)で 65C02 にできます。65C02 では、モニタの `L` がこれらの命令を表示します。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

```
glow65-cli --cpu 65c02
*E 0300 B2 12 7C 34 12 34 10 89 10 80 FE 0F 10 FE 1A
*L 0300 030E
0300  B2 12     LDA   ($12)
0302  7C 34 12  JMP   ($1234,X)
0305  34 10     BIT   $10,X
0307  89 10     BIT   #$10
0309  80 FE     BRA   $0309
030B  0F 10 FE  BBR0  $10,$030C
030E  1A        INC   A
```

4.12 練習問題

答えと解き方は、次の節にあります。

問1 次の命令のアドレッシングモードを答えてください。(1) `STA $0300,X` (2) `LDX $20,Y` (3) `JMP ($0430)` (4) `LDA ($20,X)` (5) `LDA #$05`

問2 次の命令の長さ(バイト数)を答えてください。(1) `LDA $0410,X` (2) `LDA ($20),Y` (3) `INX` (4) `BNE $0306` (5) `LDA #$05`

問3 X が `$05` のとき、`LDA $10,X` が読む番地はどこですか。

問4 X が `$20` のとき、`LDA $F0,X`(ゼロページ X)が読む番地はどこですか。`LDA $00F0,X`(アブソリュート X)が読む番地は、どこですか。

問5 Y が `$FF` のとき、`LDA $0420,Y` が読む番地はどこですか。ページをまたぎますか。

問6 ゼロページの `$0042` に `00`、`$0043` に `07` が入っています。X が `$02` のとき、`LDA ($40,X)` が読む番地はどこですか。

問7 ゼロページの `$0050` に `FF`、`$0051` に `05` が入っています。Y が `$01` のとき、`LDA ($50),Y` が読む番地はどこですか。ページをまたぎますか。

問8 `$0310` にある `BNE`(`D0 F0`)の飛び先はどこですか。

問9 `$06FF` に `50`、`$0700` に `05`、`$0600` に `04` が入っています。`JMP ($06FF)` の飛び先は、NMOS 6502 と 65C02 で、それぞれどこですか。

(c) 2026 masanorisakai / MIT License

4.13 練習問題の答えと解き方

問1 (1) アブソリュート X (2) ゼロページ Y (3) インダイレクト (4) インデックスト・インダイレクト (5) イミディエイト

- (1) 2 バイトの番地に `,X`。(2) 1 バイトの番地に `,Y`。`LDX` が持つモードです。(3) `JMP` だけが持つモードで、括弧の中は番地。(4) 括弧の中に `,X` があるのは `(zp,X)`。(5) `#` は値そのもの。

問2 (1) 3 バイト (2) 2 バイト (3) 1 バイト (4) 2 バイト (5) 2 バイト

- (1) オペコードと 2 バイトの番地。(2) オペコードとゼロページの 1 バイト。(3) オペランドなし。(4) 分岐は距離の 1 バイト。(5) オペコードと値の 1 バイト。

問3 `$0015`

- `$10 + $05 = $15`。上位バイトは `$00`。

問4 ゼロページ X は `$0010`。アブソリュート X は `$0110`。

- ゼロページ X は、`$F0 + $20 = $110` を 256 で折り返して `$10`。アブソリュート X は折り返さず、`$00F0 + $20 = $0110`。

問5 `$051F`。ページをまたぐ。

- `$0420 + $FF = $051F`。上位バイトが `$04` から `$05` に変わります。

問6 `$0700`

- `$40 + $02 = $42`。`$0042` の `00` が下位、`$0043` の `07` が上位で、`$0700`。

問7 `$0600`。ページをまたぐ。

- 基準の番地は、`$0050` の `FF` が下位、`$0051` の `05` が上位で `$05FF`。Y を足して `$0600`。上位バイトが `$05` から `$06` に変わります。

問8 `$0302`

- `BNE` は 2 バイトの命令なので、読み終えた PC は `$0312`。`$F0` は -16。`$0312 − 16 = $0302`。

問9 NMOS 6502 は `$0450`。65C02 は `$0550`。

- NMOS は、下位を `$06FF(50)`、上位を同じページの先頭 `$0600(04)`から読んで `$0450`。65C02 は、上位を `$0700(05)`から読んで `$0550`。

出典

1. MOS Technology, "MCS6500 Microcomputer Family Hardware Manual", 1976年1月版: https://www.commodore.ca/wp-content/uploads/2018/11/kim1_hardware.txt

(c) 2026 masanorisakai / MIT License

第5章 命令リファレンス

この章は、6502 の公式の 151 オペコード(56 の命令)を、機能別にまとめたリファレンスです。命令ごとに、動作、影響するフラグ、オペコード表、使い方の例を載せています。命令は、分類ごとのファイルに分けています。

この章の 151 オペコードの表は、NMOS 6502 だけを対象にしています。65C02 で加わる命令とモードは、5.7 65C02 の追加命令にまとめています。

読み方

命令の書式

書き方	意味
`#$nn`	値そのもの(イミディエイト)。`nn` は 1 バイト
`$nn`	ゼロページの番地。`nn` は 1 バイト
`$nnnn`	2 バイトの番地。機械語では、下位バイトが先に並ぶ
`($nn,X)`、`($nn),Y`、`($nnnn)`	インダイレクトの 3 つの形
`A`	アキュムレータ(A レジスタ自身)
`label`	分岐先の番地。アセンブラが、距離に直す

アドレッシングモードは、第4章で説明しています。

フラグ表の記号

各命令の「影響するフラグ」の表は、N、V、D、I、Z、C の 6 つを、次の記号で示します。B(ブレーク)とビット 5 は、P の中に実体がないので、表にありません。

記号	意味
○	結果に応じて 0 か 1 になる
-	変わらない
0、1	いつも 0、いつも 1 になる
M7、M6	メモリの値のビット 7、ビット 6 が、そのまま入る
復元	スタックから取り出した値で置き換わる

オペコード表の見方

「オペコード表」の列は、次のとおりです。

(c) 2026 masanorisakai / MIT License

列	意味
アドレッシングモード	その命令が持つモード
書き方	アセンブリでの書き方
オペコード	機械語の最初の 1 バイト(16進数)
バイト数	命令全体の長さ
サイクル数	実行にかかるサイクル数

サイクル数の記号は、次のとおりです。

表記	意味
`4`	いつも 4 サイクル
`4+`	読み出し命令で、足した結果が別のページになると 1 サイクル増える(5 サイクル)
`2 (+1,+2)`	分岐。条件が成り立たないとき 2、成り立つと 3、成り立って別のページへ飛ぶと 4

ページをまたぐときの 1 サイクルの増加と、分岐のサイクル数は、MOS 社のマニュアル[1]の表にあります。

Glow65 のサイクルとバスアクセス

実機の 6502 は、アドレスの計算のときなどに、結果に使われない余分な読み出し(ダミーリード)と、書き込み(ダミーライト)を行います。Glow65 は、これを再現しません。サイクル数は実機と同じ値を数えますが、パネルや図に現れるバスアクセスの回数は、実機より少ない場合があります。たとえば `CLC` は、サイクル数は 2 で、バスアクセスは 1 回です。

分類

分類	命令
5.1 ロードとストア	LDA、LDX、LDY、STA、STX、STY
5.2 レジスタ間の転送	TAX、TAY、TXA、TYA、TSX、TXS
5.3 算術演算と比較	ADC、SBC、CMP、CPX、CPY、INC、DEC、INX、INY、DEX、DEY
5.4 論理演算とシフト	AND、ORA、EOR、BIT、ASL、LSR、ROL、ROR
5.5 分岐とジャンプ	BCC、BCS、BEQ、BNE、BMI、BPL、BVC、BVS、JMP、JSR、RTS、BRK、RTI
5.6 スタックとフラグ操作	PHA、PLA、PHP、PLP、CLC、SEC、CLI、SEI、CLD、SED、CLV、NOP
5.7 65C02 の追加命令	BRA、PHX、PHY、PLX、PLY、STZ、TRB、TSB、WAI、STP、BBR、BBS、RMB、SMB、新しいモード

(c) 2026 masanorisakai / MIT License

全 56 命令の一覧

ニーモニック	名前	機能	影響するフラグ
LDA	Load Accumulator(ロード A)	メモリの値を A に入れる	N Z
LDX	Load X Register(ロード X)	メモリの値を X に入れる	N Z
LDY	Load Y Register(ロード Y)	メモリの値を Y に入れる	N Z
STA	Store Accumulator(ストア A)	A の値をメモリに書く	なし
STX	Store X Register(ストア X)	X の値をメモリに書く	なし
STY	Store Y Register(ストア Y)	Y の値をメモリに書く	なし
TAX	Transfer A to X(A を X へ転送)	A の値を X にコピーする	N Z
TAY	Transfer A to Y(A を Y へ転送)	A の値を Y にコピーする	N Z
TXA	Transfer X to A(X を A へ転送)	X の値を A にコピーする	N Z
TYA	Transfer Y to A(Y を A へ転送)	Y の値を A にコピーする	N Z
TSX	Transfer Stack Pointer to X(SP を X へ転送)	SP の値を X にコピーする	N Z
TXS	Transfer X to Stack Pointer(X を SP へ転送)	X の値を SP にコピーする	なし
ADC	Add with Carry(キャリー付き加算)	A にメモリの値とキャリーを足す	N V Z C
SBC	Subtract with Carry(キャリー付き減算)	A からメモリの値と借りを引く	N V Z C
CMP	Compare Accumulator(A の比較)	A とメモリの値を比べる	N Z C
CPX	Compare X Register(X の比較)	X とメモリの値を比べる	N Z C
CPY	Compare Y Register(Y の比較)	Y とメモリの値を比べる	N Z C
INC	Increment Memory(メモリを 1 増やす)	メモリの値に 1 を足す	N Z
DEC	Decrement Memory(メモリを 1 減らす)	メモリの値から 1 を引く	N Z
INX	Increment X Register(X を 1 増やす)	X に 1 を足す	N Z
INY	Increment Y Register(Y を 1 増やす)	Y に 1 を足す	N Z
DEX	Decrement X Register(X を 1 減らす)	X から 1 を引く	N Z
DEY	Decrement Y Register(Y を 1 減らす)	Y から 1 を引く	N Z

(c) 2026 masanorisakai / MIT License

ニーモニック	名前	機能	影響するフラグ
AND	Logical AND(論理積)	A とメモリの値のビットごとの AND	N Z
ORA	Logical OR(論理和)	A とメモリの値のビットごとの OR	N Z
EOR	Exclusive OR(排他的論理和)	A とメモリの値のビットごとの XOR	N Z
BIT	Bit Test(ビットテスト)	A とメモリの値を AND して Z を決め、メモリの上位 2 ビットを N と V に写す	N V Z
ASL	Arithmetic Shift Left(算術左シフト)	左に 1 ビットずらし、最上位ビットを C に出す	N Z C
LSR	Logical Shift Right(論理右シフト)	右に 1 ビットずらし、最下位ビットを C に出す	N Z C
ROL	Rotate Left(左ローテート)	C を含めて左に 1 ビット回す	N Z C
ROR	Rotate Right(右ローテート)	C を含めて右に 1 ビット回す	N Z C
BCC	Branch if Carry Clear(C が 0 なら分岐)	C が 0 のとき分岐する	なし
BCS	Branch if Carry Set(C が 1 なら分岐)	C が 1 のとき分岐する	なし
BEQ	Branch if Equal(Z が 1 なら分岐)	Z が 1 のとき分岐する	なし
BNE	Branch if Not Equal(Z が 0 なら分岐)	Z が 0 のとき分岐する	なし
BMI	Branch if Minus(N が 1 なら分岐)	N が 1 のとき分岐する	なし
BPL	Branch if Plus(N が 0 なら分岐)	N が 0 のとき分岐する	なし
BVC	Branch if Overflow Clear(V が 0 なら分岐)	V が 0 のとき分岐する	なし
BVS	Branch if Overflow Set(V が 1 なら分岐)	V が 1 のとき分岐する	なし
JMP	Jump(ジャンプ)	指定の番地へ飛ぶ	なし
JSR	Jump to Subroutine(サブルーチン呼び出し)	戻り番地を積んで、指定の番地へ飛ぶ	なし
RTS	Return from Subroutine(サブルーチンから戻る)	戻り番地を取り出して戻る	なし
BRK	Break(ブレーク)	ソフトウェア割り込みを起こす	I
RTI	Return from Interrupt(割り込みから戻る)	P と戻り番地を取り出して戻る	N V D I Z C
PHA	Push Accumulator(A を積む)	A をスタックに積む	なし
PLA	Pull Accumulator(A に取り出す)	スタックから A に取り出す	N Z
PHP	Push Processor Status(P を積む)	P をスタックに積む	なし

(c) 2026 masanorisakai / MIT License

ニーモニック	名前	機能	影響するフラグ
PLP	Pull Processor Status(P に取り出す)	スタックから P に取り出す	N V D I Z C
CLC	Clear Carry Flag(C を 0 にする)	C を 0 にする	C
SEC	Set Carry Flag(C を 1 にする)	C を 1 にする	C
CLI	Clear Interrupt Disable(I を 0 にする)	割り込みを許可する	I
SEI	Set Interrupt Disable(I を 1 にする)	割り込みを禁止する	I
CLD	Clear Decimal Mode(D を 0 にする)	10進モードを解除する	D
SED	Set Decimal Mode(D を 1 にする)	10進モードにする	D
CLV	Clear Overflow Flag(V を 0 にする)	V を 0 にする	V
NOP	No Operation(何もしない)	何もしない	なし

未定義のオペコード

公式の 151 オペコード以外の 105 個は、公式の命令ではありません。Glow65 は、これらを、オペランドを持たない 1 バイトの 2 サイクルの NOP として実行します。実機の NMOS 6502 では、未定義のオペコードが、オペコードごとに別の動作をしますが、Glow65 は再現しません。

65C02 では、未定義だったオペコードもすべて NOP になります。65C02 で加わる命令(`BRA`、`STZ`、`PHX` など)は、5.7 65C02 の追加命令にあります。

Glow65 で確かめられること

未定義のオペコード `$02` を置いて実行します。`L` は `.BYTE` と表示します。

```
*E 0300 02 A2 05 8E F0 03 60
*L 0300 0301
0300  02        .BYTE $02
0301  A2 05     LDX   #$05
*G 0300
*D 03F0 03F0
03F0: 05 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`$02` は、1 バイトの NOP として動きます。続く `LDX #$05` が実行されて、`$03F0` に `05` が書かれます。

(c) 2026 masanorisakai / MIT License

出典

1. MOS Technology, "MCS6500 Microcomputer Family Hardware Manual", 1976年1月版: https://www.commodore.ca/wp-content/uploads/2018/11/kim1_hardware.txt

(c) 2026 masanorisakai / MIT License

5.1 ロードとストア

ロードは、メモリの値をレジスタに入れる命令です。ストアは、レジスタの値をメモリに書く命令です。ロード命令は N と Z を変えます。ストア命令は、フラグを変えません。

この分類の命令は、どれも 8 ビットを 1 つ扱います。16 ビットの値は、2 回に分けて扱います。

LDA(Load Accumulator)ロード A

メモリの値を A に入れます。値に応じて N と Z が変わります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
イミディエイト	`LDA #$nn`	`$A9`	2	2
ゼロページ	`LDA $nn`	`$A5`	2	3
ゼロページ X	`LDA $nn,X`	`$B5`	2	4
アブソリュート	`LDA $nnnn`	`$AD`	3	4
アブソリュート X	`LDA $nnnn,X`	`$BD`	3	4+
アブソリュート Y	`LDA $nnnn,Y`	`$B9`	3	4+
インデックスト・インダイレクト	`LDA ($nn,X)`	`$A1`	2	6
インダイレクト・インデックスト	`LDA ($nn),Y`	`$B1`	2	5+

使い方の例

```
        LDA #$80
```

A が `$80` になります。最上位ビットが 1 なので、N が 1 になります。

```
        LDA #$00
```

A が `$00` になります。値が 0 なので、Z が 1 になります。

(c) 2026 masanorisakai / MIT License

LDX(Load X Register)ロード X

メモリの値を X に入れます。値に応じて N と Z が変わります。インデックスを使うモードは、ゼロページ Y とアブソリュート Y です。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
イミディエイト	`LDX #$nn`	`$A2`	2	2
ゼロページ	`LDX $nn`	`$A6`	2	3
ゼロページ Y	`LDX $nn,Y`	`$B6`	2	4
アブソリュート	`LDX $nnnn`	`$AE`	3	4
アブソリュート Y	`LDX $nnnn,Y`	`$BE`	3	4+

使い方の例

```
        LDX #$05
```

X が `$05` になります。

前提: メモリの `$0013` に `$5A` が入っている。

```
        LDY #$03
        LDX $10,Y
```

ゼロページ Y。Y の `$03` を `$10` に足した `$0013` の内容 `$5A` が、X に入ります。

LDY(Load Y Register)ロード Y

メモリの値を Y に入れます。値に応じて N と Z が変わります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

(c) 2026 masanorisakai / MIT License

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
イミディエイト	`LDY #$nn`	`$A0`	2	2
ゼロページ	`LDY $nn`	`$A4`	2	3
ゼロページ X	`LDY $nn,X`	`$B4`	2	4
アブソリュート	`LDY $nnnn`	`$AC`	3	4
アブソリュート X	`LDY $nnnn,X`	`$BC`	3	4+

使い方の例

```
        LDY #$FF
```

Y が `$FF` になります。最上位ビットが 1 なので、N が 1 になります。

STA(Store Accumulator)ストア A

A の値をメモリに書きます。A もフラグも変わりません。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
ゼロページ	`STA $nn`	`$85`	2	3
ゼロページ X	`STA $nn,X`	`$95`	2	4
アブソリュート	`STA $nnnn`	`$8D`	3	4
アブソリュート X	`STA $nnnn,X`	`$9D`	3	5
アブソリュート Y	`STA $nnnn,Y`	`$99`	3	5
インデックスト・インダイレクト	`STA ($nn,X)`	`$81`	2	6
インダイレクト・インデックスト	`STA ($nn),Y`	`$91`	2	6

使い方の例

前提: メモリの `$0400` に `$00` が入っている。

```
        LDA #$55
        STA $0400
```

(c) 2026 masanorisakai / MIT License

`$0400` に A の `$55` が書かれます。

前提: メモリの `$0403` に `$00` が入っている。

```
        LDA #$55
        LDX #$03
        STA $0400,X
```

X の `$03` を `$0400` に足した `$0403` に、`$55` が書かれます。

STA のアブソリュート X と Y、インダイレクト・インデックストは、ページをまたがなくても、またぐ場合と同じサイクル数です。

STX(Store X Register)ストア X

X の値をメモリに書きます。ゼロページ Y のモードがあります。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
ゼロページ	`STX $nn`	`$86`	2	3
ゼロページ Y	`STX $nn,Y`	`$96`	2	4
アブソリュート	`STX $nnnn`	`$8E`	3	4

使い方の例

前提: メモリの `$0010` に `$00` が入っている。

```
        LDX #$07
        STX $10
```

`$0010` に X の `$07` が書かれます。

STY(Store Y Register)ストア Y

Y の値をメモリに書きます。ゼロページ X のモードがあります。

影響するフラグ

(c) 2026 masanorisakai / MIT License

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
ゼロページ	`STY $nn`	`$84`	2	3
ゼロページ X	`STY $nn,X`	`$94`	2	4
アブソリュート	`STY $nnnn`	`$8C`	3	4

使い方の例

前提: メモリの `$0400` に `$00` が入っている。

```
        LDY #$09
        STY $0400
```

`$0400` に Y の `$09` が書かれます。

> **Glow65 で確かめられること**
>
> `LDA #$80` のあと、`PHP` と `PLA` で P を A に取り出し、`$03F0` に書きます。
>
> ```
> *E 0300 A9 80 08 68 8D F0 03 60
> *L 0300 0304
> 0300  A9 80     LDA   #$80
> 0302  08        PHP
> 0303  68        PLA
> 0304  8D F0 03  STA   $03F0
> *G 0300
> *D 03F0 03F0
> 03F0: B4 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```
>
> `$B4` は `%10110100` です。N(ビット 7)が 1 になっています。ビット 5 と B(ビット 4)は、PHP が積むときに 1 にします。I(ビット 2)も 1 です。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

3 つのストア命令で、A、X、Y の値を `$03F0` から順に書きます。

```
*E 0300 A9 55 A2 07 A0 09 8D F0 03 8E F1 03 8C F2 03 60
*G 0300
*D 03F0 03F2
03F0: 55 07 09 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

(c) 2026 masanorisakai / MIT License

5.2 レジスタ間の転送

レジスタの値を、別のレジスタにコピーします。コピー元の値は変わりません。TXS を除いて、N と Z がコピーした値で決まります。

A と X、A と Y、X と SP の間だけ、直接コピーできます。X と Y の間の転送命令はありません。

TAX(Transfer A to X)A を X へ転送

A の値を X にコピーします。N と Z が、コピーした値で決まります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`TAX`	`$AA`	1	2

使い方の例

```
LDA #$80
TAX
```

A の `$80` が X にコピーされます。A はそのままで、N が 1 になります。

TAY(Transfer A to Y)A を Y へ転送

A の値を Y にコピーします。N と Z が、コピーした値で決まります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`TAY`	`$A8`	1	2

使い方の例

(c) 2026 masanorisakai / MIT License

```
        LDY #$55
        LDA #$00
        TAY
```

Y が `$55` のとき、A の `$00` を Y にコピーすると、Y は `$00` になります。Z が 1 になります。

TXA(Transfer X to A)X を A へ転送

X の値を A にコピーします。N と Z が、コピーした値で決まります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`TXA`	`$8A`	1	2

使い方の例

```
        LDX #$33
        LDA #$00
        TXA
```

A が `$00` のとき、X の `$33` が A にコピーされます。

TYA(Transfer Y to A)Y を A へ転送

Y の値を A にコピーします。N と Z が、コピーした値で決まります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`TYA`	`$98`	1	2

使い方の例

(c) 2026 masanorisakai / MIT License

```
LDY #$44
TYA
```

Y の `$44` が A にコピーされます。

TSX(Transfer Stack Pointer to X)SP を X へ転送

SP の値を X にコピーします。N と Z が、コピーした値で決まります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`TSX`	`$BA`	1	2

使い方の例

```
LDX #$F0
TXS
LDX #$00
TSX
```

SP を `$F0` にして `TSX` で読むと、X は `$F0` になり、N が 1 になります。

SP を書き換えると、スタックの位置が変わります。この例は、実行の前に SP を保存し、終わる前に元へ戻しています。

TXS(Transfer X to Stack Pointer)X を SP へ転送

X の値を SP にコピーします。TXS は、フラグを変えません。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

(c) 2026 masanorisakai / MIT License

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`TXS`	`$9A`	1	2

使い方の例

```
        LDX #$F0
        LDA #$00
        TXS
```

X が `$F0` でも、直前の `LDA #$00` による Z=1、N=0 が、そのまま残ります。

TXS はフラグを変えません。ほかの転送命令と違う点です。

> **Glow65 で確かめられること**
>
> `LDA #$80` と `TAX`、`LDA #$00` と `TXA` で、A と X の間で値をコピーします。`L` で命令を確かめられます。
>
> ```
> *E 0300 A9 80 AA A9 00 8A 8D F0 03 8E F1 03 60
> *L 0300 0305
> 0300  A9 80     LDA   #$80
> 0302  AA        TAX
> 0303  A9 00     LDA   #$00
> 0305  8A        TXA
> *G 0300
> *D 03F0 03F1
> 03F0: 80 80 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```
>
> 最後の `TXA` で、A が `$00` から X の `$80` になります。X は `$80` のままです。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

SP を `$F0` にして `TSX` で読みます。`$03E4` に元の SP、`$03E5` に `TSX` で読んだ X、`$03E6` に P を書きます。

```
*E 0300 BA 8E E4 03 A2 F0 9A A2 00 BA 08 68 8D E6 03 8E
*E 0310 E5 03 AE E4 03 9A 60
*G 0300
*D 03E4 03E6
03E4: FD F0 B4 00 00 00 00 00 00 00 00 00 00 00 00 00
```

`$03E4` の `FD` は、書き換える前の SP です。`$03E5` の `F0` が `TSX` の結果で、`$03E6` の `B4` は N が 1 であることを示します。

(c) 2026 masanorisakai / MIT License

5.3 算術演算と比較

足し算、引き算、比較、1 ずつの増減をまとめた分類です。ADC と SBC は、C(キャリー)を計算に使います。ADC の前には CLC、SBC の前には SEC で、C を決めておきます。

CMP、CPX、CPY は、引き算の結果を捨てて、フラグだけを変えます。結果は、C(以上かどうか)、Z(等しいかどうか)、N で読みます。

ADC(Add with Carry)キャリー付き加算

A に、メモリの値と C を足して、結果を A に入れます。結果が `$FF` を超えると C が 1 になります。N、V、Z も変わります。

影響するフラグ

N	V	D	I	Z	C
○	○	-	-	○	○

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
イミディエイト	`ADC #$nn`	`$69`	2	2
ゼロページ	`ADC $nn`	`$65`	2	3
ゼロページ X	`ADC $nn,X`	`$75`	2	4
アブソリュート	`ADC $nnnn`	`$6D`	3	4
アブソリュート X	`ADC $nnnn,X`	`$7D`	3	4+
アブソリュート Y	`ADC $nnnn,Y`	`$79`	3	4+
インデックスト・インダイレクト	`ADC ($nn,X)`	`$61`	2	6
インダイレクト・インデックスト	`ADC ($nn),Y`	`$71`	2	5+

使い方の例

```
        CLC
        LDA #$50
        ADC #$50
```

`$50 + $50 = $A0`。符号付きでは 80 + 80 が範囲(-128 〜 127)を超えるので、V が 1 になります。N も 1 です。

(c) 2026 masanorisakai / MIT License

```
        CLC
        LDA #$FF
        ADC #$01
```

`$FF + $01` は 8 ビットに収まらず、A は `$00`、C は 1、Z は 1 になります。

```
        SEC
        LDA #$01
        ADC #$01
```

C が 1 のとき、`1 + 1 + 1` で A は `$03` になります。

前提: メモリの `$0400` から `FF 12 01 00` が入っている。

```
        CLC
        LDA $0400
        ADC $0402
        STA $0404
        LDA $0401
        ADC $0403
        STA $0405
```

16 ビットの足し算です。`$12FF + $0001` を、下位バイトから順に足します。下位の桁上がりは C に残り、上位の ADC が拾います。結果 `$1300` が `$0404`(下位)と `$0405`(上位)に書かれます。

V フラグ: 符号付きの数として足したとき、結果が -128 〜 127 を超えると 1 になります。同じ符号の 2 つの数を足して、符号が変わると 1 です。

10進モード: D が 1 のとき、ADC は BCD(各 4 ビットを 0 〜 9 の数字として扱う形式)で足します。`$09 + $01` は `$0A` ではなく `$10` になります。C は 10進の桁上がりを表します。NMOS 6502 では、10進モードの N、V、Z は、結果の値と一致しない場合があります。10進モードを使い終わったら、CLD で戻します。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

V フラグが変わる様子を、パネルで見られます。`E 0300 B8 A9 50 18 69 50 4C 00 03` を入力して `G 0300` を実行し、**RUN/STOP** を STOP、**STEP MODE** を INSTR にします。プログラムは、`CLV`、`LDA #$50`、`CLC`、`ADC #$50`、`JMP $0300` を繰り返します。

`CLC` を実行した直後は、A が `$50`(A6 と A4 が点灯)で、N と V は消灯しています(図 5-7)。`ADC #$50` を実行すると、A が `$A0`(A7 と A5)になり、N と V が点灯します(図 5-8)。符号付きの数として、正の数 2 つ(80 + 80)の和が負の数(-96)になったので、V が 1 になります。C は 0 のままです。



図 5-7 `CLC` のあと。A は `$50`、V は消灯



図 5-8 `ADC #$50` のあと。A は `$A0`、N と V が点灯

SBC(Subtract with Carry)キャリー付き減算

A から、メモリの値と借りを引いて、結果を A に入れます。借りは、C を反転した値です。C が 1 なら借りなし、0 なら借りありです。引き算の前に SEC で C を 1 にします。

影響するフラグ

N	V	D	I	Z	C
○	○	-	-	○	○

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
イミディエイト	`SBC #$nn`	`$E9`	2	2

(c) 2026 masanorisakai / MIT License

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
ゼロページ	`SBC $nn`	`$E5`	2	3
ゼロページ X	`SBC $nn,X`	`$F5`	2	4
アブソリュート	`SBC $nnnn`	`$ED`	3	4
アブソリュート X	`SBC $nnnn,X`	`$FD`	3	4+
アブソリュート Y	`SBC $nnnn,Y`	`$F9`	3	4+
インデックスト・インダイレクト	`SBC ($nn,X)`	`$E1`	2	6
インダイレクト・インデックスト	`SBC ($nn),Y`	`$F1`	2	5+

使い方の例

```
        SEC
        LDA #$50
        SBC #$30
```

`SEC` のあと、`$50 − $30 = $20`。結果が負にならないので、C は 1 のままです。

```
        SEC
        LDA #$30
        SBC #$50
```

`$30 − $50` は負になります。A は `$E0`(-32)、C は 0(借りあり)、N は 1 です。

```
        SEC
        LDA #$50
        SBC #$B0
```

`$50 − $B0` は、符号付きでは 80 - (-80) = 160 で範囲を超えます。V が 1 になります。

10進モード: D が 1 のとき、SBC も BCD で引きます。`SED`、`SEC`、`LDA #$10` のあと `SBC #$01` を実行すると、A は `$09` になります。C は、借りがなければ 1 のままです。

CMP(Compare Accumulator)A の比較

A とメモリの値を比べます。A から値を引いた結果を計算しますが、捨てて、フラグだけを変えます。A は変わりません。A が値以上なら C が 1、等しければ Z が 1、結果の最上位ビットが N になります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	○

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
イミディエイト	`CMP #$nn`	`$C9`	2	2
ゼロページ	`CMP $nn`	`$C5`	2	3
ゼロページ X	`CMP $nn,X`	`$D5`	2	4
アブソリュート	`CMP $nnnn`	`$CD`	3	4
アブソリュート X	`CMP $nnnn,X`	`$DD`	3	4+
アブソリュート Y	`CMP $nnnn,Y`	`$D9`	3	4+
インデックスト・インダイレクト	`CMP ($nn,X)`	`$C1`	2	6
インダイレクト・インデックスト	`CMP ($nn),Y`	`$D1`	2	5+

使い方の例

```
LDA #$30
CMP #$30
```

A と値が等しいとき、Z が 1、C が 1 になります。

```
LDA #$30
CMP #$40
```

A が値より小さいとき、C が 0 になります。結果が負なので N が 1 です。

```
LDA #$30
CMP #$20
```

A が値より大きいとき、C が 1、Z が 0 になります。

比べたあとは、BEQ(等しい)、BNE(等しくない)、BCS(以上)、BCC(未満)で分岐します。比較は、符号なしの数として行われます。

CPX(Compare X Register)X の比較

X とメモリの値を比べます。CMP と同じ規則で、C、Z、N が変わります。X は変わりません。

影響するフラグ

(c) 2026 masanorisakai / MIT License

N	V	D	I	Z	C
○	-	-	-	○	○

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
イミディエイト	`CPX #$nn`	`$E0`	2	2
ゼロページ	`CPX $nn`	`$E4`	2	3
アブソリュート	`CPX $nnnn`	`$EC`	3	4

使い方の例

```
LDX #$10
CPX #$20
```

X が `$10`、値が `$20` なので、C が 0、N が 1 になります。

CPY(Compare Y Register)Y の比較

Y とメモリの値を比べます。CMP と同じ規則で、C、Z、N が変わります。Y は変わりません。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	○

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
イミディエイト	`CPY #$nn`	`$C0`	2	2
ゼロページ	`CPY $nn`	`$C4`	2	3
アブソリュート	`CPY $nnnn`	`$CC`	3	4

使い方の例

```
LDY #$20
CPY #$20
```

Y と値が等しいので、Z が 1、C が 1 になります。

(c) 2026 masanorisakai / MIT License

INC(Increment Memory)メモリを 1 増やす

メモリの値に 1 を足します。`$FF` の次は `$00` に戻ります。N と Z が変わり、C は変わりません。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
ゼロページ	`INC $nn`	`$E6`	2	5
ゼロページ X	`INC $nn,X`	`$F6`	2	6
アブソリュート	`INC $nnnn`	`$EE`	3	6
アブソリュート X	`INC $nnnn,X`	`$FE`	3	7

使い方の例

前提: メモリの `$0400` に `$FF` が入っている。

```
        INC $0400
```

`$0400` の `$FF` が `$00` になり、Z が 1 になります。

DEC(Decrement Memory)メモリを 1 減らす

メモリの値から 1 を引きます。`$00` の次は `$FF` に戻ります。N と Z が変わり、C は変わりません。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
ゼロページ	`DEC $nn`	`$C6`	2	5
ゼロページ X	`DEC $nn,X`	`$D6`	2	6
アブソリュート	`DEC $nnnn`	`$CE`	3	6
アブソリュート X	`DEC $nnnn,X`	`$DE`	3	7

使い方の例

(c) 2026 masanorisakai / MIT License

前提: メモリの `$0400` に `$00` が入っている。

```
        DEC $0400
```

`$0400` の `$00` が `$FF` になり、N が 1 になります。

INX(Increment X Register)X を 1 増やす

X に 1 を足します。N と Z が変わります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`INX`	`$E8`	1	2

使い方の例

```
        LDX #$FF
        INX
```

X の `$FF` が `$00` になり、Z が 1 になります。

INY(Increment Y Register)Y を 1 増やす

Y に 1 を足します。N と Z が変わります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`INY`	`$C8`	1	2

使い方の例

(c) 2026 masanorisakai / MIT License

```
        LDY #$7F
        INY
```

Y の `$7F` が `$80` になり、N が 1 になります。

DEX(Decrement X Register)X を 1 減らす

X から 1 を引きます。N と Z が変わります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`DEX`	`$CA`	1	2

使い方の例

```
        LDX #$01
        DEX
```

X の `$01` が `$00` になり、Z が 1 になります。

DEY(Decrement Y Register)Y を 1 減らす

Y から 1 を引きます。N と Z が変わります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`DEY`	`$88`	1	2

使い方の例

(c) 2026 masanorisakai / MIT License

```
        LDY #$00
        DEY
```

Y の `$00` が `$FF` になり、N が 1 になります。

INC と DEC は、メモリを対象にします。A に 1 を足す命令は、NMOS 6502 にはありません。`CLC` のあと `ADC #$01` を使います。

> **Glow65 で確かめられること**
>
> `CLC`、`LDA #$50`、`ADC #$50` の結果の A を `$03F0` に、P を `$03F1` に書きます。
>
> ```
> *E 0300 18 A9 50 69 50 8D F0 03 08 68 8D F1 03 60
> *G 0300
> *D 03F0 03F1
> 03F0: A0 F4 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```
>
> A は `$A0` です。P の `$F4` は `%11110100` で、N(ビット 7)と V(ビット 6)が 1、C(ビット 0)が 0 です。

> **Glow65 で確かめられること**
>
> 16 ビットの足し算です。`$0400` と `$0401` の `$12FF`、`$0402` と `$0403` の `$0001` を足し、`$0404` と `$0405` に書きます。
>
> ```
> *E 0400 FF 12 01 00 00 00
> *E 0300 18 AD 00 04 6D 02 04 8D 04 04 AD 01 04 6D 03 04
> *E 0310 8D 05 04 60
> *G 0300
> *D 0400 0405
> 0400: FF 12 01 00 00 13 00 00 00 00 00 00 00 00 00 00
> ```
>
> `$0404` が `00`、`$0405` が `13` で、答えは `$1300` です。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

10進モードで `$09 + $01` を足します。

```
*E 0300 F8 18 A9 09 69 01 D8 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 10 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

結果は `$10` です。2 進数のままなら `$0A` になります。

(c) 2026 masanorisakai / MIT License

5.4 論理演算とシフト

ビットごとの論理演算(AND、OR、XOR)と、ビットを横にずらすシフト、キャリーを含めて回すローテートを扱う分類です。

シフトとローテートは、図 5-1 のように、キャリー(C)を出入口にします。アキュムレータとメモリの、どちらにも使えます。



図 5-1 ASL、LSR、ROL、ROR のビットの動き。

(c) 2026 masanorisakai / MIT License

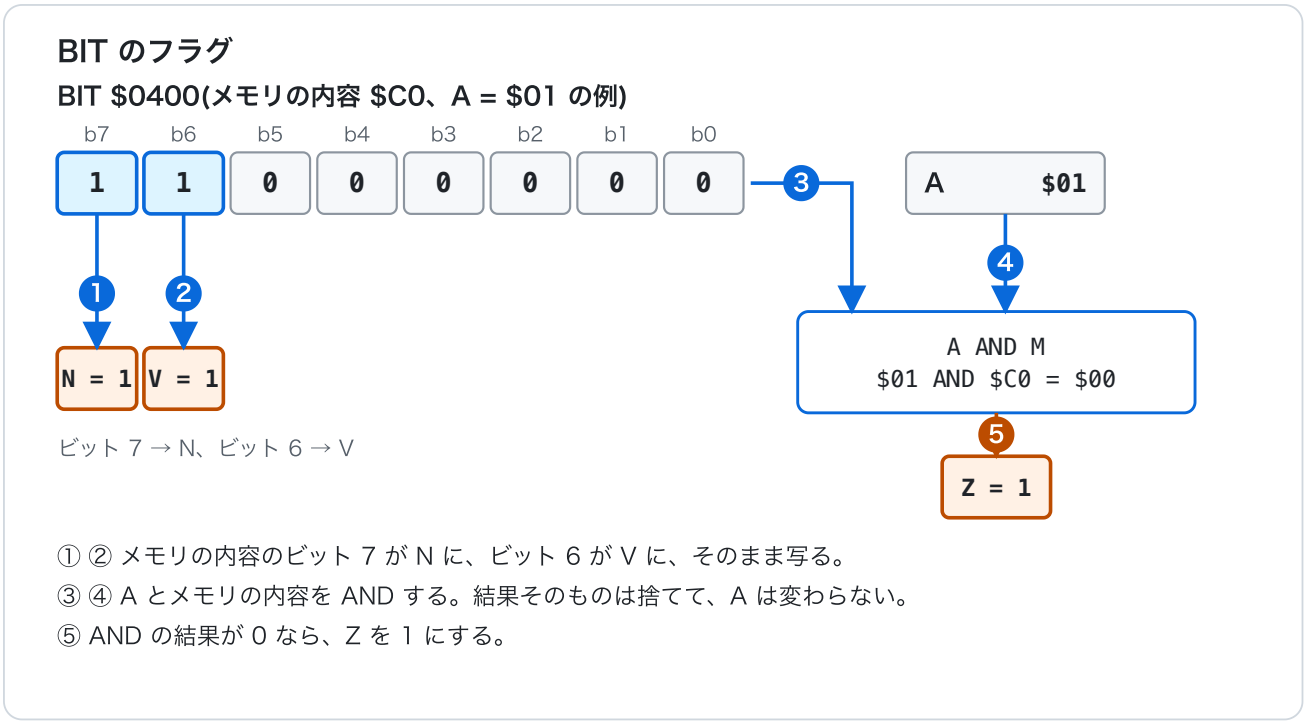


図 5-2 BIT が N、V、Z を決める流れ。

AND(Logical AND)論理積

A とメモリの値を、ビットごとに AND します。両方のビットが 1 のときだけ 1 になります。特定のビットを取り出す(それ以外を 0 にする)ときに使います。N と Z が変わります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
イミディエイト	`AND #$nn`	`$29`	2	2
ゼロページ	`AND $nn`	`$25`	2	3
ゼロページ X	`AND $nn,X`	`$35`	2	4
アブソリュート	`AND $nnnn`	`$2D`	3	4
アブソリュート X	`AND $nnnn,X`	`$3D`	3	4+
アブソリュート Y	`AND $nnnn,Y`	`$39`	3	4+
インデックスト・インダイレクト	`AND ($nn,X)`	`$21`	2	6
インダイレクト・インデックスト	`AND ($nn),Y`	`$31`	2	5+

(c) 2026 masanorisakai / MIT License

使い方の例

```
        LDA #$B5
        AND #$0F
```

`$B5 AND $0F` は `$05` です。下位 4 ビットだけが残ります。

```
        LDA #$F0
        AND #$3C
```

`$F0 AND $3C` は `$30` です。

ORA(Logical OR)論理和

A とメモリの値を、ビットごとに OR します。どちらかのビットが 1 なら 1 になります。特定のビットを 1 にするときに使います。N と Z が変わります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
イミディエイト	`ORA #$nn`	`$09`	2	2
ゼロページ	`ORA $nn`	`$05`	2	3
ゼロページ X	`ORA $nn,X`	`$15`	2	4
アブソリュート	`ORA $nnnn`	`$0D`	3	4
アブソリュート X	`ORA $nnnn,X`	`$1D`	3	4+
アブソリュート Y	`ORA $nnnn,Y`	`$19`	3	4+
インデックスト・インダイレクト	`ORA ($nn,X)`	`$01`	2	6
インダイレクト・インデックスト	`ORA ($nn),Y`	`$11`	2	5+

使い方の例

```
        LDA #$30
        ORA #$05
```

`$30 ORA $05` は `$35` です。

(c) 2026 masanorisakai / MIT License

```
        LDA #$00
        ORA #$80
```

`$80` の ORA で、最上位ビットが 1 になります。N が 1 です。

EOR(Exclusive OR)排他的論理和

A とメモリの値を、ビットごとに XOR(排他的論理和)します。ビットが違うときだけ 1 になります。特定のビットを反転するときに使います。N と Z が変わります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
イミディエイト	`EOR #$nn`	`$49`	2	2
ゼロページ	`EOR $nn`	`$45`	2	3
ゼロページ X	`EOR $nn,X`	`$55`	2	4
アブソリュート	`EOR $nnnn`	`$4D`	3	4
アブソリュート X	`EOR $nnnn,X`	`$5D`	3	4+
アブソリュート Y	`EOR $nnnn,Y`	`$59`	3	4+
インデックスト・インダイレクト	`EOR ($nn,X)`	`$41`	2	6
インダイレクト・インデックスト	`EOR ($nn),Y`	`$51`	2	5+

使い方の例

```
        LDA #$FF
        EOR #$0F
```

`$FF EOR $0F` は `$F0` です。`$0F` のビットが反転します。

```
        LDA #$5A
        EOR #$5A
```

同じ値どうしの XOR は `$00` です。Z が 1 になります。

(c) 2026 masanorisakai / MIT License

BIT(Bit Test)ビットテスト

A とメモリの値を AND して、結果が 0 なら Z を 1 にします。結果そのものは捨てて、A は変わりません。メモリの値のビット 7 が N に、ビット 6 が V に、そのまま写ります。

影響するフラグ

N	V	D	I	Z	C
M7	M6	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
ゼロページ	`BIT $nn`	`$24`	2	3
アブソリュート	`BIT $nnnn`	`$2C`	3	4

使い方の例

前提: メモリの `$0400` に `$C0` が入っている。

```
        LDA #$01
        BIT $0400
```

メモリの値 `$C0` はビット 7 と 6 が 1 なので、N と V が 1 になります。`$01 AND $C0` は 0 なので、Z も 1 です。

前提: メモリの `$0400` に `$3F` が入っている。

```
        LDA #$01
        BIT $0400
```

メモリの値 `$3F` は、ビット 7 と 6 が 0 なので、N と V が 0 になります。`$01 AND $3F` は 1 なので、Z は 0 です。

BIT は、A を壊さずにビットを調べるときに使います。N と V は、A ではなく、メモリの値で決まります。

ASL(Arithmetic Shift Left)算術左シフト

左に 1 ビットずらします。最上位ビットが C に出て、最下位ビットには 0 が入ります。値が 2 倍になります(255 を超えた分は C に出ます)。N、Z、C が変わります。

影響するフラグ

(c) 2026 masanorisakai / MIT License

N	V	D	I	Z	C
○	-	-	-	○	○

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
アキュムレータ	`ASL A`	`$0A`	1	2
ゼロページ	`ASL $nn`	`$06`	2	5
ゼロページ X	`ASL $nn,X`	`$16`	2	6
アブソリュート	`ASL $nnnn`	`$0E`	3	6
アブソリュート X	`ASL $nnnn,X`	`$1E`	3	7

使い方の例

```
        LDA #$81
        ASL A
```

`%10000001` が `%00000010` になります。最上位ビットだった 1 が、C に出ます。

前提: メモリの `$0400` に `$40` が入っている。

```
        ASL $0400
```

`$0400` の `$40` が `$80` になります。N が 1、C が 0 です。

LSR(Logical Shift Right)論理右シフト

右に 1 ビットずらします。最下位ビットが C に出て、最上位ビットには 0 が入ります。値が 2 分の 1 になります(端数は C に出ます)。N は、いつも 0 になります。Z、C も変わります。

影響するフラグ

N	V	D	I	Z	C
0	-	-	-	○	○

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
アキュムレータ	`LSR A`	`$4A`	1	2
ゼロページ	`LSR $nn`	`$46`	2	5
ゼロページ X	`LSR $nn,X`	`$56`	2	6

(c) 2026 masanorisakai / MIT License

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
アブソリュート	`LSR $nnnn`	`$4E`	3	6
アブソリュート X	`LSR $nnnn,X`	`$5E`	3	7

使い方の例

```
        LDA #$03
        LSR A
```

`$03` が `$01` になります。最下位ビットだった 1 が、C に出ます。

```
        LDA #$80
        LSR A
```

`$80` が `$40` になります。C は 0 です。

ROL(Rotate Left)左ローテート

C を含めて、左に 1 ビット回します。最上位ビットが C に出て、元の C が最下位ビットに入ります。N、Z、C が変わります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	○

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
アキュムレータ	`ROL A`	`$2A`	1	2
ゼロページ	`ROL $nn`	`$26`	2	5
ゼロページ X	`ROL $nn,X`	`$36`	2	6
アブソリュート	`ROL $nnnn`	`$2E`	3	6
アブソリュート X	`ROL $nnnn,X`	`$3E`	3	7

使い方の例

```
        SEC
        LDA #$80
        ROL A
```

(c) 2026 masanorisakai / MIT License

C が 1 のとき、`$80` は `$01` になります。元の C の 1 が最下位ビットに入り、最上位ビットだった 1 が C に出ます。

```
        CLC
        LDA #$40
        ROL A
```

C が 0 のとき、`$40` は `$80` になります。N が 1 です。

前提: メモリの `$0400` から `81 42` が入っている。

```
        ASL $0400
        ROL $0401
```

16 ビットの値を左にずらします。`$4281` を `ASL` と `ROL` で 2 倍にして、`$8502` を得ます。下位の ASL で出た C が、上位の ROL の最下位ビットに入ります。

ROR(Rotate Right)右ローテート

C を含めて、右に 1 ビット回します。最下位ビットが C に出て、元の C が最上位ビットに入ります。N、Z、C が変わります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	○

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
アキュムレータ	`ROR A`	`$6A`	1	2
ゼロページ	`ROR $nn`	`$66`	2	5
ゼロページ X	`ROR $nn,X`	`$76`	2	6
アブソリュート	`ROR $nnnn`	`$6E`	3	6
アブソリュート X	`ROR $nnnn,X`	`$7E`	3	7

使い方の例

```
        SEC
        LDA #$01
        ROR A
```

(c) 2026 masanorisakai / MIT License

C が 1 のとき、`$01` は `$80` になります。N が 1、C も 1 です。

```
        CLC
        LDA #$01
        ROR A
```

C が 0 のとき、`$01` は `$00` になります。Z が 1、C が 1 です。

シフトとローテートのアブソリュート X(例: `ASL $0410,X`)は、メモリを読んでから書き戻す命令です。ページをまたがなくても、またぐ場合と同じサイクル数です。

> **Glow65 で確かめられること**
>
> `AND`、`ORA`、`EOR` の結果を、`$03F0`、`$03F1`、`$03F2` に書きます。
>
> ```
> *E 0300 A9 B5 29 0F 8D F0 03 A9 30 09 05 8D F1 03 A9 FF
> *E 0310 49 0F 8D F2 03 60
> *G 0300
> *D 03F0 03F2
> 03F0: 05 35 F0 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```
>
> `$B5 AND $0F` は `05`、`$30 ORA $05` は `35`、`$FF EOR $0F` は `F0` です。

> **Glow65 で確かめられること**
>
> `$0400` と `$0401` に `$4281`(下位が先なので `81 42`)を置き、`ASL` と `ROL` で 2 倍にします。
>
> ```
> *E 0400 81 42
> *E 0300 0E 00 04 2E 01 04 60
> *G 0300
> *D 0400 0401
> 0400: 02 85 00 00 00 00 00 00 00 00 00 00 00 00 00 00
> ```
>
> `$0400` が `02`、`$0401` が `85` で、答えは `$8502` です。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

`$0400` に `$C0` を置き、`LDA #$01` のあと `BIT $0400` を実行して、P を `$03F0` に書きます。

```
*E 0400 C0
*E 0300 A9 01 2C 00 04 08 68 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: F6 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`$F6` は `%11110110` です。N(ビット 7)、V(ビット 6)、Z(ビット 1)が 1 になっています。ビット 5 と B(ビット 4)は PHP が 1 にし、I(ビット 2)も 1 です。

(c) 2026 masanorisakai / MIT License

5.5 分岐とジャンプ

実行の流れを変える命令です。条件分岐(BCC、BCS、BEQ、BNE、BMI、BPL、BVC、BVS)は、フラグを見て、次の命令の番地からの距離だけ飛びます。JMP は、指定の番地へ無条件に飛びます。

JSR と RTS は、サブルーチン(呼び出して使う小さなプログラム)の呼び出しと復帰です。BRK と RTI は、割り込みの入口と出口です。これらはスタックを使います。

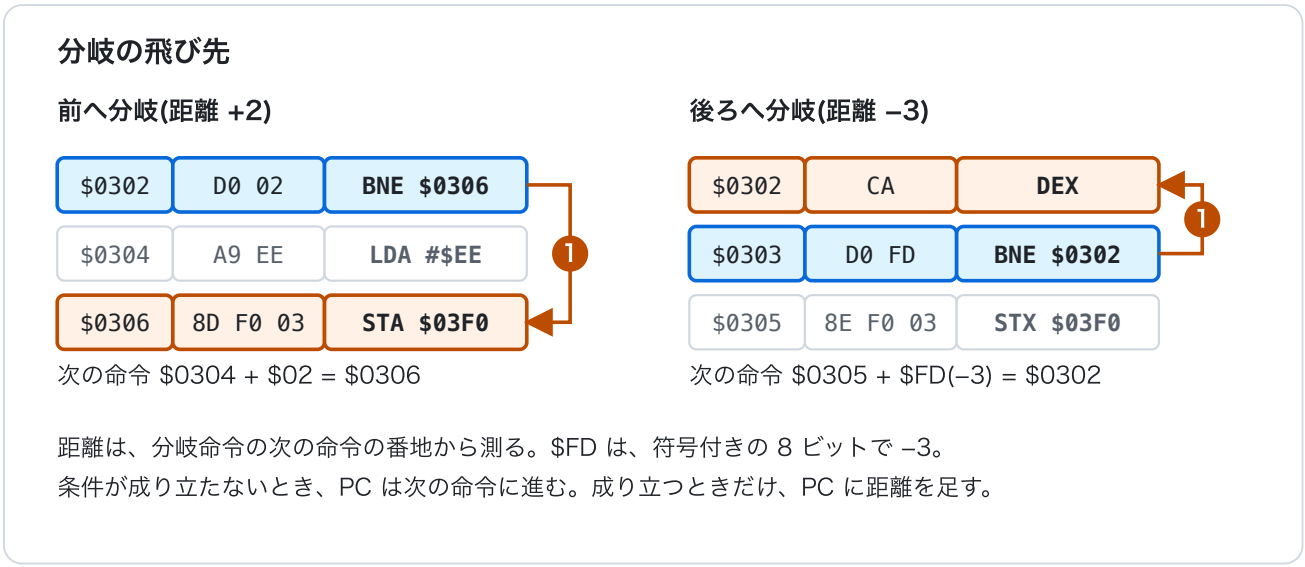


図 5-3 分岐の飛び先。前へも後ろへも飛べる。

(c) 2026 masanorisakai / MIT License

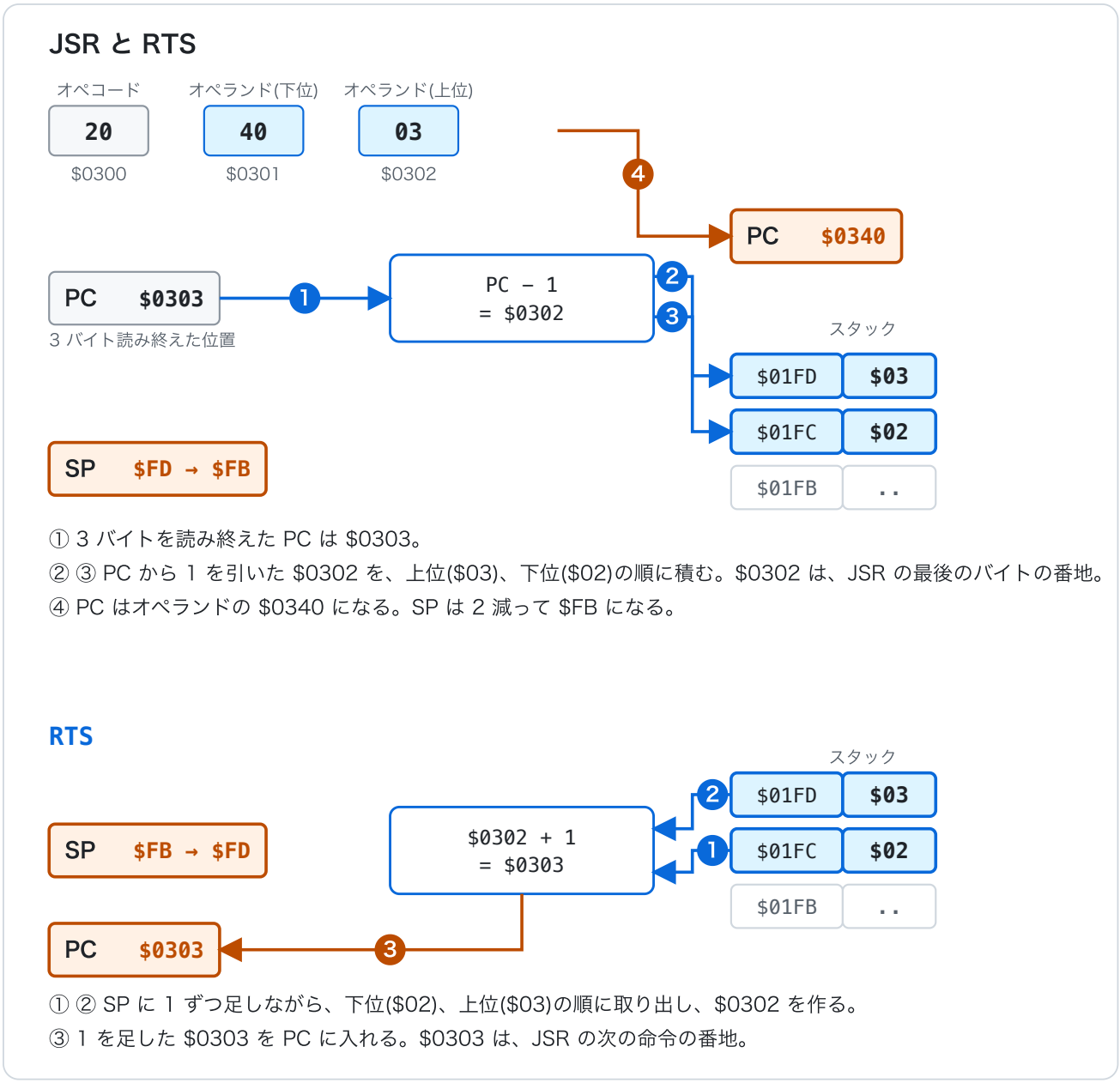


図 5-4 JSR と RTS。積まれる戻り番地は、JSR の最後のバイトの番地。

(c) 2026 masanorisakai / MIT License

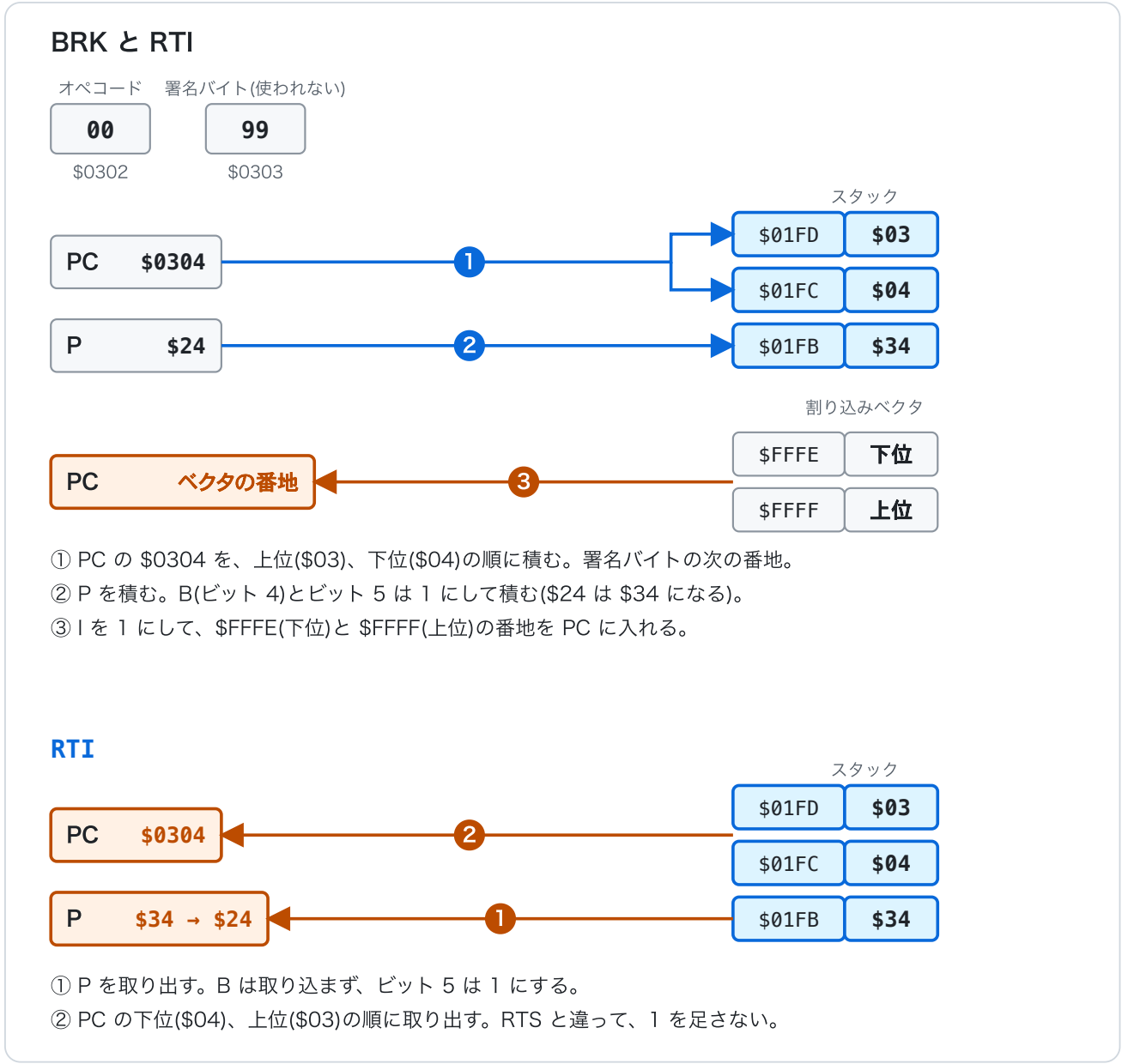


図 5-5 BRK と RTI。

分岐命令は、フラグを変えません。距離は、符号付きの 1 バイトで、-128 から +127 です。サイクル数は、条件が成り立たないとき 2、成り立つと 3、成り立って別のページへ飛ぶと 4 です[1]。

BCC(Branch if Carry Clear)C が 0 なら分岐

C が 0 のとき、分岐します。CMP で「未満」の判定にも使います。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

(c) 2026 masanorisakai / MIT License

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
リラティブ	`BCC label`	`$90`	2	2 (+1,+2)

使い方の例

```
        CLC
        LDX #$00
        BCC skip
        LDX #$EE
skip:   NOP
```

C が 0 なので分岐し、`LDX #$EE` を飛ばします。X は `$00` のままです。

BCS(Branch if Carry Set)C が 1 なら分岐

C が 1 のとき、分岐します。CMP で「以上」の判定にも使います。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
リラティブ	`BCS label`	`$B0`	2	2 (+1,+2)

使い方の例

```
        CLC
        LDX #$00
        BCS skip
        LDX #$EE
skip:   NOP
```

C が 0 なので分岐しません。`LDX #$EE` が実行されて、X は `$EE` になります。

BEQ(Branch if Equal)Z が 1 なら分岐

Z が 1 のとき、分岐します。直前の結果が 0、または CMP で等しかったときです。

影響するフラグ

(c) 2026 masanorisakai / MIT License

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
リラティブ	`BEQ label`	`$F0`	2	2 (+1,+2)

使い方の例

```
        LDX #$00
        LDA #$00
        BEQ skip
        LDX #$EE
skip:   NOP
```

`LDA #$00` で Z が 1 になっているので、分岐します。X は `$00` のままです。

BNE(Branch if Not Equal)Z が 0 なら分岐

Z が 0 のとき、分岐します。ループの終わりの判定によく使います。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
リラティブ	`BNE label`	`$D0`	2	2 (+1,+2)

使い方の例

```
        LDY #$00
        LDX #$03
loop:   INY
        DEX
        BNE loop
```

`DEX` で X が 0 になるまで、`INY` を繰り返します。3 回まわって、Y が `$03` になります。

(c) 2026 masanorisakai / MIT License

BMI(Branch if Minus)N が 1 なら分岐

N が 1 のとき、分岐します。直前の結果の最上位ビットが 1 のときです。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
リラティブ	`BMI label`	`$30`	2	2 (+1,+2)

使い方の例

```
        LDX #$00
        LDA #$80
        BMI skip
        LDX #$EE
skip:   NOP
```

`LDA #$80` で N が 1 になっているので、分岐します。X は `$00` のままです。

BPL(Branch if Plus)N が 0 なら分岐

N が 0 のとき、分岐します。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
リラティブ	`BPL label`	`$10`	2	2 (+1,+2)

使い方の例

(c) 2026 masanorisakai / MIT License

```
        LDX #$00
        LDA #$7F
        BPL skip
        LDX #$EE
skip:   NOP
```

`LDA #$7F` で N が 0 になっているので、分岐します。X は `$00` のままです。

BVC(Branch if Overflow Clear)V が 0 なら分岐

V が 0 のとき、分岐します。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
リラティブ	`BVC label`	`$50`	2	2 (+1,+2)

使い方の例

```
        CLV
        LDX #$00
        BVC skip
        LDX #$EE
skip:   NOP
```

`CLV` で V が 0 になっているので、分岐します。X は `$00` のままです。

BVS(Branch if Overflow Set)V が 1 なら分岐

V が 1 のとき、分岐します。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

(c) 2026 masanorisakai / MIT License

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
リラティブ	`BVS label`	`$70`	2	2 (+1,+2)

使い方の例

```
        CLV
        LDX #$00
        BVS skip
        LDX #$EE
skip:   NOP
```

`CLV` で V が 0 なので分岐しません。`LDX #$EE` が実行されて、X は `$EE` になります。

JMP(Jump)ジャンプ

指定の番地へ飛びます。フラグは変わりません。アブソリュートとインダイレクトのモードがあります。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
アブソリュート	`JMP $nnnn`	`$4C`	3	3
インダイレクト	`JMP ($nnnn)`	`$6C`	3	5

使い方の例

```
        LDA #$00
        JMP skip
        LDA #$EE
skip:   LDX #$11
```

`LDA #$EE` を飛ばして、`LDX #$11` から実行します。A は `$00` のままです。

インダイレクト(`JMP ($nnnn)`)は、NMOS 6502 では、ポインタがページの最後のバイト(`$xxFF`)にあるときの動作に癖があります。第4章 4.6 で説明しています。

(c) 2026 masanorisakai / MIT License

JSR(Jump to Subroutine)サブルーチン呼び出し

次の命令の番地から 1 を引いた値(JSR の最後のバイトの番地)をスタックに積み、指定の番地へ飛びます。上位バイト、下位バイトの順に積みます[1]。フラグは変わりません。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
アブソリュート	`JSR $nnnn`	`$20`	3	6

使い方の例

```
        LDA #$00
        JSR $0340
        TAX
```

`$0340` にあるサブルーチン:

```
        LDA #$AB
        RTS
```

`$0340` のサブルーチンが A を `$AB` にして戻ります。戻ったあとの `TAX` で、X も `$AB` になります。

```
        JSR $0340
```

`$0340` にあるサブルーチン:

```
        TSX
        LDA $0101,X
        STA $03E4
        LDA $0102,X
        STA $03E5
        RTS
```

サブルーチンの中で、スタックに積まれた戻り番地を読みます。`$0300` の JSR の最後のバイトの番地 `$0302` が積まれているので、下位は `$02`、上位は `$03` です。

(c) 2026 masanorisakai / MIT License

RTS(Return from Subroutine)サブルーチンから戻る

スタックから戻り番地を取り出し、1 を足して PC に入れます[1]。JSR の次の命令へ戻ります。フラグは変わりません。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`RTS`	`$60`	1	6

使い方の例

```
        LDX #$00
        LDA #$03
        PHA
        LDA #$0A
        PHA
        RTS
        LDX #$EE
        LDY #$01
```

`$030A` を自分で積んで RTS を実行すると、`$030A` の 1 つ先の `$030B`(`LDY #$01`)から続きます。`LDX #$EE` は飛ばされます。

JSR が積む値は、次の命令の番地より 1 小さい値です。RTS が 1 を足します。戻り番地をスタックに自分で積んで RTS を使うと、その番地の次から実行を続けられます。

BRK(Break)ブレーク

ソフトウェア割り込みを起こします。PC(BRK の番地 + 2)と、B を 1 にした P をスタックに積み、I を 1 にして、`$FFFE` と `$FFFF` の番地へ飛びます。

影響するフラグ

N	V	D	I	Z	C
-	-	-	1	-	-

オペコード

(c) 2026 masanorisakai / MIT License

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`BRK`	`$00`	1	7

使い方の例

```
        LDX #$00
        LDY #$00
        BRK
        .byte $99
        LDY #$55
```

`$0340` にある割り込みハンドラ:

```
        TSX
        LDA $0101,X
        STA $03E4
        LDA $0102,X
        STA $03E5
        LDA $0103,X
        STA $03E6
        LDX #$77
        RTI
```

`BRK` の次の 1 バイトは署名バイトで、使われません。戻り番地は、署名バイトの次の `$0306` です。ハンドラが X を `$77` にして RTI で戻るので、`LDY #$55` が続きます。

BRK は 1 バイトの命令ですが、PC が 2 バイト分進みます。次のバイトは、BRK の種類を示す署名として使えます。

Glow65 のモニタは、`$FFFE` と `$FFFF` のベクタから、RAM の `$03FA` と `$03FB` にあるハンドラの番地へ飛びます。この例では、`E 03FA 40 03` で、ハンドラの番地を `$0340` にしています。割り込みは第6章で説明します。

RTI(Return from Interrupt)割り込みから戻る

スタックから P、PC の下位、PC の上位の順に取り出して、割り込みから戻ります。RTS と違って、PC に 1 を足しません。P の B(ビット 4)は取り込まれません。

影響するフラグ

N	V	D	I	Z	C
復元	復元	復元	復元	復元	復元

(c) 2026 masanorisakai / MIT License

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`RTI`	`$40`	1	6

使い方の例

`BRK` の例のハンドラは、最後の `RTI` で、署名バイトの次の `$0306` に戻ります。

> **Glow65 で確かめられること**
>
> `INY` と `DEX` を、`BNE` で 3 回繰り返します。`L` で `BNE $0304` の飛び先を確かめられます。
>
> ```
> *E 0300 A0 00 A2 03 C8 CA D0 FC 8C F0 03 60
> *L 0300 030A
> 0300  A0 00     LDY   #$00
> 0302  A2 03     LDX   #$03
> 0304  C8        INY
> 0305  CA        DEX
> 0306  D0 FC     BNE   $0304
> 0308  8C F0 03  STY   $03F0
> *G 0300
> *D 03F0 03F0
> 03F0: 03 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```
>
> `D0 FC` の距離 `$FC` は -4 です。次の命令 `$0308` から 4 戻って、`$0304` になります。

> **Glow65 で確かめられること**
>
> `$0340` のサブルーチンが、スタックに積まれた戻り番地を `$03F2` と `$03F3` に書きます。
>
> ```
> *E 0340 BA BD 01 01 8D F2 03 BD 02 01 8D F3 03 60
> *E 0300 20 40 03 60
> *L 0300 0300
> 0300  20 40 03  JSR   $0340
> *G 0300
> *D 03F2 03F3
> 03F2: 02 03 00 00 00 00 24 F4 24 F4 00 00 00 00 00 00
> ```
>
> 下位の `02` と上位の `03` で、積まれた値は `$0302` です。`JSR` の最後のバイトの番地です。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

`BRK` で `$0340` のハンドラへ飛び、積まれた戻り番地を `$03F2`(上位)と `$03F3`(下位)に書いて、RTI で戻ります。

```
*E 03FA 40 03
*E 0340 BA BD 03 01 8D F2 03 BD 02 01 8D F3 03 A2 77 40
*E 0300 A2 00 A0 00 00 99 A0 55 8E F0 03 8C F1 03 60
*L 0300 0304
0300  A2 00     LDX   #$00
0302  A0 00     LDY   #$00
0304  00        BRK
*G 0300
*D 03F0 03F3
03F0: 77 55 03 06 00 00 00 00 24 F4 40 03 00 00 00 00
```

`$03F0` と `$03F1` は、戻ったあとの X と Y で、`77` と `55` です。`$03F2` と `$03F3` は戻り番地で、`$0306` です。署名バイトの `$0305` の次の番地です。なお、`$03FA` と `$03FB` の `40 03` は、この例で書いたハンドラの番地です。

出典

1. MOS Technology, "MCS6500 Microcomputer Family Hardware Manual", 1976年1月版: https://www.commodore.ca/wp-content/uploads/2018/11/kim1_hardware.txt

(c) 2026 masanorisakai / MIT License

5.6 スタックとフラグ操作

スタックは、`$0100` から `$01FF` のメモリを、後から入れたものを先に取り出す順(後入れ先出し)で使う領域です。SP(スタックポインタ)が、次に書く番地の下位バイトを指します。積むときは、書いてから SP を 1 減らします。取り出すときは、SP を 1 増やしてから読みます。

この分類には、スタックに積む命令(PHA、PHP)、取り出す命令(PLA、PLP)、フラグを直接変える命令、何もしない NOP が入ります。

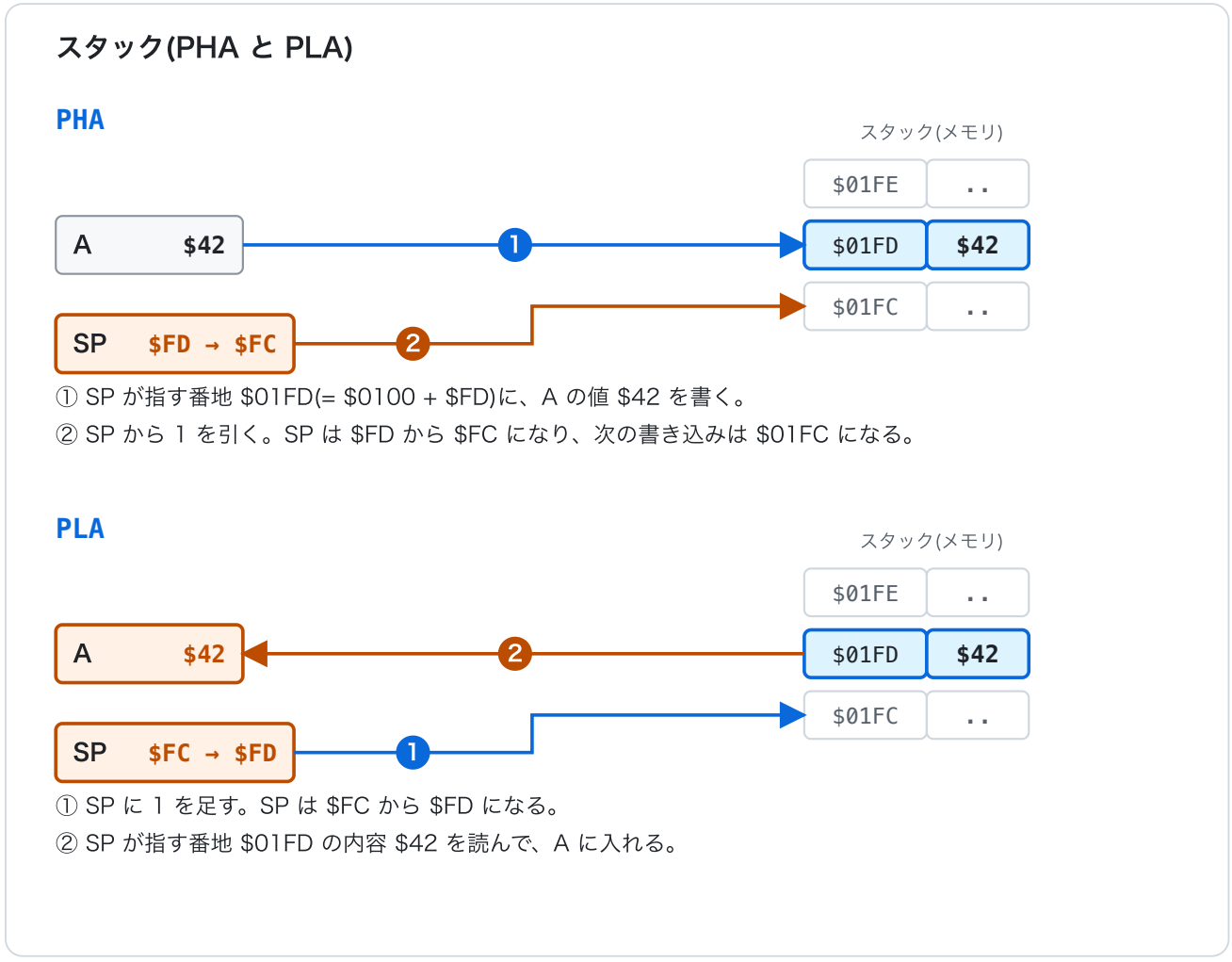


図 5-6 PHA と PLA。スタックは、大きい番地から小さい番地へ伸びる。

PHA(Push Accumulator)A を積む

A の値を `$0100 + SP` に書いて、SP から 1 を引きます。A もフラグも変わりません。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

(c) 2026 masanorisakai / MIT License

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`PHA`	`$48`	1	3

使い方の例

```
LDA #$42
PHA
TSX
LDA $0101,X
STA $03E4
LDA #$00
PLA
```

A の `$42` を積んだあと、`TSX` で SP を X に読み、`LDA $0101,X` で SP の 1 つ上の番地を読むと、積んだ `$42` が見つかります。そのあと A を `$00` にして、`PLA` で `$42` に戻します。

PLA(Pull Accumulator)A に取り出す

SP に 1 を足して、`$0100 + SP` の値を A に入れます。N と Z が、取り出した値で決まります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`PLA`	`$68`	1	4

使い方の例

```
LDA #$42
PHA
LDA #$00
PLA
```

積んでおいた `$42` が、`PLA` で A に戻ります。N も Z も 0 になります。

(c) 2026 masanorisakai / MIT License

PHP(Push Processor Status)P を積む

P をスタックに積みます。積む値は、B(ビット 4)とビット 5 を 1 にした P です。P は変わりません。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`PHP`	`$08`	1	3

使い方の例

```
        SEI
        CLD
        CLV
        LDA #$01
        SEC
        PHP
        PLA
```

`SEI`、`CLD`、`CLV`、`LDA #$01`、`SEC` で、C と I だけが 1 の状態にして積みます。取り出した値は `$35`(`%00110101`)です。ビット 5 と B が 1 になっています。

PLP(Pull Processor Status)P に取り出す

スタックから取り出した値で、P を置き換えます。取り出した値の B(ビット 4)は取り込まず、ビット 5 は 1 のままです。N、V、D、I、Z、C が、すべて取り出した値になります。

影響するフラグ

N	V	D	I	Z	C
復元	復元	復元	復元	復元	復元

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`PLP`	`$28`	1	4

使い方の例

(c) 2026 masanorisakai / MIT License

```
        SEC
        PHP
        CLC
        PLP
```

`SEC` のあと P を積み、`CLC` で C を 0 にしてから、`PLP` で戻します。C は 1 に戻ります。

```
        LDA #$C3
        PHA
        PLP
```

`$C3`(`%11000011`)を積んで `PLP` で取り込むと、N、V、Z、C が 1 になり、D が 0 になります。

PHP と PLP は、割り込みやサブルーチンの前後で、フラグを保存して戻すときに使います。

CLC(Clear Carry Flag)C を 0 にする

C を 0 にします。ADC の前に使います。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	0

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`CLC`	`$18`	1	2

使い方の例

```
        SEC
        CLC
```

C が 0 になります。

SEC(Set Carry Flag)C を 1 にする

C を 1 にします。SBC の前に使います。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	1

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`SEC`	`$38`	1	2

使い方の例

```
    CLC
    SEC
```

C が 1 になります。

CLI(Clear Interrupt Disable)I を 0 にする

I を 0 にします。割り込み(IRQ)を受け付けるようになります。

影響するフラグ

N	V	D	I	Z	C
-	-	-	0	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`CLI`	`$58`	1	2

使い方の例

```
    SEI
    CLI
```

I が 0 になります。

SEI(Set Interrupt Disable)I を 1 にする

I を 1 にします。割り込み(IRQ)を受け付けなくなります。

影響するフラグ

(c) 2026 masanorisakai / MIT License

N	V	D	I	Z	C
-	-	-	1	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`SEI`	`$78`	1	2

使い方の例

```
    CLI
    SEI
```

I が 1 になります。

CLD(Clear Decimal Mode)D を 0 にする

D を 0 にします。ADC と SBC が 2 進数で計算するようになります。

影響するフラグ

N	V	D	I	Z	C
-	-	0	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`CLD`	`$D8`	1	2

使い方の例

```
    SED
    CLD
```

D が 0 になります。

SED(Set Decimal Mode)D を 1 にする

D を 1 にします。ADC と SBC が BCD で計算するようになります。

影響するフラグ

(c) 2026 masanorisakai / MIT License

N	V	D	I	Z	C
-	-	1	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`SED`	`$F8`	1	2

使い方の例

```
        CLD
        SED
```

D が 1 になります。

10進モードを使ったら、CLD で戻します。D が 1 のままだと、あとの足し算と引き算の結果が変わります。

CLV(Clear Overflow Flag)V を 0 にする

V を 0 にします。V を 1 にする命令は、6502 にはありません。ADC、SBC、BIT が、結果に応じて 1 にします。

影響するフラグ

N	V	D	I	Z	C
-	0	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`CLV`	`$B8`	1	2

使い方の例

```
        CLC
        LDA #$7F
        ADC #$01
        CLV
```

`ADC` で `$7F + $01 = $80` となり、V が 1 になったあとに、`CLV` で 0 に戻ります。N は 1 のままです。

(c) 2026 masanorisakai / MIT License

NOP(No Operation)何もしない

何もしません。1 バイトの命令で、2 サイクルかかります。レジスタもフラグも変わりません。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`NOP`	`$EA`	1	2

使い方の例

```
        LDA #$80
        NOP
```

`NOP` のあとも、A の `$80` と N の 1 が変わりません。

NOP は、プログラムの途中を一時的に空ける、実行時間を調整する、などに使います。

> **Glow65 で確かめられること**
>
> `SEI`、`CLD`、`CLV`、`LDA #$01`、`SEC` のあと `PHP` で積み、`PLA` で A に取り出して、`$03F0` に書きます。
>
> ```
> *E 0300 78 D8 B8 A9 01 38 08 68 8D F0 03 60
> *G 0300
> *D 03F0 03F0
> 03F0: 35 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```
>
> `$35` は `%00110101` です。C(ビット 0)と I(ビット 2)が 1 で、ビット 5 と B(ビット 4)は PHP が 1 にしています。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

`PHA` と `PLA` で、A の値を退避して戻します。`LDA #$00` で A を消したあとも、`PLA` で `$42` が戻ります。

```
*E 0300 A9 42 48 A9 00 68 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 42 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

Glow65 で確かめられること

`SEC`、`PHP`、`CLC`、`PLP` で、C を保存して戻します。

```
*E 0300 38 08 18 28 08 68 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 35 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`$35` は、C が 1 に戻っていることを示します。

(c) 2026 masanorisakai / MIT License

5.7 65C02 の追加命令

65C02 は、6502 の改良版です。6502 は NMOS という方式で作られ、65C02 は CMOS という方式で作られています。CMOS は、消費電力が少ない方式です。6502 の公式の 151 オペコードは、65C02 にもあります。65C02 は、それに新しい命令とモードを足し、いくつかの動作を直しています。

Glow65 の 65C02 は、WDC 社の W65C02S と同じ命令を持ちます。基本の 65C02 の命令に、Rockwell 社が足した BBR、BBS、RMB、SMB と、WDC 社が足した WAI、STP を加えたものです。

6502 との違い

新しい命令は 14 個あります。一覧は、「新しい命令の一覧」にあります。

新しいモードは 3 つあります。ゼロページ・インダイレクト `($nn)`、アブソリュート X・インダイレクト `($nnnn,X)`、ゼロページ・リラティブ `$nn,label` です。`INC A`、`DEC A`、`BIT #$nn`、`BIT $nn,X`、`BIT $nnnn,X` のように、6502 の命令が新しい形を持つ場合もあります。

動作は、次の点が変わっています。

項目	NMOS 6502	65C02
`JMP ($xxFF)`	上位バイトを、同じページの先頭から読む。5 サイクル	上位バイトを、ポインタの次の番地から読む。6 サイクル
割り込み(IRQ、NMI、BRK)を受け付けたとき	D は変わらない	D を 0 にする
10進モードの ADC	Z は 2 進で足した結果から、N は途中の値から決まる	N と Z は、調整後の A から決まる。1 サイクル増える
10進モードの SBC	N と Z は、2 進で引いた結果から決まる	N と Z は、調整後の A から決まる。1 サイクル増える
ASL、LSR、ROL、ROR の `$nnnn,X`	7 サイクル	6 サイクル。別のページへ出ると 7
未定義のオペコード	Glow65 では、1 バイトで 2 サイクルの NOP	すべて NOP。長さとサイクル数は、オペコードごとに決まっている

D を 0 にする変更は、割り込みハンドラの中が、10進モードの影響を受けないようにするためです。`JMP ($xxFF)` の動作は、第4章の「インダイレクト」で説明しています。

Glow65 で 65C02 を動かす

パネルの **CPU** スイッチで、6502 と 65C02 を選びます。スイッチは、POWER が OFF のときだけ動かせます。コマンドラインでは、`glow65-cli --cpu 65c02` と指定します。指定しないときは 6502 です。

(c) 2026 masanorisakai / MIT License

65C02 を選んだときだけ、モニタの `L` コマンドが、新しい命令を逆アセンブルします。6502 では、同じバイトが `.BYTE` と表示されます。

> **Glow65 で確かめられること**
>
> 同じ 5 バイトを、65C02 と 6502 で逆アセンブルします。`$CB` は WAI、`$DB` は STP です。
>
> 65C02:
>
> ```
> *E 0300 1A DA FA CB DB
> *L 0300 0304
> 0300  1A        INC   A
> 0301  DA        PHX
> 0302  FA        PLX
> 0303  CB        WAI
> 0304  DB        STP
> ```
>
> 6502:
>
> ```
> *E 0300 1A DA FA CB DB
> *L 0300 0304
> 0300  1A        .BYTE $1A
> 0301  DA        .BYTE $DA
> 0302  FA        .BYTE $FA
> 0303  CB        .BYTE $CB
> 0304  DB        .BYTE $DB
> ```

実行したときの違いも見ます。次のプログラムは、`INC A` で A を 1 増やし、`PHX` と `PLX` で X をスタックに積んで戻し、`STZ` で `$03F2` に 0 を書きます。`$03F2` には、先に `$FF` を入れておきます。

```
        LDA #$05
        INC A
        STA $03F0
        LDX #$07
        PHX
        LDX #$00
        PLX
        STX $03F1
        STZ $03F2
        RTS
```

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

65C02:

```
*E 03F2 FF
*E 0300 A9 05 1A 8D F0 03 A2 07 DA A2 00 FA 8E F1 03 9C F2 03 60
*G 0300
*D 03F0 03F2
03F0: 06 07 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

6502 では、同じバイトを入れても結果が変わります。`1A`、`DA`、`FA`、`9C` は何もしない命令として実行され、`$03F2` の `$FF` は書き換わりません。

```
*E 03F2 FF
*E 0300 A9 05 1A 8D F0 03 A2 07 DA A2 00 FA 8E F1 03 9C F2 03 60
*G 0300
*D 03F0 03F2
03F0: 05 00 FF 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

ca65 でアセンブルするときは、ソースの先頭に `.setcpu "65C02"` を書きます。`Tools/assemble.sh` で、`$0300` に置くバイナリにできます。

```
        .setcpu "65C02"
        LDA #$05
        INC A
```

新しい命令の一覧

ニーモニック	名前	機能	影響するフラグ
BRA	Branch Always(いつも分岐)	いつも分岐する	なし
PHX	Push X Register(X を積む)	X をスタックに積む	なし
PHY	Push Y Register(Y を積む)	Y をスタックに積む	なし
PLX	Pull X Register(X に取り出す)	スタックから X に取り出す	N Z
PLY	Pull Y Register(Y に取り出す)	スタックから Y に取り出す	N Z
STZ	Store Zero(0 を書く)	メモリに 0 を書く	なし
TRB	Test and Reset Bits(ビットを調べて 0 にする)	A とメモリを AND して Z を決め、A の 1 のビットをメモリで 0 にする	Z

(c) 2026 masanorisakai / MIT License

ニーモニック	名前	機能	影響するフラグ
TSB	Test and Set Bits(ビットを調べて 1 にする)	A とメモリを AND して Z を決め、A の 1 のビットをメモリで 1 にする	Z
WAI	Wait for Interrupt(割り込みを待つ)	割り込みが来るまで止まる	なし
STP	Stop the Clock(停止)	RESET まで止まる	なし
BBR	Branch on Bit Reset(ビットが 0 なら分岐)	ゼロページの値のビット n が 0 のとき分岐する	なし
BBS	Branch on Bit Set(ビットが 1 なら分岐)	ゼロページの値のビット n が 1 のとき分岐する	なし
RMB	Reset Memory Bit(メモリのビットを 0 にする)	ゼロページの値のビット n を 0 にする	なし
SMB	Set Memory Bit(メモリのビットを 1 にする)	ゼロページの値のビット n を 1 にする	なし

`BBR`、`BBS`、`RMB`、`SMB` は、ビット番号ごとに別のオペコードを持ちます。`BBR0` から `BBR7` のように、番号を付けて書きます。

新しい命令

BRA(Branch Always)いつも分岐

条件なしで、いつも分岐します。距離は、ほかの分岐命令と同じく、符号付きの 1 バイトで、-128 から +127 です。フラグは変わりません。

サイクル数は 3 です。飛び先が別のページにあると 4 です。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
リラティブ	`BRA label`	`$80`	2	3 (+1)

使い方の例

(c) 2026 masanorisakai / MIT License

```
        LDX #$00
        BRA skip
        LDX #$EE
skip:   STX $03F0
```

`BRA` は必ず分岐するので、`LDX #$EE` を飛ばします。X は `$00` のままです。6502 では、いつも分岐させたいとき、`JMP` を使うか、フラグが決まっている分岐命令を使っていました。`BRA` は 2 バイトで済み、置く場所が変わっても書き換えが要りません。

> **Glow65 で確かめられること**
>
> ```
> *E 0300 A2 00 80 02 A2 EE 8E F0 03 60
> *G 0300
> *D 03F0 03F0
> 03F0: 00 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```

PHX(Push X Register)X を積む

X の値を `$0100 + SP` に書いて、SP から 1 を引きます。X もフラグも変わりません。6502 では、X を積むとき、`TXA` と `PHA` を使うので、A が壊れました。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`PHX`	`$DA`	1	3

使い方の例

```
        LDX #$07
        PHX
        LDX #$00
        PLX
```

X の `$07` を積んでから X を `$00` にして、`PLX` で `$07` に戻します。

(c) 2026 masanorisakai / MIT License

PHY(Push Y Register)Y を積む

Y の値をスタックに積みます。動作は PHX と同じで、対象が Y です。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`PHY`	`$5A`	1	3

使い方の例

```
        LDY #$33
        PHY
        LDY #$00
        PLY
        STY $03F0
```

Y の `$33` を積んで、Y を `$00` にしたあと、`PLY` で `$33` に戻します。`$03F0` には `$33` が書かれます。

> **Glow65 で確かめられること**
>
> ```
> *E 0300 A0 33 5A A0 00 7A 8C F0 03 60
> *G 0300
> *D 03F0 03F0
> 03F0: 33 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```

PLX(Pull X Register)X に取り出す

SP に 1 を足して、`$0100 + SP` の値を X に入れます。N と Z が、取り出した値で決まります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

(c) 2026 masanorisakai / MIT License

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`PLX`	`$FA`	1	4

使い方の例

`PHX` の例を見てください。`PLX` で X が `$07` に戻り、N と Z は 0 になります。

PLY(Pull Y Register)Y に取り出す

スタックから Y に取り出します。N と Z が、取り出した値で決まります。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`PLY`	`$7A`	1	4

使い方の例

`PHY` の例を見てください。`PLY` で Y が `$33` に戻ります。

STZ(Store Zero)メモリに 0 を書く

メモリに 0 を書きます。レジスタもフラグも変わりません。6502 では、メモリを 0 にするのに、A を `$00` にして `STA` を使うので、A が壊れました。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
ゼロページ	`STZ $nn`	`$64`	2	3
ゼロページ X	`STZ $nn,X`	`$74`	2	4
アブソリュート	`STZ $nnnn`	`$9C`	3	4
アブソリュート X	`STZ $nnnn,X`	`$9E`	3	5

使い方の例

(c) 2026 masanorisakai / MIT License

```
        LDA #$55
        STZ $03F2
```

`$03F2` が `$00` になります。A は `$55` のままです。実行例は、「Glow65 で 65C02 を動かす」にあります。

TRB(Test and Reset Bits)ビットを調べて 0 にする

A とメモリの値を AND して、結果が 0 のときだけ Z を 1 にします。結果そのものは捨てます。そのあと、メモリの値のうち、A で 1 のビットを 0 にして書き戻します。A は変わりません。

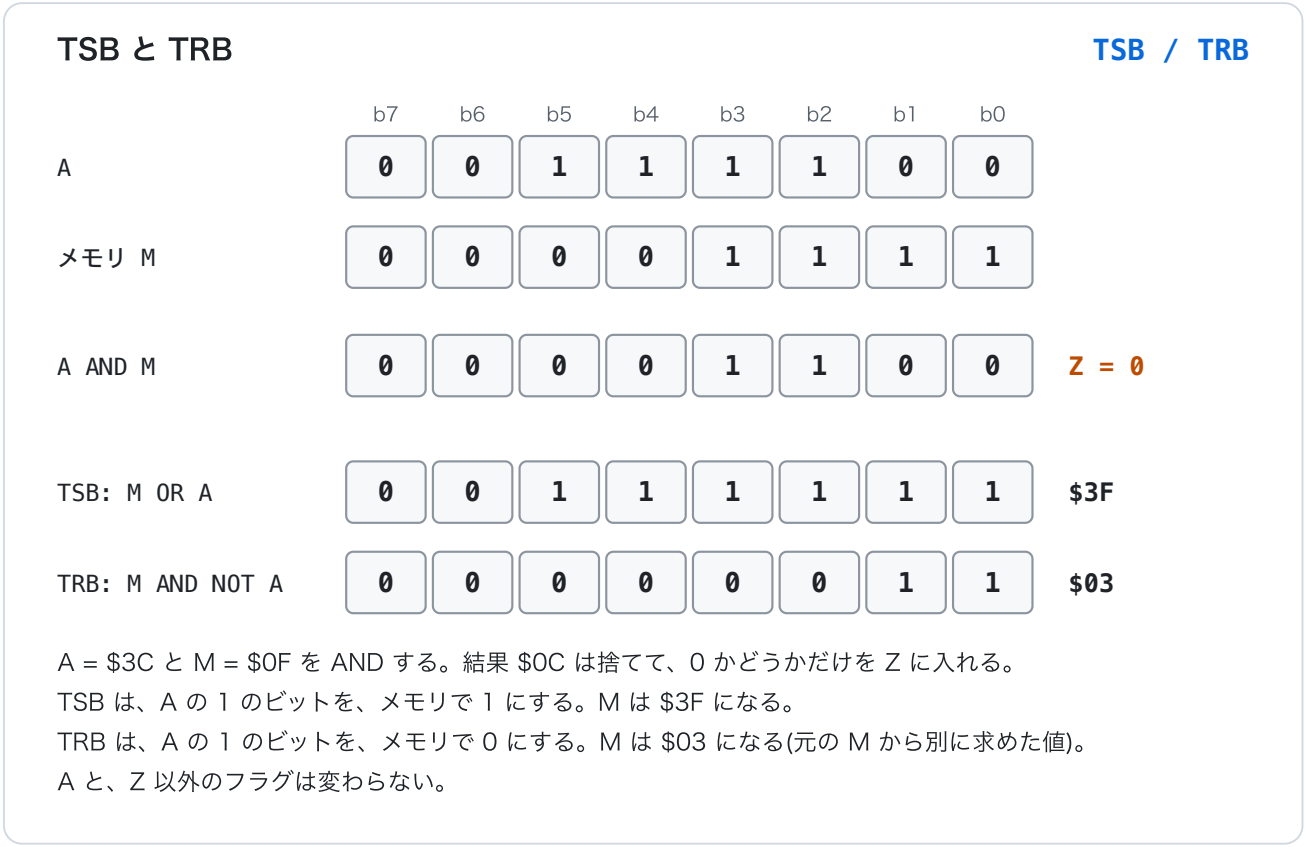


図 5-9 TSB と TRB。A の 1 のビットが、メモリの書き換え位置を決める。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
ゼロページ	`TRB $nn`	`$14`	2	5
アブソリュート	`TRB $nnnn`	`$1C`	3	6

(c) 2026 masanorisakai / MIT License

使い方の例

```
        LDA #$0F
        STA $20
        LDA #$3C
        TRB $20
```

`$20` の `$0F` と A の `$3C` を AND すると `$0C` で、0 ではないので Z は 0 です。A の 1 のビットを `$20` で 0 にして、`$20` は `$03` になります。

TSB(Test and Set Bits)ビットを調べて 1 にする

A とメモリの値を AND して、結果が 0 のときだけ Z を 1 にします。そのあと、メモリの値のうち、A で 1 のビットを 1 にして書き戻します。A は変わりません。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
ゼロページ	`TSB $nn`	`$04`	2	5
アブソリュート	`TSB $nnnn`	`$0C`	3	6

使い方の例

```
        LDA #$0F
        STA $20
        LDA #$3C
        TSB $20
```

`$20` は `$0F | $3C = $3F` になります。Z は 0 です。

TSB と TRB は、メモリの特定のビットを 1 つの命令で立てたり落としたりするときに使います。調べる命令と書き換える命令を、続けて書く必要がありません。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

`$20` に `$0F` を置き、A に `$3C` を入れて、`TSB` のあとの `$20` を `$03F0`、`TRB` のあとの `$20` を `$03F1` に書きます。

```
*E 0300 A9 0F 85 20 A9 3C 04 20 A6 20 8E F0 03 14 20 A6 20 8E F1 03 60
*L 0300 0310
0300  A9 0F     LDA   #$0F
0302  85 20     STA   $20
0304  A9 3C     LDA   #$3C
0306  04 20     TSB   $20
0308  A6 20     LDX   $20
030A  8E F0 03  STX   $03F0
030D  14 20     TRB   $20
030F  A6 20     LDX   $20
*G 0300
*D 03F0 03F1
03F0: 3F 03 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`TSB` のあとが `$3F`、続く `TRB` のあとが `$03` です。`TRB` は、`TSB` で立てたビットのうち、A の 1 のビットを落とします。

WAI(Wait for Interrupt)割り込みを待つ

実行を止めて、割り込みを待ちます。NMI か IRQ の信号が来ると、再開します。フラグは変わりません。

I が 0 のときは、割り込みを受け付けて、ハンドラを実行します。ハンドラが RTI で戻ると、WAI の次の命令から続きます。I が 1 のときは、割り込みを受け付けずに、WAI の次の命令へ進みます。

待っている間、CPU はバスに触れません。Glow65 では、パネルの HALT LED が点灯します。待ちの間、1 ステップは 1 サイクルです。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`WAI`	`$CB`	1	3

使い方の例

(c) 2026 masanorisakai / MIT License

```
        CLI
        WAI
        INX
```

I を 0 にして、割り込みを待ちます。割り込みが来てハンドラが戻ったあとで、`INX` が実行されます。

STP(Stop the Clock)停止

CPU を止めます。RESET を押すまで、再開しません。NMI や IRQ が来ても、再開しません。フラグは変わりません。

止まっている間、CPU はバスに触れません。Glow65 では、WAI と同じく、パネルの HALT LED が点灯します。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インプライド	`STP`	`$DB`	1	3

使い方の例

```
        LDA #$01
        STA $03F0
        STP
```

`$03F0` に書いたあと、CPU が止まります。

BBR(Branch on Bit Reset)ビットが 0 なら分岐

ゼロページの値のビット n を調べて、0 のときに分岐します。n は 0 から 7 で、`BBR0` から `BBR7` の 8 つのオペコードがあります。命令は 3 バイトで、オペコード、ゼロページの番地、分岐の距離の順に並びます。

距離は、次の命令の番地から測ります。ほかの分岐命令と同じです。フラグは変わりません。

調べるビットの番号 n は、オペコードのビット 4 から 6 に入っています。`$3F` は `%0011 1111` で、ビット 6 から 4 が `011` なので、`BBR3` です。

(c) 2026 masanorisakai / MIT License



図 5-10 BBR3。ゼロページ `$10` の値 `$24` のビット 3 は 0 なので、分岐する。

サイクル数は 5 です。分岐するとき 1 増え、別のページへ飛ぶとさらに 1 増えます。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
ゼロページ・リラティブ	`BBR0 $nn,label`	`$0F`	3	5 (+1,+2)
ゼロページ・リラティブ	`BBR1 $nn,label`	`$1F`	3	5 (+1,+2)
ゼロページ・リラティブ	`BBR2 $nn,label`	`$2F`	3	5 (+1,+2)
ゼロページ・リラティブ	`BBR3 $nn,label`	`$3F`	3	5 (+1,+2)
ゼロページ・リラティブ	`BBR4 $nn,label`	`$4F`	3	5 (+1,+2)
ゼロページ・リラティブ	`BBR5 $nn,label`	`$5F`	3	5 (+1,+2)
ゼロページ・リラティブ	`BBR6 $nn,label`	`$6F`	3	5 (+1,+2)
ゼロページ・リラティブ	`BBR7 $nn,label`	`$7F`	3	5 (+1,+2)

(c) 2026 masanorisakai / MIT License

使い方の例

```
        LDA #$00
        STA $10
        SMB3 $10
        LDX #$00
        BBS3 $10,yes
        BRA done
yes:    LDX #$AA
done:   STX $03F0
```

`SMB3` でゼロページ `$10` のビット 3 を 1 にしてから、`BBS3` で調べて `yes` へ分岐します。X は `$AA` になります。

BBR と BBS は、A も X も使わずに、ゼロページのビットを調べられます。

BBS(Branch on Bit Set)ビットが 1 なら分岐

ゼロページの値のビット n を調べて、1 のときに分岐します。`BBS0` から `BBS7` の 8 つのオペコードがあります。命令の長さ、距離の測り方、サイクル数、ビット番号の入り方は、BBR と同じです。BBR との違いは、オペコードのビット 7 が 1 であることです。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
ゼロページ・リラティブ	`BBS0 $nn,label`	`$8F`	3	5 (+1,+2)
ゼロページ・リラティブ	`BBS1 $nn,label`	`$9F`	3	5 (+1,+2)
ゼロページ・リラティブ	`BBS2 $nn,label`	`$AF`	3	5 (+1,+2)
ゼロページ・リラティブ	`BBS3 $nn,label`	`$BF`	3	5 (+1,+2)
ゼロページ・リラティブ	`BBS4 $nn,label`	`$CF`	3	5 (+1,+2)
ゼロページ・リラティブ	`BBS5 $nn,label`	`$DF`	3	5 (+1,+2)
ゼロページ・リラティブ	`BBS6 $nn,label`	`$EF`	3	5 (+1,+2)
ゼロページ・リラティブ	`BBS7 $nn,label`	`$FF`	3	5 (+1,+2)

使い方の例

(c) 2026 masanorisakai / MIT License

`BBR` の例を見てください。`BBS3 $10,yes` は、`$10` のビット 3 が 1 のときに、`yes` へ分岐します。

RMB(Reset Memory Bit)メモリのビットを 0 にする

ゼロページの値のビット n を 0 にします。n は 0 から 7 で、`RMB0` から `RMB7` の 8 つのオペコードがあります。ほかのビットは変わりません。フラグも変わりません。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
ゼロページ	`RMB0 $nn`	`$07`	2	5
ゼロページ	`RMB1 $nn`	`$17`	2	5
ゼロページ	`RMB2 $nn`	`$27`	2	5
ゼロページ	`RMB3 $nn`	`$37`	2	5
ゼロページ	`RMB4 $nn`	`$47`	2	5
ゼロページ	`RMB5 $nn`	`$57`	2	5
ゼロページ	`RMB6 $nn`	`$67`	2	5
ゼロページ	`RMB7 $nn`	`$77`	2	5

使い方の例

```
        LDA #$FF
        STA $10
        RMB0 $10
        RMB7 $10
```

`$10` は `$FF` から、ビット 0 とビット 7 が 0 になって、`$7E` になります。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

```
*E 0300 A9 FF 85 10 07 10 77 10 60
*L 0304 0306
0304  07 10     RMB0  $10
0306  77 10     RMB7  $10
*G 0300
*D 0010 0010
0010: 7E 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

SMB(Set Memory Bit)メモリのビットを 1 にする

ゼロページの値のビット n を 1 にします。`SMB0` から `SMB7` の 8 つのオペコードがあります。ほかのビットは変わりません。フラグも変わりません。

影響するフラグ

N	V	D	I	Z	C
-	-	-	-	-	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
ゼロページ	`SMB0 $nn`	`$87`	2	5
ゼロページ	`SMB1 $nn`	`$97`	2	5
ゼロページ	`SMB2 $nn`	`$A7`	2	5
ゼロページ	`SMB3 $nn`	`$B7`	2	5
ゼロページ	`SMB4 $nn`	`$C7`	2	5
ゼロページ	`SMB5 $nn`	`$D7`	2	5
ゼロページ	`SMB6 $nn`	`$E7`	2	5
ゼロページ	`SMB7 $nn`	`$F7`	2	5

使い方の例

`BBR` の例の `SMB3 $10` は、`$10` のビット 3 を 1 にします。`$10` が `$00` だったので、`$08` になります。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

`SMB3` で `$10` のビット 3 を 1 にして、`BBS3` で分岐し、X に `$AA` を入れて `$03F0` に書きます。

```
*E 0300 A9 00 85 10 B7 10 A2 00 BF 10 02 80 02 A2 AA 8E F0 03 60
*L 0304 030C
0304  B7 10     SMB3  $10
0306  A2 00     LDX   #$00
0308  BF 10 02  BBS3  $10,$030D
030B  80 02     BRA   $030F
*G 0300
*D 03F0 03F0
03F0: AA 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
*D 0010 0010
0010: 08 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

`BBS3` の距離 `$02` は、次の命令の番地 `$030B` に足されて、`$030D` になります。

6502 の命令に加わるモード

次の命令は、6502 にもありました。65C02 では、新しいモードを持ちます。

INC A

A に 1 を足します。6502 の `INC` は、メモリにしか使えませんでした。65C02 では、A に直接使えます。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
アキュムレータ	`INC A`	`$1A`	1	2

使い方の例

```
        LDA #$05
        INC A
```

A が `$06` になります。実行例は、「Glow65 で 65C02 を動かす」にあります。

(c) 2026 masanorisakai / MIT License

DEC A

A から 1 を引きます。

影響するフラグ

N	V	D	I	Z	C
○	-	-	-	○	-

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
アキュムレータ	`DEC A`	`$3A`	1	2

使い方の例

```
        LDA #$00
        DEC A
```

A が `$FF` になり、N が 1、Z が 0 になります。

ゼロページ・インダイレクト `($nn)`

ORA、AND、EOR、ADC、STA、LDA、CMP、SBC が、このモードを持ちます。ゼロページの 2 バイトを、実効アドレスとして使います。`($nn),Y` と違って、Y は足しません。

(c) 2026 masanorisakai / MIT License

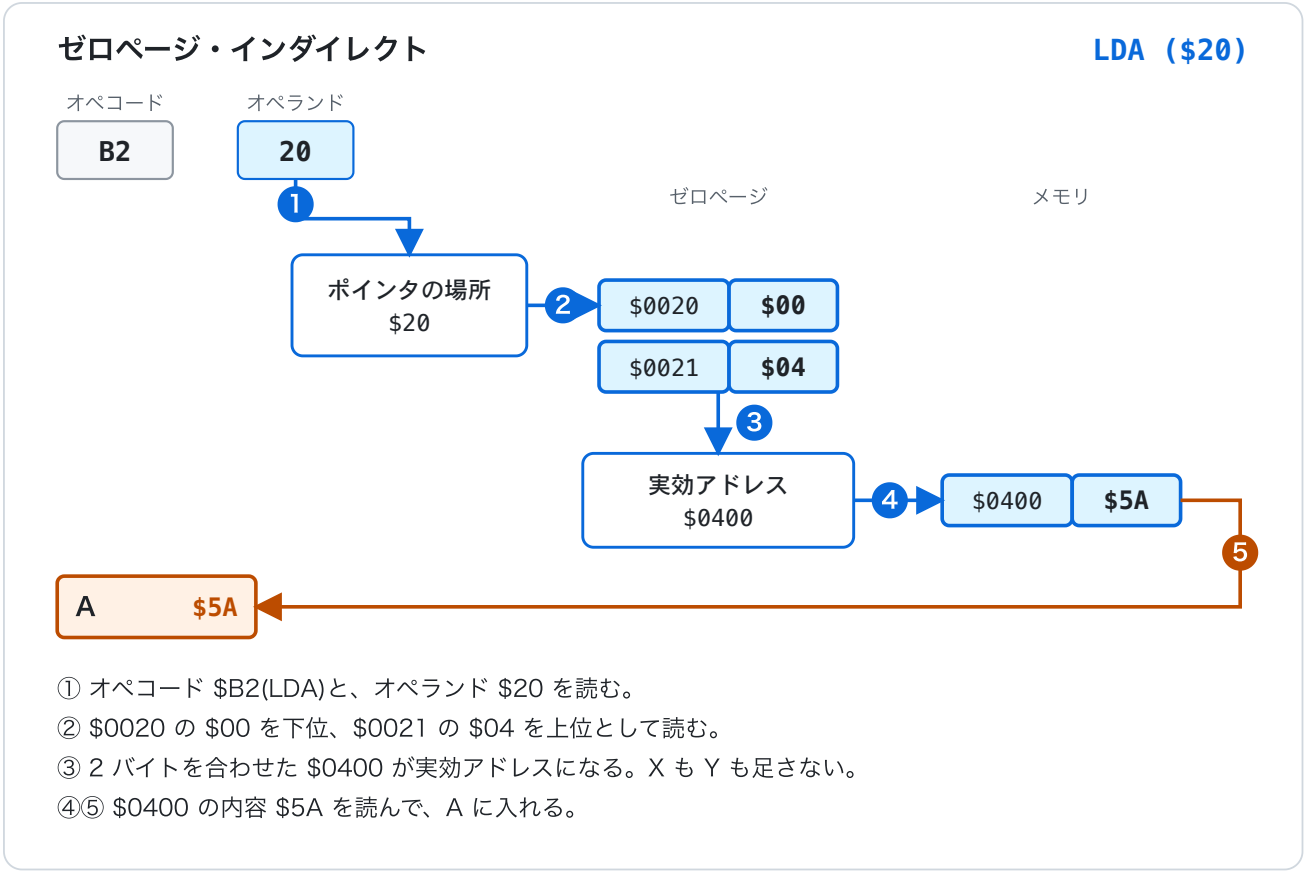


図 5-11 ゼロページ・インダイレクト(`LDA ($20)`)。実効アドレスは `$0400`。

フラグの変化と動作は、それぞれの命令と同じです。サイクル数は、どの命令も 5 です。ADC と SBC は、10進モードのとき 1 増えます。

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
ゼロページ・インダイレクト	`ORA ($nn)`	`$12`	2	5
ゼロページ・インダイレクト	`AND ($nn)`	`$32`	2	5
ゼロページ・インダイレクト	`EOR ($nn)`	`$52`	2	5
ゼロページ・インダイレクト	`ADC ($nn)`	`$72`	2	5
ゼロページ・インダイレクト	`STA ($nn)`	`$92`	2	5
ゼロページ・インダイレクト	`LDA ($nn)`	`$B2`	2	5
ゼロページ・インダイレクト	`CMP ($nn)`	`$D2`	2	5
ゼロページ・インダイレクト	`SBC ($nn)`	`$F2`	2	5

使い方の例

(c) 2026 masanorisakai / MIT License

```
        LDA ($20)
        STA $03F0
```

`$0020` に `00`、`$0021` に `04` が入っているとき、`LDA ($20)` は `$0400` を読みます。`$0400` が `$5A` なら、A は `$5A` になります。

6502 で、同じことをするには、Y を `$00` にして `LDA ($20),Y` と書く必要がありました。

> **Glow65 で確かめられること**
>
> ```
> *E 0020 00 04
> *E 0400 5A
> *E 0300 B2 20 8D F0 03 60
> *L 0300 0300
> 0300  B2 20     LDA   ($20)
> *G 0300
> *D 03F0 03F0
> 03F0: 5A 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```

BIT の新しいモード

BIT に、イミディエイト `#$nn`、ゼロページ X `$nn,X`、アブソリュート X `$nnnn,X` が加わります。

ゼロページ X とアブソリュート X は、BIT と同じように、N、V、Z を変えます。アブソリュート X は、別のページへ出ると 1 サイクル増えます。

イミディエイトの BIT は、Z だけを変えます。N と V は変わりません。値そのものが対象なので、メモリのビット 7 と 6 を写す意味がないためです。

影響するフラグ(ゼロページ X、アブソリュート X)

N	V	D	I	Z	C
M7	M6	-	-	○	-

影響するフラグ(イミディエイト)

N	V	D	I	Z	C
-	-	-	-	○	-

オペコード

(c) 2026 masanorisakai / MIT License

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
イミディエイト	`BIT #$nn`	`$89`	2	2
ゼロページ X	`BIT $nn,X`	`$34`	2	4
アブソリュート X	`BIT $nnnn,X`	`$3C`	3	4+

使い方の例

```
        LDA #$01
        BIT #$80
        PHP
        PLA
        STA $03F0
```

`$01` と `$80` を AND すると 0 なので、Z が 1 になります。N は 0 のままです。`BIT $10` で、メモリが `$80` のときは、N が 1 になります。

> **Glow65 で確かめられること**
>
> 前半は `BIT #$80`、後半は、`$10` に `$80` を置いた `BIT $10` です。どちらも A は `$01` で、AND の結果は 0 です。P を取り出して、`$03F0` と `$03F1` に書きます。
>
> ```
> *E 0300 A9 01 89 80 08 68 8D F0 03 A9 80 85 10 A9 01 24 10 08
> *E 0312 68 8D F1 03 60
> *G 0300
> *D 03F0 03F1
> 03F0: 36 B6 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```
>
> `$36` は `%00110110` で、Z(ビット 1)が 1、N(ビット 7)は 0 です。`$B6` は `%10110110` で、N も 1 です。

JMP のアブソリュート X・インダイレクト `($nnnn,X)`

`$nnnn + X` の番地にある 2 バイトを、飛び先として使います。ジャンプテーブル(飛び先の番地を並べた表)から、X で選んだ項目へ飛べます。番地は 2 バイトなので、X は 2 ずつ増やします。サイクル数は 6 です。

`JMP ($nnnn)` は、動作が 1 つ変わっています。上位バイトを、ポインタの次の番地から読みます。6502 では、ポインタがページの最後(`$xxFF`)にあると、同じページの先頭から読んでいました。サイクル数は 5 から 6 になります。

影響するフラグ

フラグは変わりません。

(c) 2026 masanorisakai / MIT License

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
インダイレクト	`JMP ($nnnn)`	`$6C`	3	6
アブソリュート X・インダイレクト	`JMP ($nnnn,X)`	`$7C`	3	6

使い方の例

```
        LDX #$02
        JMP ($0340,X)
```

X が `$02` なので、`$0342` と `$0343` の 2 バイトが飛び先です。`$0340` から、`$0320`、`$0330` の順に表を置くと、`$0330` へ飛びます。

> **Glow65 で確かめられること**
>
> `$0340` に 2 つの番地(`$0320`、`$0330`)を置き、それぞれのコードが `$03F0` に `$11` と `$22` を書きます。X は `$02` なので、2 つ目の `$0330` へ飛びます。
>
> ```
> *E 0340 20 03 30 03
> *E 0330 A9 22 8D F0 03 60
> *E 0320 A9 11 8D F0 03 60
> *E 0300 A2 02 7C 40 03
> *L 0300 0302
> 0300  A2 02     LDX   #$02
> 0302  7C 40 03  JMP   ($0340,X)
> *G 0300
> *D 03F0 03F0
> 03F0: 22 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```

ASL、LSR、ROL、ROR のアブソリュート X

動作は変わりません。サイクル数が、7 から 6 に減ります。別のページへ出ると、1 増えて 7 になります。

オペコード

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
アブソリュート X	`ASL $nnnn,X`	`$1E`	3	6+
アブソリュート X	`LSR $nnnn,X`	`$5E`	3	6+
アブソリュート X	`ROL $nnnn,X`	`$3E`	3	6+

(c) 2026 masanorisakai / MIT License

アドレッシングモード	書き方	オペコード	バイト数	サイクル数
アブソリュート X	`ROR $nnnn,X`	`$7E`	3	6+

未定義だったオペコード

6502 の未定義のオペコードは、65C02 では、すべて NOP になります。長さとサイクル数は、オペコードごとに決まっています。レジスタもフラグも変わりません。メモリは読みません。

バイト数	サイクル数	オペコード
1	1	`$03`, `$0B`, `$13`, `$1B`, `$23`, `$2B`, `$33`, `$3B`, `$43`, `$4B`, `$53`, `$5B`, `$63`, `$6B`, `$73`, `$7B`, `$83`, `$8B`, `$93`, `$9B`, `$A3`, `$AB`, `$B3`, `$BB`, `$C3`, `$D3`, `$E3`, `$EB`, `$F3`, `$FB`
2	2	`$02`, `$22`, `$42`, `$62`, `$82`, `$C2`, `$E2`
2	3	`$44`
2	4	`$54`, `$D4`, `$F4`
3	4	`$DC`, `$FC`
3	8	`$5C`

10進モードの違い

10進モードの `ADC` と `SBC` で、N と Z の決まり方が違います。次のプログラムは、`$99 + $01` を 10進モードで足します。

```
        SED
        CLC
        LDA #$99
        ADC #$01
        PHP
        STA $03F0
        PLA
        STA $03F1
        CLD
```

答えは `$00` で、C が 1 になります。結果が 0 なので、Z は 1 になるのが自然です。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

A の値を `$03F0` に、`ADC` 直後の P を `$03F1` に書きます。

65C02:

```
*E 0300 F8 18 A9 99 69 01 08 8D F0 03 68 8D F1 03 D8 60
*G 0300
*D 03F0 03F1
03F0: 00 3F 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

6502:

```
*E 0300 F8 18 A9 99 69 01 08 8D F0 03 68 8D F1 03 D8 60
*G 0300
*D 03F0 03F1
03F0: 00 BD 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

どちらも A は `$00` です。65C02 の P は `$3F`(`%00111111`)で、N が 0、Z が 1、C が 1 です。6502 の P は `$BD`(`%10111101`)で、N が 1、Z が 0、C が 1 です。6502 では、A が `$00` なのに Z が 0 になります。

練習問題

答えと解き方は、次の節にあります。

問1 次の命令を機械語(16進数のバイト列)に直してください。(1) `STZ $0310` (2) `INC A` (3) `LDA ($30)` (4) `BBS5 $40,label`(`label` は、この命令の次の命令の番地から 2 バイト先)

問2 `$0310` に `80 F4` があります。65C02 での飛び先はどこですか。

問3 ゼロページ `$10` に `$81`(`%10000001`)が入っています。`BBR0`、`BBR1`、`BBS6`、`BBS7` のうち、分岐するのはどれですか。

問4 A が `$0F`、ゼロページ `$21` が `$F0` のとき、`TSB $21` と `TRB $21` を、それぞれ実行します。実行後の `$21` と Z を答えてください。2 つの命令は、それぞれ `$F0` の状態から実行するものとします。

問5 ゼロページ `$30` に `00`、`$31` に `05` が入っています。`$0500` は `$7E`、`$0501` は `$9C` です。Y が `$01` のとき、`LDA ($30)` と `LDA ($30),Y` で A に入る値は、それぞれ何ですか。

問6 バイト列 `A9 05 1A 8D F0 03 60` を `$0300` に置いて実行します。`$03F0` に書かれる値は、65C02 と 6502 で、それぞれ何ですか。

(c) 2026 masanorisakai / MIT License

練習問題の答えと解き方

問1 (1) `9C 10 03` (2) `1A` (3) `B2 30` (4) `DF 40 02`

- (1) `STZ` のアブソリュートは `$9C`。番地は、下位から先に並べて `10 03`。(2) `INC A` は 1 バイトで `$1A`。(3) ゼロページ・インダイレクトの `LDA` は `$B2`。(4) `BBS5` は `$DF`(ビット 4 から 6 が `101`、ビット 7 が 1)。続けて、ゼロページの番地 `40`、距離 `02` を並べる。

問2 `$0306`

- `80 F4` は 2 バイトの命令なので、読み終えた PC は `$0312`。`$F4` は -12。`$0312 - 12 = $0306`。

問3 `BBR1` と `BBS7`

- `$81` は、ビット 7 と 0 が 1 で、ほかが 0。`BBR0` は、ビット 0 が 1 なので分岐しない。`BBR1` は、ビット 1 が 0 なので分岐する。`BBS6` は、ビット 6 が 0 なので分岐しない。`BBS7` は、ビット 7 が 1 なので分岐する。

問4 `TSB $21` は `$FF` で、Z は 1。`TRB $21` は `$F0` のままで、Z は 1。

- どちらも、AND の結果は `$0F AND $F0 = $00` なので、Z は 1。TSB は A の 1 のビット(下位 4 ビット)を立てるので、`$F0 OR $0F = $FF`。TRB は A の 1 のビットを落とすが、`$F0` の下位 4 ビットは、すでに 0 なので変わらない。

問5 `LDA ($30)` は `$7E`。`LDA ($30),Y` は `$9C`。

- `$30` と `$31` から、基準の番地は `$0500`。`($30)` は、Y を足さないので `$0500` を読み、`$7E`。`($30),Y` は、Y の `$01` を足して `$0501` を読み、`$9C`。

問6 65C02 は `$06`。6502 は `$05`。

- 65C02 では、`1A` が `INC A` なので、A が `$05` から `$06` になる。6502 では、`1A` が未定義のオペコードで、何もしないので、A は `$05` のまま。

(c) 2026 masanorisakai / MIT License

第6章 フラグ、スタック、割り込み

第3章から第5章で、レジスタと命令を見ました。この章では、それらの裏で働く 3 つの仕組みを説明します。結果を覚えておく**フラグ**、戻り番地や一時的な値をしまう**スタック**、実行中のプログラムを中断して別の処理に移る**割り込み**です。最後に、動き出しの条件を決める**リセット**を説明します。

手を動かして試す割り込みの演習は、チュートリアル第6章にあります。この章は、その背景と全体像です。

6.1 フラグ(P レジスタ)

P レジスタは 8 ビットで、1 ビットずつが**フラグ**です。実体のあるフラグは、N、V、D、I、Z、C の 6 つです。ビット 5 とビット 4(B)は、P の中に実体がありません。



図 6-1 P レジスタの 8 ビット。

6.1.1 N と Z(結果のフラグ)

N(ネガティブ)は、直前の結果の最上位ビット(ビット 7)のコピーです。符号付きの数として読むと、1 なら負です。**Z**(ゼロ)は、結果が 0 のとき 1 になります。

ロード、転送、足し算、論理演算、シフト、増減の命令が、結果に応じて N と Z を変えます。ストアと分岐、ジャンプの命令は、変えません。第3章 3.6 の例が、この続きです。

6.1.2 C(キャリー)

C は、8 ビットに収まらなかった桁を持つフラグです。命令ごとに、次のように使われます。

(c) 2026 masanorisakai / MIT License

命令	C の働き
ADC	結果が `$FF` を超えると 1。足し算のときは、C も足される
SBC	引き算で借りがないと 1。借りがあると 0。引き算の前に SEC で 1 にする
ASL、ROL	最上位ビットが C に出る
LSR、ROR	最下位ビットが C に出る
ROL、ROR	元の C が、空いたビットに入る
CMP、CPX、CPY	レジスタが値以上のとき 1

16 ビットの足し算では、下位バイトの ADC が出した C を、上位バイトの ADC が受け取ります。この受け渡しは、5.3 の 16 ビットの例と、チュートリアル第2章にあります。

> **Glow65 で確かめられること**
>
> `$FF + $01`、`$30 - $50`、`$50 - $30` のあとの P を、`$03F0` から順に書きます。
>
> ```
> *E 0300 18 A9 FF 69 01 08 68 8D F0 03 38 A9 30 E9 50 08
> *E 0310 68 8D F1 03 38 A9 50 E9 30 08 68 8D F2 03 60
> *G 0300
> *D 03F0 03F2
> 03F0: 37 B4 35 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```
>
> `$37` は C と Z が 1(`$FF + $01` が 0 になり、桁があふれた)、`$B4` は C が 0 で N が 1(`$30 - $50` で借りが出た)、`$35` は C が 1(`$50 - $30` は借りなし)です。

6.1.3 V(オーバーフロー)

V は、符号付きの数として計算したときの桁あふれを示します。8 ビットの符号付きの数は、-128 から 127 です。同じ符号の 2 つの数を足して、結果の符号が変わると、V が 1 になります。

計算	結果	符号付きで読むと	V
`$50 + $50`	`$A0`	80 + 80 = 160(範囲を超える)	1
`$7F + $01`	`$80`	127 + 1 = 128(範囲を超える)	1
`$50 + $10`	`$60`	80 + 16 = 96(範囲内)	0
`$80 + $80`	`$00`	-128 + -128 = -256(範囲を超える)	1

符号なしの数として見るときは C、符号付きの数として見るときは V を見ます。同じ足し算でも、2 つのフラグは別の意味を持ちます。たとえば `$80 + $80` は、符号なしの 128 + 128 = 256 で C も 1 です。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

4 つの足し算の P を、`$03F0` から順に書きます。足す前に `CLC` で C を 0 にします。

```
*E 0300 18 A9 50 69 50 08 68 8D F0 03 18 A9 7F 69 01 08
*E 0310 68 8D F1 03 18 A9 50 69 10 08 68 8D F2 03 18 A9
*E 0320 80 69 80 08 68 8D F3 03 60
*G 0300
*D 03F0 03F3
03F0: F4 F4 34 77 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`$F4` は N と V が 1、`$34` は N と V が 0 です。`$77` は V、Z、C が 1 で、N は 0 です(`$80 + $80` は `$00` に C を出す)。

6.1.4 D(10進モード)

D が 1 のとき、ADC と SBC は、各 4 ビットを 0 から 9 の数字として計算します(BCD)。`$09 + $01` は、`$0A` ではなく `$10` になります。例は、5.3 の 10進モードの説明にあります。

Glow65 の CPU は、既定で NMOS 6502 の動作をします。NMOS 6502 では、10進モードでの N、V、Z は、結果と一致しない場合があります。また、NMOS 6502 は、割り込みを受け付けても、D を変えません。ハンドラで足し算や引き算を使うなら、ハンドラの先頭で CLD を実行します。

Glow65 で確かめられること

`SED` のあと、P を `$03F0` に書き、`CLD` のあと、もう一度 `$03F1` に書きます。

```
*E 0300 F8 08 68 8D F0 03 D8 08 68 8D F1 03 60
*G 0300
*D 03F0 03F1
03F0: 3C 34 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

D はビット 3 です。`$3C`(`%00111100`)は D が 1、`$34`(`%00110100`)は D が 0 です。

6.1.5 I(割り込み禁止)

I が 1 の間、CPU は IRQ を受け付けません。NMI は、I に関係なく受け付けます。SEI で 1 に、CLI で 0 にします。リセットのあとは 1 で、割り込みを受け付けた直後も 1 です。

6.1.6 B とビット 5

B(ビット 4)は、P の中にあるフラグではありません。PHP と BRK が P をスタックに積むときだけ、B が 1 になります。IRQ と NMI が積む P では、B は 0 です。ハンドラは、積まれた P の B を見て、BRK から

(c) 2026 masanorisakai / MIT License

来たのか、IRQ から来たのかを区別できます。

ビット 5 は、いつも 1 です。PLP と RTI は、取り出した値の B を取り込まず、ビット 5 を 1 のままにします。図 6-1 の下の 2 つの行が、積む値の違いです。

6.1.7 条件分岐が見るフラグ

条件分岐の命令は、フラグを 1 つ見て、分岐するかを決めます。

命令	見るフラグ	分岐する条件
BCC	C	0
BCS	C	1
BEQ	Z	1
BNE	Z	0
BMI	N	1
BPL	N	0
BVC	V	0
BVS	V	1

6.1.8 比較(CMP)のあと

CMP は、A から値を引いた結果を捨てて、フラグだけを変えます。A が値以上のとき C が 1、等しいとき Z が 1 です。N は、A - 値の結果の最上位ビットです。

CMP の結果とフラグ

A = $30 のとき、M を変えて比べた結果

		C	Z	N	使う分岐
A < M	CMP #$40	0	0	1	BCC(C = 0)
A = M	CMP #$30	1	1	0	BEQ(Z = 1)
A > M	CMP #$20	1	0	0	BCS(C = 1)と BNE(Z = 0)

C は A が M 以上のとき 1。Z は A と M が等しいとき 1。N は A – M の結果の最上位ビット。
N は、符号付きの大小を直接表すものではない。比較は、符号なしの数として行われる。

図 6-2 CMP のあとのフラグと分岐。

(c) 2026 masanorisakai / MIT License

A が値より小さいとき C が 0、等しいとき Z が 1、大きいとき C が 1 で Z が 0 です。符号なしの数の大小は、C と Z で判断します。N は、符号付きの大小を直接表しません。

> **Glow65 で確かめられること**
>
> `LDA #$30` のあと、`CMP #$30`、`CMP #$40`、`CMP #$20` の P を、`$03F0` から順に書きます。
>
> ```
> *E 0300 A9 30 C9 30 08 68 8D F0 03 A9 30 C9 40 08 68 8D
> *E 0310 F1 03 A9 30 C9 20 08 68 8D F2 03 60
> *G 0300
> *D 03F0 03F2
> 03F0: 37 B4 35 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```
>
> `$37` は C と Z が 1(等しい)、`$B4` は C が 0(小さい)、`$35` は C が 1 で Z が 0(大きい)です。

6.2 スタック

6.2.1 場所と SP

スタックは、メモリの `$0100` から `$01FF` の 256 バイトです。後から入れた値を、先に取り出します(後入れ先出し)。SP(スタックポインタ)は 8 ビットで、`$0100` に SP を足した番地が、次に書く場所です。

積むときは、その場所に書いてから、SP を 1 減らします。取り出すときは、SP に 1 を足してから、その番地を読みます。スタックは、大きい番地から小さい番地へ伸びます。図は、5.6 の図 5-6 にあります。

Glow65 の CPU は、リセットで SP を `$FD` にします。モニタは、リセットの直後に SP を `$FF` にします。モニタの `G` で呼んだプログラムは、戻り番地(2 バイト)が積まれたあとの `$FD` から始まります。

6.2.2 スタックに積まれるもの

動作	積むもの	取り出す命令
PHA	A(1 バイト)	PLA
PHP	P(1 バイト、B とビット 5 が 1)	PLP
JSR	戻り番地(2 バイト。JSR の最後のバイトの番地。上位が先)	RTS(1 を足して PC に入れる)
BRK、IRQ、NMI	PC の上位、PC の下位、P(3 バイト)	RTI

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

3 つの値を `PHA` で積み、`TSX` で SP を読みます。スタックページを `$0500` にコピーして、`PLA` で 3 つ取り出します。

```
*E 0300 A9 11 48 A9 22 48 A9 33 48 BA 8E F0 03 A2 0F BD
*E 0310 F0 01 9D 00 05 CA 10 F7 68 8D F1 03 68 8D F2 03
*E 0320 68 8D F3 03 60
*G 0300
*D 03F0 03F3
03F0: FA 33 22 11 00 00 00 00 24 F4 24 F4 00 00 00 00
*D 0500 050F
0500: 00 00 00 00 00 00 00 00 00 00 00 33 22 11 37 F2
```

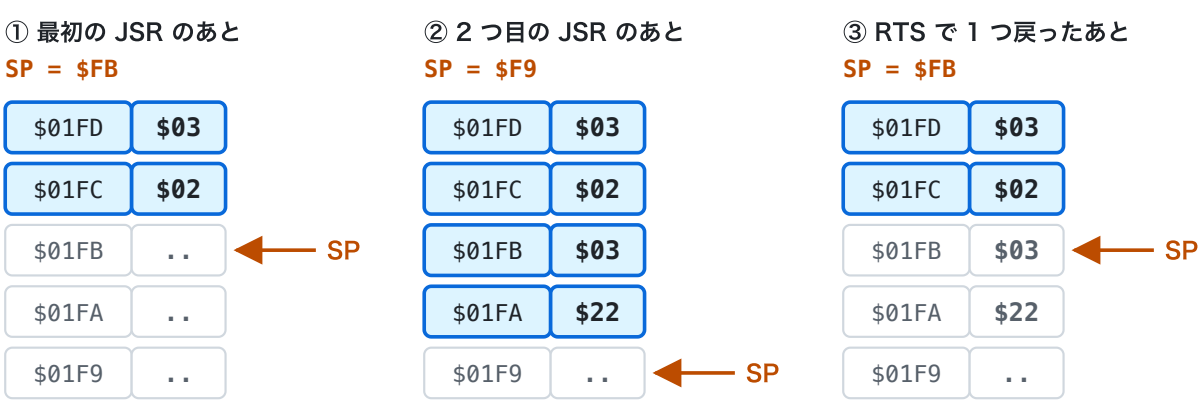
`$03F0` は積んだあとの SP で、`$FA` です。`$03F1` から `$03F3` は、取り出した順に `33`、`22`、`11` で、積んだ順の逆です。`$0500` のコピーでは、`$050B` から `$050D` が `$01FB` から `$01FD` にあたり、`33 22 11` が下の番地から並んでいます。続く `37 F2` は、モニタの `G` が積んだ戻り番地です。

6.2.3 JSR を重ねたとき

サブルーチンの中から、別のサブルーチンを呼べます。戻り番地は、呼ぶたびにスタックに積まれ、RTS のたびに、新しいほうから取り出されます。

ネストした JSR とスタック

SP = $FD から始める



$01FD と $01FC は、最初の JSR が積んだ戻り番地($0302)。上位が先、下位が次。
$01FB と $01FA は、2 つ目の JSR が積んだ戻り番地($0322)。SP は 2 つずつ下がる。
RTS は、SP を戻して番地を取り出す。取り出した値は、メモリに残るが、次の PHA や JSR で上書きされる。

図 6-3 JSR を 2 回重ねたときのスタック。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

`$0300` のプログラムが `$0320` のサブルーチンを呼び、`$0320` が `$0330` を呼びます。`$0330` は SP を `$03F0` に書き、スタックページを `$0500` にコピーします。

```
*E 0320 20 30 03 60
*E 0330 BA 8E F0 03 A2 0F BD F0 01 9D 00 05 CA 10 F7 60
*E 0300 20 20 03 60
*G 0300
*D 03F0 03F0
03F0: F9 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
*D 0500 050F
0500: 00 00 00 00 00 00 00 00 00 00 22 03 02 03 37 F2
```

`$03F0` の `F9` は、2 つ目の JSR のあとの SP です。コピーの `$050A` から `$050D` が `22 03 02 03` で、`$01FA` から `$01FD` にあたります。上位が `$01FD` の `03`、下位が `$01FC` の `02` で、最初の JSR の戻り番地 `$0302`、`$01FB` と `$01FA` が 2 つ目の `$0322` です。

6.2.4 SP の折り返し

SP は 8 ビットなので、`$00` の次は `$FF` に戻ります。スタックは、`$0100` から `$01FF` の中で回り、ほかのページには出ません。積みすぎると、古い値が上書きされます。

Glow65 で確かめられること

SP を `$00` にして `PHA` を実行します。`$03F0` に元の SP、`$03F1` に積んだあとの SP を書き、`$0100` を読みます。

```
*E 0300 BA 8E F0 03 A2 00 9A A9 AA 48 BA 8E F1 03 AE F0
*E 0310 03 9A 60
*G 0300
*D 03F0 03F1
03F0: FD FF 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
*D 0100 0100
0100: AA 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

`$03F0` の `FD` は元の SP、`$03F1` の `FF` は積んだあとの SP です。A の `$AA` は、`$0100` に書かれています。

(c) 2026 masanorisakai / MIT License

6.2.5 TSX と TXS

TSX は SP を X に読み、TXS は X を SP に書きます。SP を書き換えると、スタックの位置が変わります。モニタの下で動かすプログラムは、SP を変えたら、モニタに戻る前に元の値へ戻します。

`$0100` から `$01FF` は、スタック専用にします。ここに変数を置くと、積んだ値で壊れます。Glow65 では、`$0200` から `$027F` がモニタの行入力バッファ、`$0300` から `$03F7` が自作プログラム用の領域です。

6.3 割り込み

6.3.1 4 つの割り込み

CPU が、実行中のプログラムを中断して処理を移す原因は、4 つあります。

名前	発生源	I でマスク	トリガ	ベクタ	積む P の B
NMI	パネルの NMI ボタン	されない	エッジ(変化の 1 回)	`$FFFA` `$FFFB`	0
IRQ	UART の受信割り込み	される	レベル(要求が続く間)	`$FFFE` `$FFFF`	0
BRK	BRK 命令	されない(必ず実行する)	なし	`$FFFE` `$FFFF`	1
リセット	RESET スイッチ	されない	なし	`$FFFC` `$FFFD`	積まない

ベクタは、飛び先の番地を 2 バイトで持つ場所です(下位が先)。BRK と IRQ は、同じベクタを共有します。

6.3.2 受け付けの手順

割り込みの要求は、実行中の命令が終わってから受け付けます[1]。受け付けた CPU は、次のことを自動で行います。

1. PC の上位、PC の下位、P の順に、スタックに積む。
2. I を 1 にする。
3. ベクタの番地を読んで、PC に入れる。
4. ハンドラ(割り込み処理のプログラム)が始まる。

積まれる PC は、IRQ と NMI では次に実行する命令の番地、BRK では BRK の番地に 2 を足した番地です。ハンドラは、RTI で終わります。RTI は、P と PC を取り出し、PC に 1 を足さずに戻ります。フラグも元に戻ります。

実機の 6502 は、受け付けに 7 サイクルかけます。Glow65 は、余分なダミーリードを再現しないので、バスアクセスは 5 回に見えます。

(c) 2026 masanorisakai / MIT License

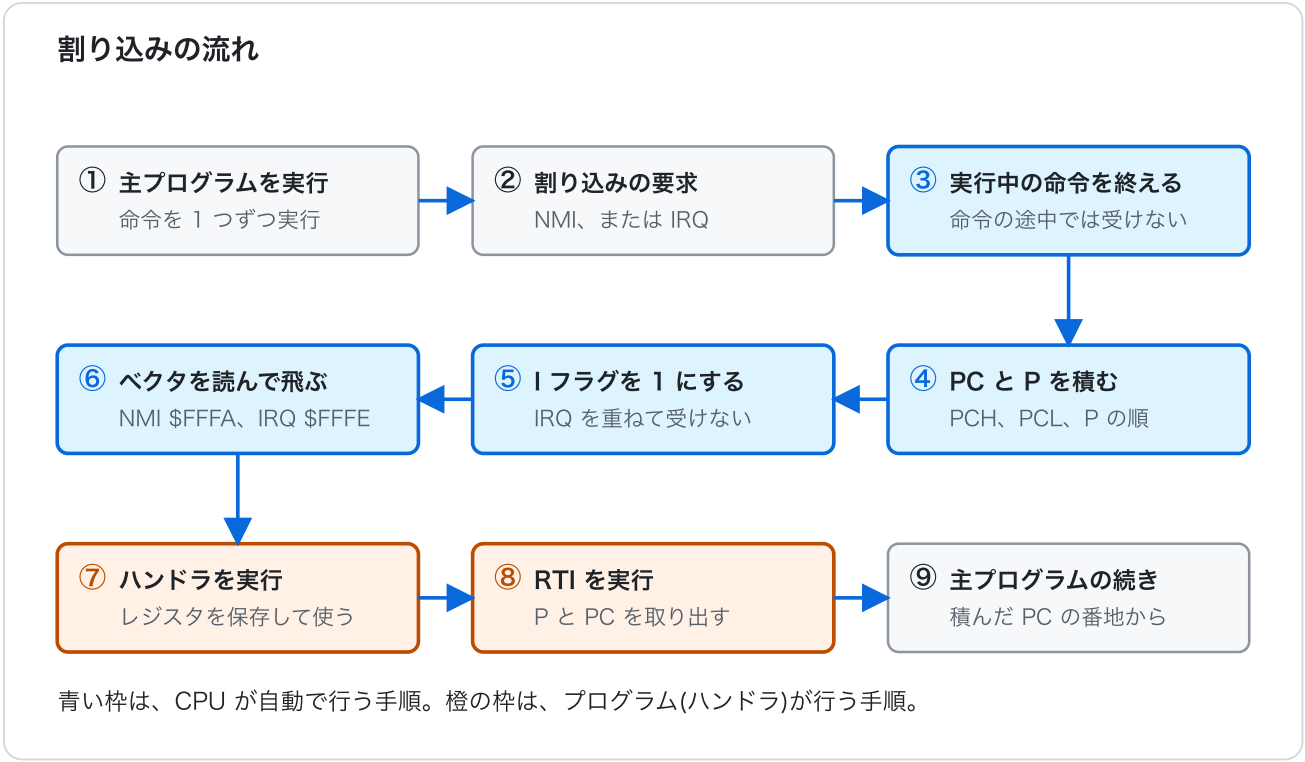


図 6-4 割り込みの流れ。

6.3.3 NMI と IRQ の違い

NMI は、要求が「なし」から「あり」に変わった 1 回ごとに、1 回だけ受け付けます。I フラグに関係なく受け付けます。

IRQ は、要求が続いている間、I が 0 なら、何度でも受け付けます。ハンドラが要因を消さずに RTI で戻ると、すぐにまた受け付けます。RTI が P を戻すので、I も 0 に戻るからです。

(c) 2026 masanorisakai / MIT License

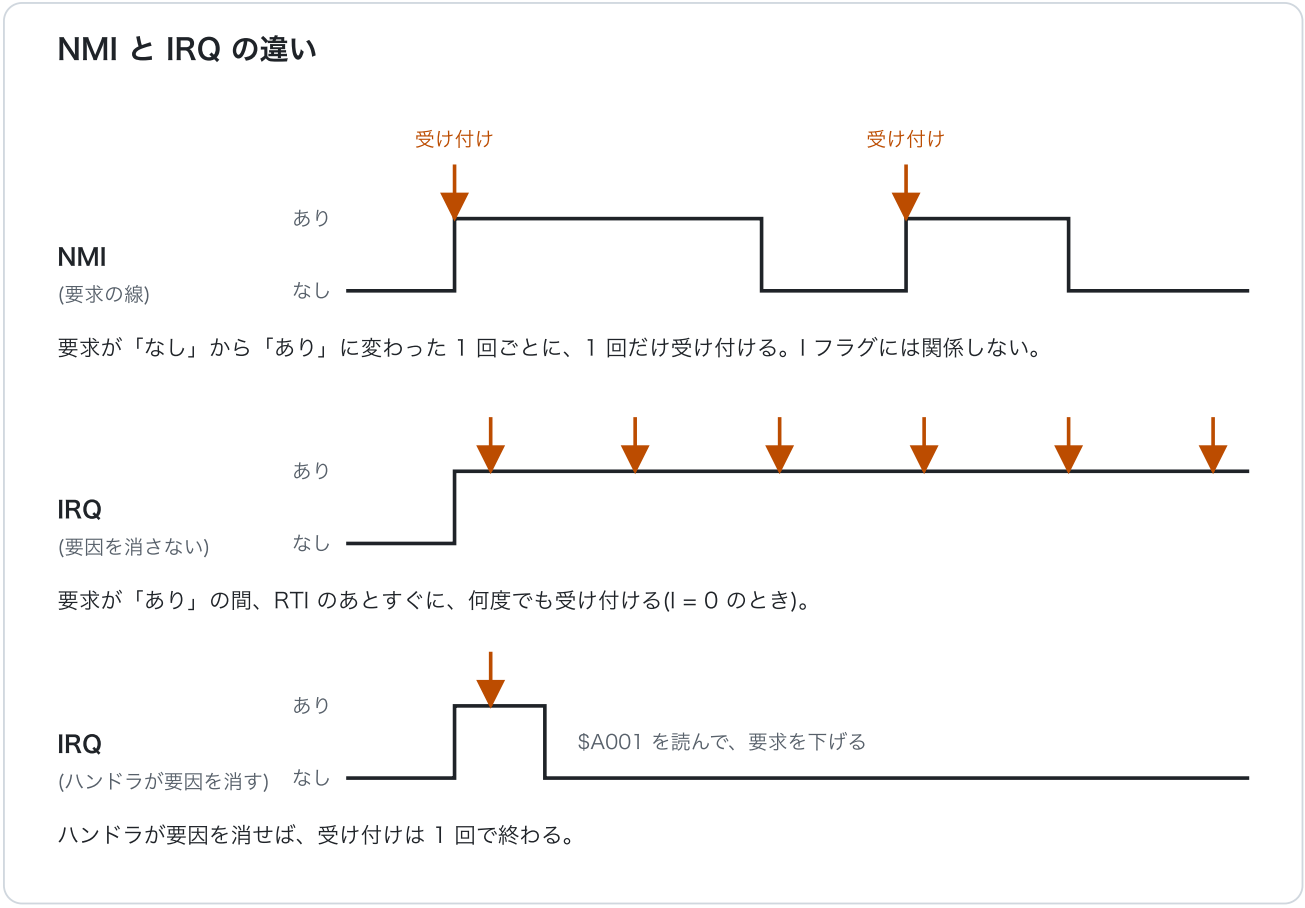


図 6-5 NMI と IRQ の違い。

Glow65 で確かめられること

UART の受信割り込みを許可して、1 文字を送ります。ハンドラは、`$A001` を読まずに、`$03F2` を 1 ずつ増やして RTI で戻ります。5 回目に、`$A001` を読んで、割り込みを止めます。

```
*E 0340 EE F2 03 AD F2 03 C9 05 D0 08 AD 01 A0 A9 00 8D
*E 0350 00 A0 40
*E 0300 78 A9 40 8D FA 03 A9 03 8D FB 03 A9 00 8D F2 03
*E 0310 8D F1 03 A9 80 8D 00 A0 58 AD F2 03 C9 05 D0 F9
*E 0320 78 A9 00 8D 00 A0 60
*G 0300
*D 03F2 03F2
03F2: 05 00 00 00 00 00 24 F4 40 03 00 00 00 00 00 00
```

1 文字の受信で、ハンドラが 5 回呼ばれました。要因(未読の受信データ)が残っていたからです。

(c) 2026 masanorisakai / MIT License

6.3.4 BRK

BRK は、プログラムから割り込みを起こす命令です。IRQ のベクタを使い、I でマスクされません。BRK の次の 1 バイトは、署名バイトです。CPU は、このバイトを使わずに飛ばします(積まれる PC は、BRK の番地 + 2)。ハンドラが、積まれた PC を読めば、署名バイトの番地は PC の 1 つ前です。

> **Glow65 で確かめられること**
>
> BRK のハンドラが、スタックに積まれた P を `$03F0` に書きます。
>
> ```
> *E 03FA 40 03
> *E 0340 BA BD 01 01 8D F0 03 40
> *E 0300 A2 00 A0 00 00 99 A0 55 60
> *G 0300
> *D 03F0 03F0
> 03F0: 36 00 00 00 00 00 00 00 24 F4 40 03 00 00 00 00
> ```
>
> `$36` は `%00110110` で、B(ビット 4)が 1 です。Z と I も 1 です。

6.3.5 ハンドラの書き方

CPU が保存するのは、PC と P だけです。ハンドラが A、X、Y を使うなら、先頭で保存し、RTI の前に戻します。戻すときは、積んだ逆の順に取り出します。

```
handler:
        pha             ; A を保存
        txa
        pha             ; X を保存
        ...             ; 処理
        pla
        tax             ; X を戻す
        pla             ; A を戻す
        rti
```

次の例は、UART の受信割り込みで呼ばれるハンドラです。積まれた P を `$03F0` に、ハンドラの中の I を `$03F2` に書きます。主プログラムは、`$03F1` が 1 になるまで待ちます。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

1 文字を送ると、ハンドラが呼ばれます。

```
*E 0340 48 8A 48 BA BD 03 01 8D F0 03 08 68 29 04 8D F2
*E 0350 03 AD 01 A0 A9 01 8D F1 03 68 AA 68 40
*E 0300 78 A9 40 8D FA 03 A9 03 8D FB 03 A9 00 8D F1 03
*E 0310 A9 80 8D 00 A0 58 AD F1 03 F0 FB 78 A9 00 8D 00
*E 0320 A0 60
*G 0300
*D 03F0 03F2
03F0: 22 01 04 00 00 00 00 00 24 F4 40 03 00 00 00 00
```

`$03F0` の `22` は、積まれた P です。B(ビット 4)が 0、I(ビット 2)が 0、ビット 5 が 1、Z が 1 です。`$03F1` の `01` は、ハンドラが立てた終了の印です。`$03F2` の `04` は、ハンドラの中の I が 1 だったことを示します。

6.3.6 Glow65 の割り込み

ベクタは、ROM の `$FFFA` から `$FFFF` にあり、書き換えられません。モニタは、NMI と IRQ(と BRK)を、RAM の 2 つのアドレスを経由して、ユーザーのハンドラへ飛ばします。

RAM のアドレス	内容
`$03F8` `$03F9`	NMI ハンドラの番地(下位が先)
`$03FA` `$03FB`	IRQ と BRK のハンドラの番地(下位が先)

モニタは、リセットのとき、この 2 つを、RTI だけのルーチンに向けます。自分のハンドラを使うときは、番地を書き込みます。

UART の割り込みは、制御レジスタ(`$A000` に書く)のビット 7 で許可します。許可されていて、未読の受信データがある間、IRQ の要求が出ます。状態レジスタ(`$A000` を読む)のビット 7 は、割り込みを要求している印です。`$A001` を読むと、要求が下がります。NMI は、パネルの NMI ボタンで起こします。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

`tutorial/irq_echo.bin` を `$0300` に読み込んで実行し、`A`、`B`、Enter を送ります。

```
*G 0300
AB
*D 03F0 03F2
03F0: CA E9 0D 00 00 00 00 00 24 F4 3A 03 00 00 00 00
```

文字は、割り込みで呼ばれるハンドラがエコーします。`$03F0` と `$03F1` は、その間に主プログラムが数えた値で、実行ごとに変わります。`$03F2` の `0D` は、Enter を受け取った印です。

(c) 2026 masanorisakai / MIT License

6.3.7 パネルで割り込みを見る

Glow65 で確かめられること

割り込みの受け付けと、ハンドラからの復帰を、パネルで 1 つずつ追えます。NMI は、チュートリアル第6章の `nmi_count.bin` で見ます。

1. `tutorial/nmi_count.bin` を読み込んで、モニタで `G 0300` を実行する。プログラムは、`$030F` の `JMP $030F` を繰り返す。
2. **RUN/STOP** を STOP にして、CPU を止める。**STEP MODE** を CYCLE にする。
3. **NMI** ボタンを押す。

押した直後の状態は、次のとおりです(図 6-6)。NMI の LED が点灯し、CPU は止まったままで、PC は `$030F`、SP は `$FD` です。NMI の LED は、受け付ける前の要求を示します。

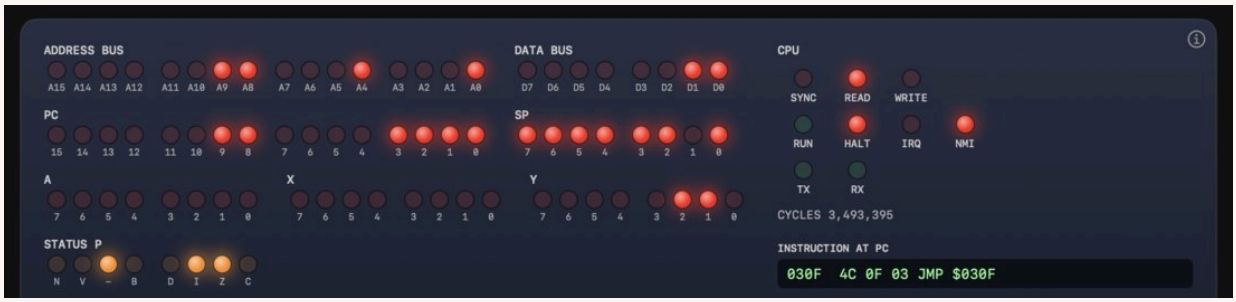


図 6-6 NMI ボタンを押した直後

STEP を押すたびに、割り込みの受け付けが 1 つずつ進みます。

STEP	ADDRESS BUS	DATA BUS	起きたこと
1 回目	`$01FD`	`$03`	WRITE が点灯。PC の上位バイトを積む。NMI の LED は消える
2 回目	`$01FC`	`$0F`	PC の下位バイトを積む
3 回目	`$01FB`	`$26`	P を積む
4 回目	`$FFFA`	`$1E`	READ が点灯。NMI ベクタの下位バイトを読む
5 回目	`$FFFB`	`$F4`	NMI ベクタの上位バイトを読む
6 回目	変わらない	変わらない	PC が `$F41E`、SP が `$FA` になり、受け付けが完了する

1 回目から 5 回目の間、PC は `$030F`、SP は `$FD` のままです。命令の途中では、PC とレジスタが実行前の値を示すためです(第3章 3.4)。1 回目の STEP で、CYCLES が 7 増えます。

(c) 2026 masanorisakai / MIT License



図 6-7 STEP 1 回目。PC の上位バイトを `$01FD` に積んだところ

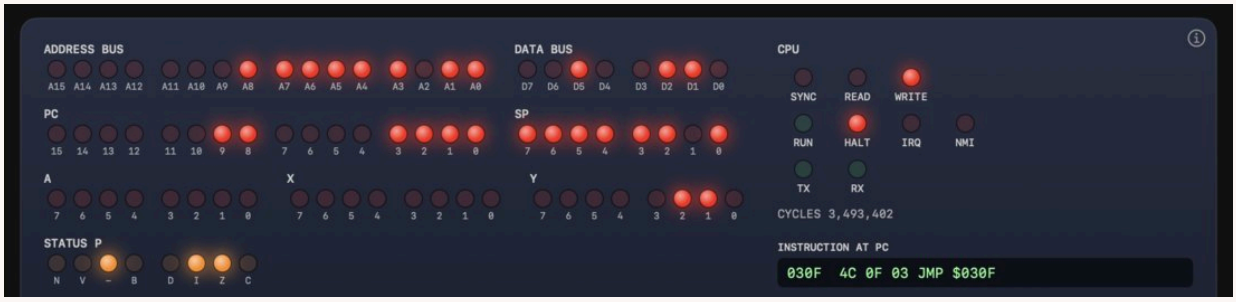


図 6-8 STEP 3 回目。P(`$26`)を `$01FB` に積んだところ



図 6-9 受け付けの完了。PC は `$F41E`(モニタの `JMP ($03F8)`)、SP は `$FA`(3 バイト積んだ)

STEP MODE を INSTR にして STEP を進めると、ハンドラの実行と復帰を見られます。受け付けは 1 回の STEP で終わります。

STEP	実行したもの	PC	SP
1 回目	割り込みの受け付け	`$F41E`	`$FA`
2 回目	`JMP ($03F8)`	`$0312`(ハンドラの `PHA`)	`$FA`
3 回目	`PHA`	`$0313`	`$F9`
11 回目	`RTI`	`$030F`	`$FD`

`PHA` で SP が `$F9` に下がり(図 6-10)、`RTI` で PC が `$030F`、SP が `$FD` に戻ります(図 6-11)。4 回目から 10 回目は、ハンドラの `INC $03F0`、`LDA $A000`、`AND #$02`、`BEQ`、`LDA #$21`、`STA $A001`、`PLA` です。

(c) 2026 masanorisakai / MIT License



図 6-10 `PHA` のあと。SP が `$F9` になる



図 6-11 `RTI` のあと。PC と SP が元に戻る

IRQ は、`tutorial/irq_echo.bin` で見ます。`G 0300` で実行し、RUN/STOP を STOP、STEP MODE を INSTR にして、端末に `A` を入力します。UART に受信文字があり、I フラグが 0 なので、次の STEP で割り込みを受け付けます。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

STEP	実行したもの	PC	IRQ の LED
1 回目	割り込みの受け付け	`$F421`	点灯
2 回目	`JMP ($03FA)`	`$033A`	点灯
3 回目	`PHA`	`$033B`	点灯
4 回目	`LDA $A001`	`$033E`	点灯。A は `$41`(`A`)
5 回目	`CMP #$0D`	`$0340`	消灯



図 6-12 IRQ の受け付け直後。PC は `$F421`(モニタの `JMP ($03FA)`)

`LDA $A001` で受信データを読むと、IRQ の要求は下がります。IRQ の LED は、命令を実行する前の線の状態を示すため、`LDA $A001` を実行した STEP ではまだ点灯していて(図 6-13)、次の STEP で消えます(図 6-14)。



図 6-13 `LDA $A001` のあと。A に `$41` が入り、IRQ の LED はまだ点灯している

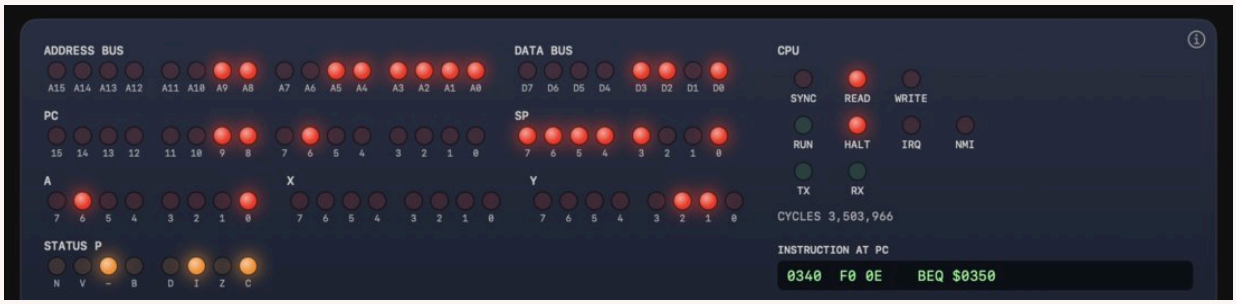


図 6-14 次の STEP のあと。IRQ の LED が消える

手順の詳細と、プログラムの書き方は、チュートリアル第6章にあります。

(c) 2026 masanorisakai / MIT License

6.4 リセット

リセットは、実行中のプログラムを捨てて、決まった状態から始め直します。Glow65 の CPU は、リセットで次の状態になります。

項目	リセット後の値
A、X、Y	0
SP	`$FD`
P	`$24` (I が 1、D が 0)
PC	`$FFFC` と `$FFFD` の 2 バイト(下位が先)

実機の 6502 は、リセット後のレジスタの内部状態が決まっていません[1]。プログラムが、最初に初期化します。Glow65 のモニタも、先頭で `SEI`、`CLD`、`LDX #$FF`、`TXS` を実行して、I を 1、D を 0、SP を `$FF` にします。続けて、UART をマスタリセットし、NMI と IRQ のベクタを RTI だけのルーチンに向けます。

RESET スイッチは、RAM の内容を保ちます。POWER を入れ直すと、RAM が消えます。

6.5 練習問題

答えと解き方は、次の節にあります。

問1 P が `$B5` のとき、1 になっているフラグ(N、V、D、I、Z、C)を答えてください。

問2 `CLC` のあと、次の足し算をします。足したあとの N、V、C を答えてください。(1) `$20 + $30` (2) `$60 + $60` (3) `$C0 + $C0`

問3 A が `$40` のとき、(1) `CMP #$41`、(2) `CMP #$40`、(3) `CMP #$3F` のあとの C、Z、N を答えてください。

問4 SP が `$FD` のとき、`PHA` を 2 回実行して、`JSR` を実行しました。サブルーチンに入った直後の SP はいくつですか。

問5 `$0300` の `JSR $0340` (`20 40 03`)を、SP が `$FD` のときに実行します。スタックに積まれる値と、置かれる番地を答えてください。

問6 SP が `$00` のとき、`PHA` を実行します。A の値は、どの番地に書かれますか。実行後の SP はいくつですか。

問7 IRQ を受け付ける直前の状態が、PC `$0312`、SP `$FD`、P `$20` でした。受け付けたあとの SP、スタックの内容、P(I の値)を答えてください。

問8 UART の受信割り込みを許可しました。ハンドラが `$A001` を読まずに RTI で戻ると、何が起きますか。

(c) 2026 masanorisakai / MIT License

問9 `$0302` にある BRK のハンドラで、スタックに積まれた PC を調べたところ `$0304` でした。署名バイトの番地は、どこですか。

問10 ハンドラに入ったとき、BRK から来たのか、IRQ から来たのかを区別する方法を答えてください。

6.6 練習問題の答えと解き方

問1 N、I、C(このほか、ビット 5 と B)。

- `$B5` は `%10110101`。ビット 7(N)、5、4(B)、2(I)、0(C)が 1。

問2 (1) N = 0、V = 0、C = 0 (2) N = 1、V = 1、C = 0 (3) N = 1、V = 0、C = 1

- (1) 結果 `$50`。 (2) 結果 `$C0`。96 + 96 = 192 は符号付きの範囲を超えるので V が 1。 (3) 結果 `$80`(-64 + -64 = -128 は範囲内なので V は 0)。`$FF` を超えるので C が 1。

問3 (1) C = 0、Z = 0、N = 1 (2) C = 1、Z = 1、N = 0 (3) C = 1、Z = 0、N = 0

- (1) `$40 - $41` は負。A が小さい。 (2) 等しい。 (3) `$40 - $3F` は 1。A が大きい。

問4 `$F9`

- `PHA` 2 回で `$FD` から `$FB`。JSR は 2 バイトを積むので `$F9`。

問5 `$01FD` に `$03`(上位)、`$01FC` に `$02`(下位)。

- 積む値は、JSR の最後のバイトの番地 `$0302`。上位を先に、`$0100 + $FD = $01FD` に書き、次に下位を `$01FC` に書く。

問6 `$0100` に書かれる。SP は `$FF` になる。

- `$0100 + $00 = $0100` に書いて、SP から 1 を引くと、`$00` の次の `$FF` に戻る。

問7 SP は `$FA`。`$01FD` に `$03`、`$01FC` に `$12`、`$01FB` に `$20`。I は 1(P は `$24`)。

- PC の上位 `$03`、下位 `$12`、P の順に積む。積む P は、B が 0、ビット 5 が 1 なので `$20` のまま。受け付けたあと、I が 1 になるので、P は `$24`。

問8 ハンドラが戻っても、要求が続いている。RTI が P を戻して I が 0 になると、CPU は、すぐに、また IRQ を受け付ける。この繰り返しになる。

- IRQ は、要因が消えるまで、レベルで働く。要因を消すには、`$A001` を読む。

問9 `$0303`

- BRK は `$0302`、署名バイトは次の `$0303`。積まれた PC は、署名バイトの次の `$0304` なので、署名バイトの番地は、PC の 1 つ前。

問10 スタックに積まれた P の、B(ビット 4)を調べる。1 なら BRK、0 なら IRQ。

(c) 2026 masanorisakai / MIT License

- BRK が積む P は B = 1、IRQ が積む P は B = 0。ハンドラの先頭で、`TSX` と `LDA $0101,X` で積まれた P を読み、`AND #$10` で調べる。

出典

1. MOS Technology, "MCS6500 Microcomputer Family Hardware Manual", 1976年1月版: https://www.commodore.ca/wp-content/uploads/2018/11/kim1_hardware.txt

(c) 2026 masanorisakai / MIT License

第7章 メモリマップと I/O

CPU が読み書きできるのは、アドレスで指定したバイトだけです。RAM も ROM も、キーボードや画面につながる装置も、アドレスの先にあります。この章では、アドレスを部品に割り振るメモリマップと、装置を操作するメモリマップド I/O を、Glow65 の UART とモニタのルーチンを例に説明します。

7.1 メモリマップ

7.1.1 アドレスの割り振り

CPU は、命令のたびに、16 ビットのアドレスをアドレスバスに出します(第3章)。そのアドレスの先には、RAM、ROM、I/O の部品がつながっています。64KB のアドレス空間を、どの部品にどのアドレスで割り当てるかを決めたものが、**メモリマップ**です。アドレスから部品を選ぶ回路を、**アドレスデコード**と呼びます。

部品に割り当てるアドレスの数が、その部品の大きさです。何も割り当てないアドレスもあります。RAM、ROM、I/O の違いは、第2章 2.8 で説明しています。

7.1.2 Glow65 のメモリマップ

アドレス	部品	内容
`$0000–$7FFF`	RAM	32KB
`$8000–$9FFF`	ROM	GAME65G(約 2.7KB を使用)
`$A000–$A0FF`	UART	偶数のアドレスが状態と制御、奇数のアドレスがデータ
`$A100–$BFFF`	なし	何もつながっていない
`$C000–$EFFF`	ROM	Microsoft BASIC(約 8.6KB を使用)
`$F000–$FFFF`	ROM	モニタ、ジャンプテーブル、ベクタ

(c) 2026 masanorisakai / MIT License

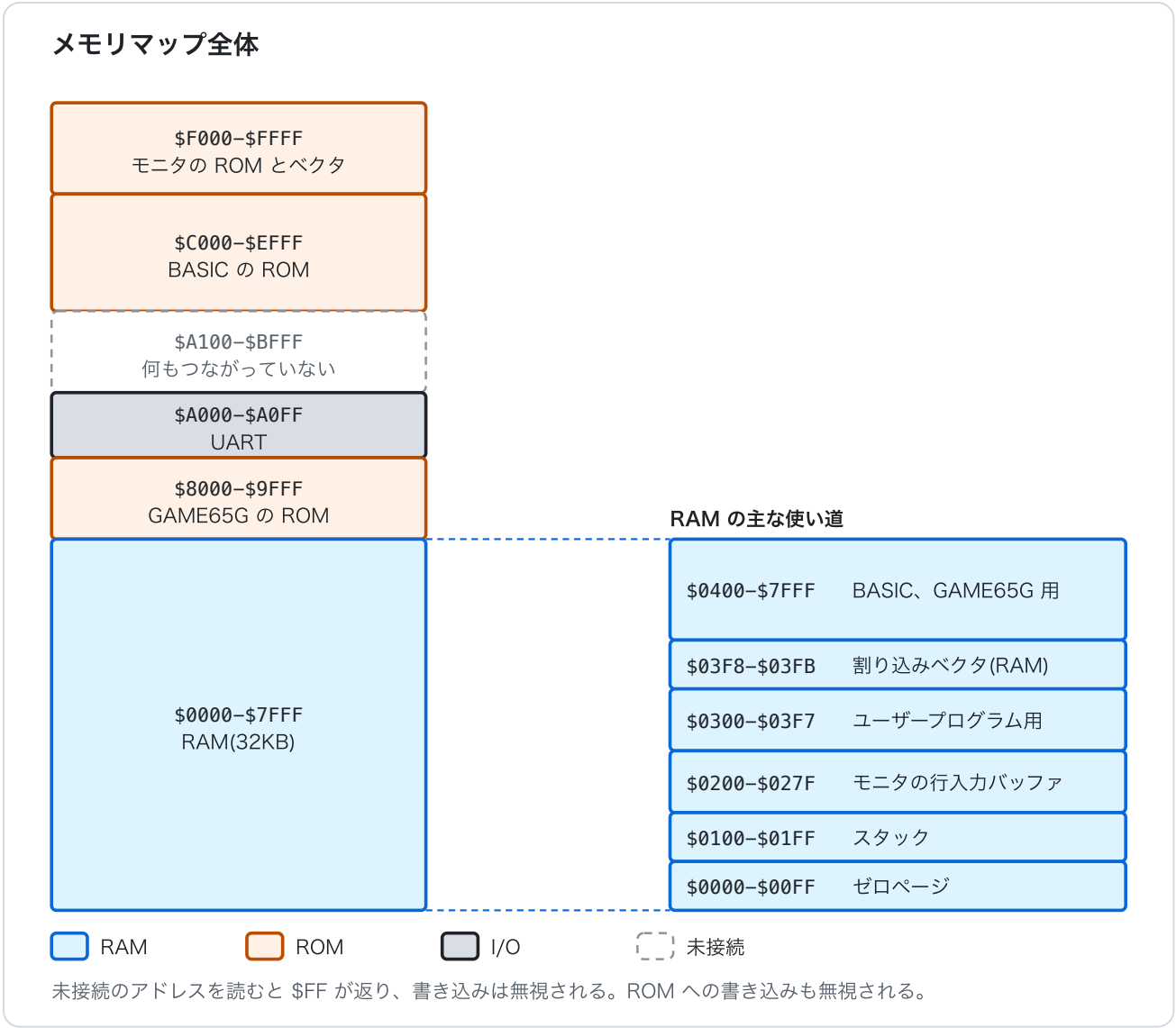


図 7-1 Glow65 のメモリマップ。

何もつながっていないアドレスを読むと、`$FF` が返ります。書き込みは無視されます。ROM への書き込みも、無視されます。

電源を入れると、RAM 全体が 0 になります。RESET スイッチは、RAM の内容を保ちます。

(c) 2026 masanorisakai / MIT License

> **Glow65 で確かめられること**
>
> 電源を入れた直後の RAM の末尾、何もつながっていない `$A100` と `$B000`、ROM の先頭の `$F000` を読みます。ROM には `E` で書き込みます。
>
> ```
> *D 7FF0 7FFF
> 7FF0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
> *D A100 A10F
> A100: FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
> *D B000 B00F
> B000: FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
> *E F000 00
> *D F000 F00F
> F000: 4C 0F F0 4C 28 F0 4C 37 F0 4C 4B F1 4C 1A F0 AD
> ```
>
> `$7FF0` は 0 で埋まっています。`$A100` と `$B000` は、すべて `FF` です。`E F000 00` のあとも、`$F000` は `4C`(`JMP` の命令)のままです。

UART は、`$A000` から `$A0FF` の 256 アドレスを占めます。アドレスの下位 1 ビットだけで、状態と制御(偶数)、データ(奇数)を選びます。そのため、2 つのレジスタが、128 回ずつ繰り返し見えます(**鏡像**)。アドレスを粗く見るだけで部品を選ぶ、単純なデコードです。

> **Glow65 で確かめられること**
>
> `$A000` と、`$A0FE`(鏡像)の状態を読みます。奇数の `$A003` に `M` を書くと、`$A001` に書いたときと同じように送信されます。
>
> ```
> *E 0300 AD 00 A0 8D F0 03 AD FE A0 8D F1 03 A9 4D 8D 03
> *E 0310 A0 A9 0D 8D 03 A0 A9 0A 8D 03 A0 60
> *G 0300
> M
> *D 03F0 03F1
> 03F0: 02 02 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
> ```
>
> 2 つの状態が、どちらも `02` です。端末には、`M` が出ます。

7.1.3 RAM の使い分け

RAM の `$0000` から `$7FFF` は、用途ごとに使い分けます。

(c) 2026 masanorisakai / MIT License

アドレス	用途
`$0000–$00FF`	ゼロページ。1 バイトのアドレスで指定できる。モニタが `$F0–$FF` を使う。BASIC は、USR のベクタ(`$0A–$0C`)や、整数と浮動小数点の変換ルーチンのベクタ(`$06–$09`)などを置く。GAME65G は `$00–$2C` と、変数 A-Z に `$80–$B3` を使う
`$0100–$01FF`	スタック(第6章)
`$0200–$027F`	モニタの行入力バッファ
`$0300–$03F7`	ユーザープログラム用の 248 バイト
`$03F8–$03F9`	NMI ハンドラの番地(下位が先)
`$03FA–$03FB`	IRQ と BRK のハンドラの番地(下位が先)
`$0400–$7FFF`	BASIC のプログラムと変数。GAME65G は `$0400–$05FF` を入力バッファ、第2スタック、先打ちのキーのバッファに、`$0600` からをプログラムに使う

ゼロページの BASIC の使い方は、チュートリアル第5章にあります。

7.2 メモリマップド I/O

I/O は、メモリと同じように、アドレスを指定した読み書きで操作します。この方式を、**メモリマップド I/O** と呼びます。専用の命令は要りません。`LDA` や `STA` が、そのまま使えます。

ただし、I/O のアドレスは、ふつうのメモリと違う動きをします。

違い	UART の例
読むと状態が変わる	`$A001` を読むと、受信した 1 バイトがキューから消える
読みと書きで、別のレジスタになる	`$A000` は、読むと状態、書くと制御。`$A001` は、読むと受信データ、書くと送信データ
書いた値が、そのまま読めるとは限らない	`$A001` に書いた値は、読んでも出てこない
値が、プログラムと関係なく変わる	文字が届くと、状態レジスタが変わる

> **Glow65 で確かめられること**
>
> `D` で `$A000` から 16 バイトを読みます。
>
> ```
> *D A000 A00F
> A000: 02 00 02 00 02 00 02 00 02 00 02 00 02 00 02 00
> ```
>
> 偶数のアドレスが状態で `02`(送信可のビットだけが 1)、奇数のアドレスがデータで `00`(受信データがない)です。

(c) 2026 masanorisakai / MIT License

`D` は、範囲のすべてのアドレスを読みます。奇数のアドレスの読み出しは、受信データがあれば、それを取り出して消します。I/O を観察するときは、プログラムで状態とデータを読み、結果をメモリに書いてから、`D` で見ます。

7.3 UART

UART は、端末とのシリアル通信を受け持つ装置です。1 バイトずつ、送信と受信を行います。Glow65 の UART は、MC6850 に似た最小の構成で、`$A000` と `$A001` の 2 つのレジスタを持ちます。

アドレス	読む	書く
`$A000`	状態レジスタ	制御レジスタ
`$A001`	受信データ	送信データ



図 7-2 UART のレジスタ。

レジスタ	ビット	名前	意味
状態	0	RDRF	1 のとき、受信データがある
状態	1	TDRE	1 のとき、送信できる
状態	7	IRQ	1 のとき、割り込みを要求している
制御	7	RIE	1 で、受信データがある間、IRQ を出す

(c) 2026 masanorisakai / MIT License

レジスタ	ビット	名前	意味
制御	0 と 1	-	両方 1(`%11`)でマスタリセット。受信データを捨て、割り込みの許可も切る

Glow65 の UART は、送信を待たせません。TDRE は、いつも 1 です。書く前に TDRE を調べる手順にしておくと、送信が遅い UART でも同じ手順で動きます。

7.3.1 ポーリング

状態レジスタを、プログラムが繰り返し読んで、ビットが 1 になるのを待つ方法を、**ポーリング**と呼びます。

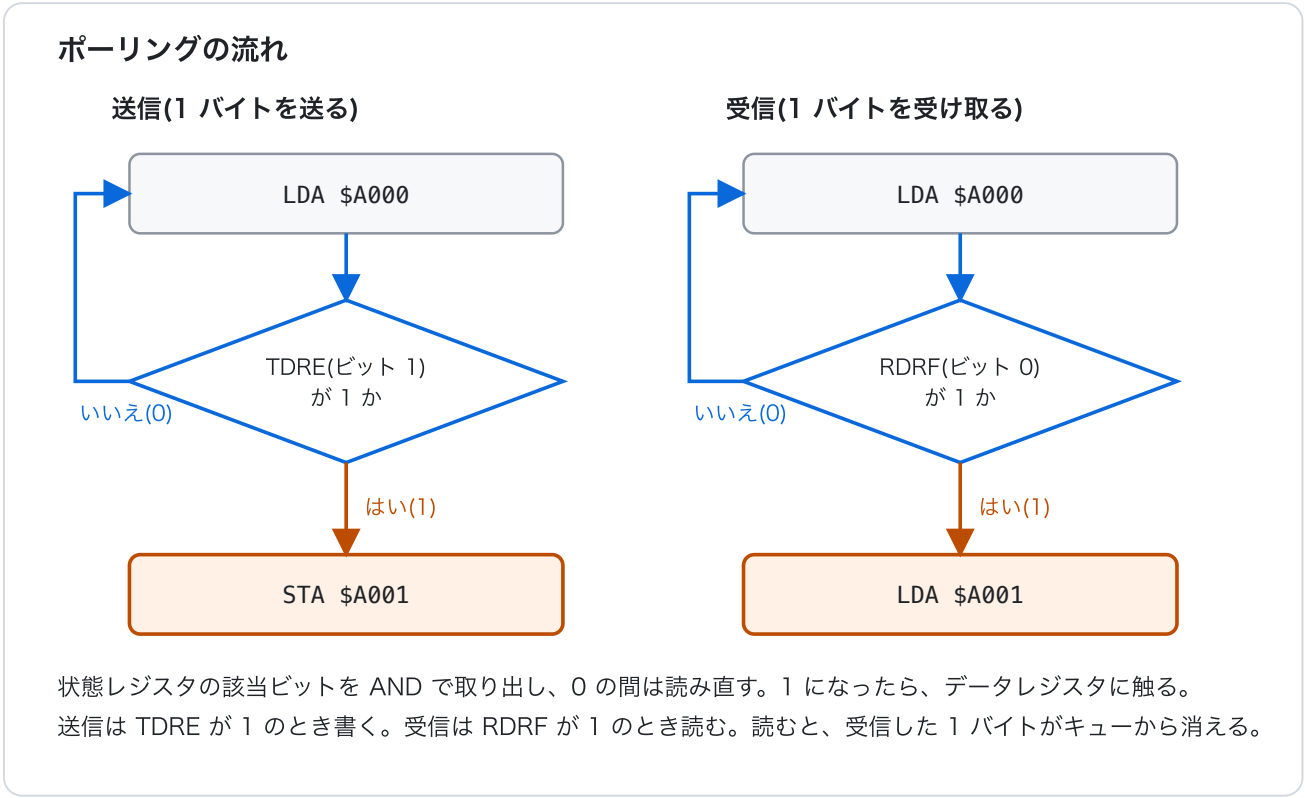


図 7-3 ポーリングの流れ。

送信と受信は、次のように書きます。モニタの `CHROUT` と `CHRIN` も、同じ手順です。

(c) 2026 masanorisakai / MIT License

```
putc:   pha
wait:   lda $A000
        and #$02        ; TDRE
        beq wait
        pla
        sta $A001       ; 送信
        rts

getc:   lda $A000
        and #$01        ; RDRF
        beq getc
        lda $A001       ; 受信
        rts
```

Glow65 で確かめられること

プログラムが、受信を待ちます。実行中に `Q` を送ります。`$A000`、`$A001`、`$A000` の順に読み、値を `$03F0` から書きます。

```
*E 0300 AD 00 A0 29 01 F0 F9 AD 00 A0 8D F0 03 AD 01 A0
*E 0310 8D F1 03 AD 00 A0 8D F2 03 60
*G 0300
*D 03F0 03F2
03F0: 03 51 02 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

1 回目の状態は `03`(RDRF と TDRE)、データは `51`(`Q`)、データを読んだあとの状態は `02` です。読むと、受信データが消えて、RDRF が 0 に戻ります。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

制御レジスタの RIE(ビット 7)を 1 にしてから、同じことをします。CPU の I は 1 のままなので、割り込みは起きません。

```
*E 0300 A9 80 8D 00 A0 AD 00 A0 29 01 F0 F9 AD 00 A0 8D
*E 0310 F0 03 AD 01 A0 8D F1 03 AD 00 A0 8D F2 03 A9 03
*E 0320 8D 00 A0 60
*G 0300
*D 03F0 03F2
03F0: 83 51 02 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

状態が `83` です。IRQ(ビット 7)が 1 で、割り込みを要求している印です。データを読むと、`02` に戻ります。

Glow65 で確かめられること

2 文字(`X` と `Y`)を送り、受信できる状態になったら、制御レジスタに `$03` を書いて、状態を読みます。

```
*E 0300 AD 00 A0 29 01 F0 F9 A9 03 8D 00 A0 AD 00 A0 8D
*E 0310 F0 03 60
*G 0300
*D 03F0 03F0
03F0: 02 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

状態は `02` です。マスタリセットで、受信データが捨てられました。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

ポーリングで状態が変わる様子を、パネルで追えます。チュートリアル第4章の `echo.bin` を使います。

1. `tutorial/echo.bin` を読み込んで、モニタで `G 0300` を実行する。プログラムは、`LDA $A000`、`AND #$01`、`BEQ` を繰り返して、受信を待つ。
2. **RUN/STOP** を STOP にして、CPU を止める。**STEP MODE** を INSTR にして、**STEP** を押す。
3. `LDA $A000` を実行した STEP では、ADDRESS BUS が `$A000`、DATA BUS が `$02` (D1 だけ点灯)で、A も `$02` になる。送信可(TDRE)だけが 1 で、受信データはありません(図 7-4)。
4. 停止したまま、端末に `h` を入力する。次に `LDA $A000` を実行した STEP では、DATA BUS が `$03` (D1 と D0)になり、A も `$03` になる。受信あり(RDRF)が 1 になりました(図 7-5)。
5. `AND #$01` と `BEQ` を進めると、分岐しないで、`LDA $A001` に進む。`LDA $A001` を実行した STEP では、ADDRESS BUS が `$A001`、DATA BUS が `$68` (`h`)で、A も `$68` になる(図 7-6)。
6. そのあとの送信待ちの `LDA $A000` では、DATA BUS が `$02` に戻る。データを読むと、RDRF が 0 になりました(図 7-7)。



図 7-4 受信データがないとき。`$A000` を読むと `$02`



図 7-5 `h` を入力したあと。`$A000` を読むと `$03`

(c) 2026 masanorisakai / MIT License

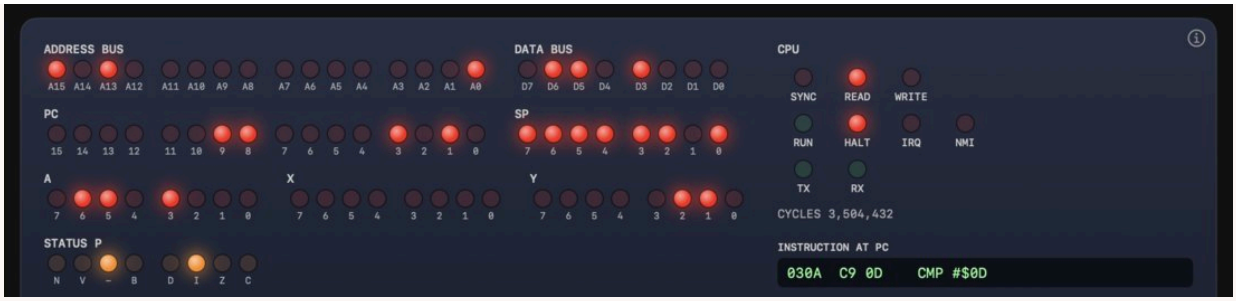


図 7-6 `$A001` を読むと、受信した `h`(`$68`)が A に入る



図 7-7 データを読んだあと。`$A000` は `$02` に戻る

割り込みを使った受信は、6.3.6 とチュートリアル第6章にあります。ポーリングによる送受信は、チュートリアル第3章と第4章にあります。

7.4 モニタのルーチン

モニタは、ROM の `$F000` から、5 つのルーチンへ飛ぶ `JMP` を並べています(ジャンプテーブル)。ルーチンの本体の場所が変わっても、このアドレスは変わりません。`JSR` で呼びます。

アドレス	名前	入力	出力	動作
`$F000`	CHRIN	なし	A = 文字	文字が届くまで待って、1 文字読む
`$F003`	CHROUT	A = 文字	なし(A は変わる)	最上位ビットを捨てて、1 文字送る。送信できるまで待つ
`$F006`	ISCNTC	なし	C = 1 なら Ctrl-C、C = 0 ならそれ以外(A は変わる)	待たない。文字があれば 1 文字読み、Ctrl-C(`$03`)かを調べる
`$F009`	ウォームスタート	なし	なし	モニタに戻る。`JSR` ではなく `JMP` で飛ぶ
`$F00C`	GETKEY	なし	A = 文字、届いていなければ A = 0	待たずに、1 文字読む

X と Y は、どのルーチンでも変わりません。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

`CHROUT` で `$48(H)`と `$C9` を送り、続けて CR(`$0D`)と LF(`$0A`)を送ります。

```
*E 0300 A9 48 20 03 F0 A9 C9 20 03 F0 A9 0D 20 03 F0 A9
*E 0310 0A 20 03 F0 60
*G 0300
HI
```

`$C9` は、最上位ビットを捨てた `$49(I)`として送られます。

Glow65 で確かめられること

`CHRIN` で 1 文字を待ち、`$03F0` に書きます。実行中に `z` を送ります。

```
*E 0300 20 00 F0 8D F0 03 60
*G 0300
*D 03F0 03F0
03F0: 7A 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`z` の ASCII コード `$7A` が読めます。

Glow65 で確かめられること

`GETKEY` を、文字が届いていないときに 1 回呼び、`$03F0` に書きます。そのあと、文字が届くまで繰り返し呼びます。実行中に `k` を送ります。

```
*E 0300 20 0C F0 8D F0 03 20 0C F0 F0 FB 8D F1 03 60
*G 0300
*D 03F0 03F1
03F0: 00 6B 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`$03F0` は `00` です。待たずに戻ったので、A が 0 になりました。`$03F1` の `6B` は、`k` です。

(c) 2026 masanorisakai / MIT License

Glow65 で確かめられること

`ISCNTC` を、Ctrl-C が届くまで繰り返し呼び、届いたら `!` を送ります。実行中に、`a` と Ctrl-C(`$03`)を送ります。

```
*E 0300 20 06 F0 90 FB A9 21 20 03 F0 A9 0D 20 03 F0 A9
*E 0310 0A 20 03 F0 60
*G 0300
!
```

`a` は読み捨てられて C = 0 で戻り、ループは続きます。Ctrl-C で C = 1 になって、`!` が出ます。

Glow65 で確かめられること

`$03F0` に `$77` を書き、`JMP $F009` でモニタに戻ります。

```
*E 0300 A9 77 8D F0 03 4C 09 F0
*G 0300
*D 03F0 03F0
03F0: 77 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`RTS` を使わずに、モニタのプロンプトに戻ります。

モニタのコマンドは、次のとおりです。

コマンド	動作
`D aaaa [bbbb]`	メモリダンプ(終了アドレスを省くと 128 バイト)
`E aaaa bb bb ...`	バイトを書き込む
`L aaaa [bbbb]`	逆アセンブル(終了アドレスを省くと 16 命令)
`G aaaa`	`aaaa` のルーチンを呼ぶ(RTS でモニタに戻る)
`B`	BASIC を起動
`T`	GAME65G を起動
`?`	ヘルプ

7.5 ベクタ領域

ROM の最後の 6 バイト(`$FFFA` から `$FFFF`)は、NMI、リセット、IRQ と BRK のベクタです。仕組みは、第3章 3.7 と、第6章 6.3 で説明しています。

(c) 2026 masanorisakai / MIT License

7.6 練習問題

答えと解き方は、次の節にあります。

問1 次のアドレスを読むと、どの部品の、何が読めますか。(1) `$0300` (2) `$A0F5` (3) `$B000` (4) `$F000`

問2 `$A0A0` と `$A0A1` は、それぞれ UART のどのレジスタですか。

問3 `STA $F000` を実行すると、どうなりますか。

問4 状態レジスタを読んだら、`$03` でした。何がわかりますか。

問5 状態レジスタを読んだら、`$83` でした。何がわかりますか。

問6 受信ポーリングで、1 バイトを読んで A に入れるルーチンを、状態レジスタの RDRF を使って書いてください。

問7 `LDA #$C1`、`JSR $F003` を実行すると、端末に何が出ますか。

問8 受信データ `Q` が 1 バイト届いています。プログラムが `$A000`、`$A001`、`$A000` の順に読むと、読んだ 3 つの値は何ですか。

問9 `a` が届いている状態で `JSR $F006` を呼ぶと、戻ったときの C と、`a` はどうなりますか。

7.7 練習問題の答えと解き方

問1 (1) RAM。書いた値が読める(電源投入直後は `$00`) (2) UART のデータレジスタ。受信データがなければ `$00` (3) 何もつながっていない。`$FF` (4) モニタの ROM。ジャンプテーブルの先頭で、`$4C`(`JMP`)

- (1) `$0300` は `$0000–$7FFF` の中。 (2) `$A0F5` は `$A000–$A0FF` の中で、奇数なので、データ。 (3) `$B000` は `$A100–$BFFF` の中で、未接続。 (4) `$F000` は ROM で、`D` で `4C` と読める。

問2 `$A0A0` は、読むと状態、書くと制御。`$A0A1` は、読むと受信データ、書くと送信データ。

- UART は `$A000–$A0FF` に鏡像で見える。偶数が状態と制御、奇数がデータ。

問3 何も起きない。ROM は変わらず、プログラムは次の命令に進む。

- ROM への書き込みは無視される。

問4 受信データがあり(RDRF = 1)、送信もできる(TDRE = 1)。割り込みの要求はない。

- `$03` は `%00000011`。ビット 0 とビット 1 が 1。ビット 7(IRQ)は 0。

問5 受信データがあり、送信もできて、割り込みを要求している。

- `$83` は `%10000011`。ビット 7(IRQ)、ビット 1(TDRE)、ビット 0(RDRF)が 1。IRQ は、制御の RIE が 1 で、未読の受信データがあるときに 1 になる。

(c) 2026 masanorisakai / MIT License

問6

```
getc:   lda $A000
        and #$01        ; RDRF
        beq getc        ; 0 なら繰り返す
        lda $A001       ; 1 なら、データを読む
        rts
```

- 状態を読み、ビット 0 を AND で取り出す。0 の間は繰り返し、1 になったら `$A001` を読む。

問7 `A`

- `CHROUT` は、最上位ビットを捨てる。`$C1` は `$41` になり、`A` の ASCII コード。

問8 `$03`、`$51`、`$02`

- 1 回目の状態は、RDRF と TDRE で `$03`。`$A001` を読むと、`Q(`$51`)`が取り出されて、受信データが消える。2 回目の状態は、RDRF が 0 に戻って `$02`。

問9 C は 0 で、`a` は読み捨てられる。

- `ISCNTC` は、文字があれば 1 文字読む。`$03` でなければ C を 0 にして戻る。読んだ文字は、返らない。

(c) 2026 masanorisakai / MIT License

第8章 用語集と索引

第1章から第7章に出てくる用語の定義と、命令、アドレッシングモード、フラグ、アドレス、図を引く索引です。

節	内容
8.1 用語集	用語の定義を、五十音順に並べたもの。72 語
8.2 索引	命令、アドレッシングモード、フラグとレジスタ、モニタのコマンドとルーチン、主なアドレス、図
8.3 チュートリアルとの対応	マニュアルの章と、チュートリアルの章の対応表

8.3 チュートリアルとの対応

マニュアルは読んで理解する資料、チュートリアルは手を動かす資料です。マニュアルの章を読んだら、対応するチュートリアルの章を試すと、内容が定着します。

チュートリアル	内容	対応するマニュアルの章
1. LED カウンタ	5 行のプログラムを読み込み、バスアクセスを追う	第2章 メモリとアドレス、第3章 フェッチと実行、第3章 命令の構成
2. 足し算とキャリーフラグ	16 ビットの足し算と C	第2章 数の範囲、第5章 算術演算と比較、第6章 C(キャリー)
3. UART と話す	`CHROUT` で文字を送る	第7章 UART、第7章 モニタのルーチン
4. キーボードを読む	ポーリングで受信するエコー	第7章 ポーリング、第7章 モニタのルーチン
5. USR	BASIC から機械語を呼ぶ	第7章 RAM の使い分け、第5章 JSR と RTS
6. 割り込み	NMI と IRQ のハンドラ	第6章 割り込み、第6章 スタック
7. 65C02 に切り替える	同じプログラムを 6502 と 65C02 で書く、未定義オペコード	第4章 65C02 で加わるモード、第5章 65C02

アドレッシングモード(第4章)と命令リファレンス(第5章)は、チュートリアルのプログラムを読み解くときの辞書として使えます。

(c) 2026 masanorisakai / MIT License

8.1 用語集

第1章から第7章に出てくる用語を、五十音順に並べています。英語の用語は、読みの位置に置いています。定義のあとのリンクは、その用語を最初に説明する節です。

あ行

- **IRQ**(Interrupt Request): UART などの装置が出す割り込みです。I フラグでマスクされます。要求が続く間、何度でも受け付けられます(レベルで働く)。(6.3.1 4 つの割り込み)
- **I/O**(Input/Output): 外の装置との窓口です。メモリの特定のアドレスに装置が割り当てられ、そのアドレスを読み書きして、装置とやり取りします。メモリマップド I/O とも呼びます。(2.8 ROM、RAM、I/O)
- **アキュムレータ**(Accumulator): A レジスタです。計算の中心で、足し算や比較の対象になります。(3.2 レジスタ)
- **ASCII**(American Standard Code for Information Interchange): 英数字と記号に、数を割り当てた文字コードです。`A` が `$41`、`0` が `$30`、CR が `$0D` です。(2.6 文字コード(ASCII))
- **アセンブラ**(Assembler): アセンブリで書いたプログラムを、機械語のバイトに変えるプログラムです。分岐先のラベルを、距離に直すのもアセンブラです。(命令の書式)
- **アセンブリ**(Assembly): ニーモニックとオペランドで、命令を文字で書き表したものです。`LDA #$10` のように書きます。(命令の書式)
- **アドレス**(Address): メモリの 1 バイトごとに付けた番号(番地)です。6502 のアドレスは 16 ビットで、`$0000` から `$FFFF` までの 65,536 個です。(2.7 メモリとアドレス)
- **アドレス空間**(Address space): CPU が指定できるアドレスの全体です。6502 では 64KB です。(2.7 メモリとアドレス)
- **アドレスデコード**(Address decoding): アドレスから、RAM、ROM、I/O のどの部品を使うかを選ぶ回路(仕組み)です。(7.1.1 アドレスの割り振り)
- **アドレスバス**(Address bus): CPU がアドレスを出す 16 本の線です。A0 から A15 です。(3.3 バス)
- **アドレッシングモード**(Addressing mode): 命令のオペランドが、どのメモリの値を対象にするかを決める方式です。6502 には 13 のモードがあります。(4.1 アドレッシングモードとは)
- **エッジ**(Edge): 信号が「なし」から「あり」に変わる瞬間です。NMI は、エッジごとに 1 回だけ受け付けられます。(6.3.3 NMI と IRQ の違い)
- **NMI**(Non-Maskable Interrupt): I フラグに関係なく受け付ける割り込みです。ベクタは `$FFFA` と `$FFFB` です。Glow65 では、パネルの NMI ボタンで起こします。(6.3.1 4 つの割り込み)
- **オペコード**(Opcode): 命令の最初の 1 バイトです。命令の種類と、アドレッシングモードを決めます。(3.4 フェッチと実行)
- **オペランド**(Operand): 命令が使う値や、番地を表すバイトです。0 から 2 バイトあります。(3.4 フェッチと実行)
- **オーバーフロー**(Overflow): V フラグです。符号付きの数として計算したときの桁あふれを示します。同じ符号の 2 つの数を足して、結果の符号が変わると 1 になります。(6.1.3 V(オーバーフロー))

(c) 2026 masanorisakai / MIT License

か行

- **機械語**(Machine code): オペコードとオペランドのバイトで書いた、CPU が直接実行するプログラムです。(3.5 命令の構成)
- **逆アセンブル**(Disassemble): メモリのバイトを命令として読み、書き方に戻すことです。モニタの `L` コマンドが行います。(3.5 命令の構成)
- **キャリー**(Carry): C フラグです。8 ビットに収まらなかった桁を持ちます。ADC では結果が `$FF` を超えると 1、SBC では借りがないと 1 になります。(6.1.2 C(キャリー))
- **鏡像**(Mirror): 1 つのレジスタが、複数のアドレスに繰り返し見えることです。Glow65 の UART は、`$A000` から `$A0FF` に 2 つのレジスタが繰り返し見えます。(7.1.2 Glow65 のメモリマップ)
- **クロック**(Clock): CPU の動きを進める、一定間隔の信号です。クロックの 1 周期が 1 サイクルです。(3.4 フェッチと実行(サイクルとクロック))

さ行

- **最下位ビット**(Least significant bit): 8 ビットのうち、右端のビット 0 です。(2.1 ビットとバイト)
- **サイクル**(Cycle): クロックの 1 周期です。CPU は、1 サイクルにバスへ 1 回アクセスします。(3.4 フェッチと実行)
- **最上位ビット**(Most significant bit): 8 ビットのうち、左端のビット 7 です。符号付きの数では、符号ビットです。(2.1 ビットとバイト)
- **サブルーチン**(Subroutine): 呼び出して使う、小さなプログラムです。JSR で呼び、RTS で戻ります。(5.5 分岐とジャンプ)
- **実効アドレス**(Effective address): 命令が実際に読み書きするメモリの番地です。アドレッシングモードごとに、求め方が違います。(4.1 アドレッシングモードとは)
- **10進モード**(Decimal mode): D フラグが 1 のとき、ADC と SBC が BCD で計算するモードです。`$09 + $01` が `$10` になります。(1.5 ファミリーコンピュータ、6.1.4 D(10進モード))
- **シフト**(Shift): ビットを横に 1 つずらすことです。ASL は左に、LSR は右にずらします。(5.4 論理演算とシフト)
- **16進数**(Hexadecimal): 4 ビットを 1 桁(0 から F)で書く表し方です。6502 では、先頭に `$` を付けます。(2.3 16進数)
- **条件分岐**(Conditional branch): フラグを見て、条件が成り立つときだけ飛ぶ命令です。BCC、BCS、BEQ、BNE、BMI、BPL、BVC、BVS があります。(6.1.7 条件分岐が見るフラグ)
- **署名バイト**(Signature byte): BRK の次の 1 バイトです。CPU は使わずに飛ばします。ハンドラが読めば、BRK の種類を区別できます。(6.3.4 BRK)
- **SYNC**(Sync): CPU が、命令の最初のバイト(オペコード)を読んでいるときに 1 になる信号です。(3.3 バス)
- **CPU**(Central Processing Unit): 中央処理装置です。メモリにある命令を 1 つずつ取り出して実行する部品です。(3.1 CPU とメモリ)
- **スタック**(Stack): `$0100` から `$01FF` のメモリを、後から入れたものを先に取り出す順で使う領域です。(6.2.1 場所と SP)

(c) 2026 masanorisakai / MIT License

- **スタックポインタ(SP)**(Stack pointer): 8 ビットのレジスタです。`$0100` に SP を足した番地が、次に書く場所です。(6.2.1 場所と SP)
- **ゼロページ**(Zero page): アドレスの上位が `$00` の、先頭の 256 バイト(`$0000` から `$00FF`)です。1 バイトのアドレスで指定でき、命令が短く速くなります。(1.2 6502 の仕様)

た行

- **ダミーリード**(Dummy read): 実機の 6502 が行う、結果に使われない余分な読み出しです。Glow65 は再現しません。(Glow65 のサイクルとバスアクセス)
- **転送**(Transfer): レジスタの値を、別のレジスタにコピーすることです。TAX、TAY、TXA、TYA、TSX、TXS があります。(5.2 レジスタ間の転送)
- **データバス**(Data bus): 読み書きするバイトが流れる 8 本の線です。D0 から D7 です。(3.3 バス)
- **トリガ**(Trigger): 割り込みを起こす条件です。NMI はエッジ、IRQ はレベルです。(6.3.1 4 つの割り込み)

な行

- **2進数**(Binary): 0 と 1 だけで書く数の表し方です。1 桁ごとに 2 倍になります。(2.2 2進数)
- **2 の補数**(Two's complement): 負の数を表す方式です。反転して 1 を足します。最上位ビットが 1 なら負です。8 ビットでは -128 から 127 を表します。(2.5 負の数と2の補数)
- **ニーモニック**(Mnemonic): `LDA` や `STA` のような、命令の名前です。6502 の公式の命令は 56 個あります。(全 56 命令の一覧)

は行

- **バイト**(Byte): 8 ビットをひとまとめにした単位です。6502 は、1 バイトずつデータを扱います。(2.1 ビットとバイト)
- **バス**(Bus): CPU とメモリの間をつなぐ、複数の線の束です。アドレスバスとデータバスがあります。(3.3 バス)
- **ハンドラ**(Handler): 割り込みを処理するプログラムです。RTI で終わります。(6.3.2 受け付けの手順)
- **ビット**(Bit): 0 か 1 のどちらかを持つ、最小の単位です。(2.1 ビットとバイト)
- **BCD**(Binary Coded Decimal): 各 4 ビットを 0 から 9 の数字として扱う形式です。10進モードの ADC と SBC が使います。(5.3 算術演算と比較)
- **P レジスタ(ステータス)**(Processor status): フラグをまとめた 8 ビットのレジスタです。実体のあるフラグは N、V、D、I、Z、C の 6 つです。(6.1 フラグ(P レジスタ))
- **フェッチ**(Fetch): PC の番地から、命令のバイトを取り出すことです。(3.4 フェッチと実行)
- **符号ビット**(Sign bit): 2 の補数で、最上位ビット(ビット 7)のことです。1 なら負、0 なら 0 以上です。(2.5 負の数と2の補数)
- **フラグ**(Flag): P レジスタの 1 ビットです。直前の命令の結果を表します。(3.2 レジスタ)
- **BRK(ブレーク)**(Break): プログラムから割り込みを起こす命令です。IRQ のベクタを使い、I でマスクされません。積まれる P では B が 1 です。(6.3.4 BRK)

(c) 2026 masanorisakai / MIT License

- **プログラムカウンタ(PC)**(Program counter): 次に実行する命令のアドレスを持つ、16 ビットのレジスタです。(3.1 CPU とメモリ)
- **ベクタ**(Vector): 飛び先の番地を 2 バイトで持つ場所です(下位が先)。NMI は `$FFFA`、リセットは `$FFFC`、IRQ と BRK は `$FFFE` です。(3.7 リセットと割り込みのベクタ)
- **ページ**(Page): 上位バイトが同じ、256 バイトのまとまりです。足した結果の上位バイトが変わることを、「ページをまたぐ」と呼びます。(4.5 アブソリュート系のモード)
- **ポーリング**(Polling): 状態レジスタを、プログラムが繰り返し読んで、ビットが 1 になるのを待つ方法です。(7.3.1 ポーリング)

ま行

- **マスク**(Mask): 割り込みを受け付けないようにすることです。I フラグが 1 の間、IRQ はマスクされます。NMI は、マスクできません。(6.3.1 4 つの割り込み)
- **マスタリセット**(Master reset): UART の制御レジスタに `%11` を書いて、UART を初期状態に戻すことです。受信データを捨て、割り込みの許可も切ります。(7.3 UART)
- **メモリ**(Memory): 1 バイトずつ入る箱が一列に並んだ棚です。箱に、先頭から順にアドレスを付けます。(2.7 メモリとアドレス)
- **メモリマップ**(Memory map): 64KB のアドレス空間を、RAM、ROM、I/O の部品に割り当てたものです。(7.1.1 アドレスの割り振り)
- **メモリマップド I/O**(Memory-mapped I/O): I/O を、メモリと同じように、アドレスを指定した読み書きで操作する方式です。(7.2 メモリマップド I/O)
- **モニタ**(Monitor): Glow65 の ROM に入っている小さなプログラムです。メモリの表示や書き込み、実行を、コマンドで行えます。(7.4 モニタのルーチン)

や行

- **UART**(Universal Asynchronous Receiver/Transmitter): 端末とのシリアル通信を受け持つ装置です。Glow65 では、`$A000` と `$A001` の 2 つのレジスタを持ちます。(7.3 UART)

ら行

- **RAM**(Random Access Memory): 読み書きできるメモリです。電源を切ると内容が消えます。(2.8 ROM、RAM、I/O)
- **リセット**(Reset): 実行中のプログラムを捨てて、決まった状態から始め直すことです。PC は、`$FFFC` と `$FFFD` のベクタから決まります。(6.4 リセット)
- **リトルエンディアン**(Little endian): 2 バイト以上の値を、下位のバイトから先に置く並べ方です。6502 のアドレスは、この順で置かれます。(2.7 メモリとアドレス)
- **レジスタ**(Register): CPU の内部にある、数バイト分の記憶領域です。メモリより速く読み書きできます。(3.2 レジスタ)
- **レベル**(Level): 信号が「あり」の間、続く状態です。IRQ は、要求が続く間、何度でも受け付けられます。(6.3.3 NMI と IRQ の違い)

(c) 2026 masanorisakai / MIT License

- **ROM**(Read Only Memory): 読み出し専用のメモリです。電源を切っても内容が残ります。起動時に動くプログラムを置きます。(2.8 ROM、RAM、I/O)
- **ロードとストア**(Load / Store): ロードは、メモリの値をレジスタに入れる命令です。ストアは、レジスタの値をメモリに書く命令です。(5.1 ロードとストア)

わ行

- **割り込み**(Interrupt): 実行中のプログラムを中断して、別の処理に移ることです。要因は、NMI、IRQ、BRK です。(6.3 割り込み)

(c) 2026 masanorisakai / MIT License

8.2 索引

8.2.1 命令の索引

アルファベット順です。第5章の各命令の見出しへ飛びます。

ニーモニック	名前
ADC	Add with Carry(キャリー付き加算)
AND	Logical AND(論理積)
ASL	Arithmetic Shift Left(算術左シフト)
BCC	Branch if Carry Clear(C が 0 なら分岐)
BCS	Branch if Carry Set(C が 1 なら分岐)
BEQ	Branch if Equal(Z が 1 なら分岐)
BIT	Bit Test(ビットテスト)
BMI	Branch if Minus(N が 1 なら分岐)
BNE	Branch if Not Equal(Z が 0 なら分岐)
BPL	Branch if Plus(N が 0 なら分岐)
BRK	Break(ブレーク)
BVC	Branch if Overflow Clear(V が 0 なら分岐)
BVS	Branch if Overflow Set(V が 1 なら分岐)
CLC	Clear Carry Flag(C を 0 にする)
CLD	Clear Decimal Mode(D を 0 にする)
CLI	Clear Interrupt Disable(I を 0 にする)
CLV	Clear Overflow Flag(V を 0 にする)
CMP	Compare Accumulator(A の比較)
CPX	Compare X Register(X の比較)
CPY	Compare Y Register(Y の比較)
DEC	Decrement Memory(メモリを 1 減らす)
DEX	Decrement X Register(X を 1 減らす)
DEY	Decrement Y Register(Y を 1 減らす)
EOR	Exclusive OR(排他的論理和)
INC	Increment Memory(メモリを 1 増やす)
INX	Increment X Register(X を 1 増やす)

(c) 2026 masanorisakai / MIT License

ニーモニック	名前
INY	Increment Y Register(Y を 1 増やす)
JMP	Jump(ジャンプ)
JSR	Jump to Subroutine(サブルーチン呼び出し)
LDA	Load Accumulator(ロード A)
LDX	Load X Register(ロード X)
LDY	Load Y Register(ロード Y)
LSR	Logical Shift Right(論理右シフト)
NOP	No Operation(何もしない)
ORA	Logical OR(論理和)
PHA	Push Accumulator(A を積む)
PHP	Push Processor Status(P を積む)
PLA	Pull Accumulator(A に取り出す)
PLP	Pull Processor Status(P に取り出す)
ROL	Rotate Left(左ローテート)
ROR	Rotate Right(右ローテート)
RTI	Return from Interrupt(割り込みから戻る)
RTS	Return from Subroutine(サブルーチンから戻る)
SBC	Subtract with Carry(キャリー付き減算)
SEC	Set Carry Flag(C を 1 にする)
SED	Set Decimal Mode(D を 1 にする)
SEI	Set Interrupt Disable(I を 1 にする)
STA	Store Accumulator(ストア A)
STX	Store X Register(ストア X)
STY	Store Y Register(ストア Y)
TAX	Transfer A to X(A を X へ転送)
TAY	Transfer A to Y(A を Y へ転送)
TSX	Transfer Stack Pointer to X(SP を X へ転送)
TXA	Transfer X to A(X を A へ転送)
TXS	Transfer X to Stack Pointer(X を SP へ転送)
TYA	Transfer Y to A(Y を A へ転送)

(c) 2026 masanorisakai / MIT License

65C02 で加わる命令

Glow65 の 65C02 モードで使える命令です。第5章の 5.7 の見出しへ飛びます。

ニーモニック	名前
BBR	Branch on Bit Reset(ビットが 0 なら分岐)
BBS	Branch on Bit Set(ビットが 1 なら分岐)
BRA	Branch Always(いつも分岐)
PHX	Push X Register(X を積む)
PHY	Push Y Register(Y を積む)
PLX	Pull X Register(X に取り出す)
PLY	Pull Y Register(Y に取り出す)
RMB	Reset Memory Bit(メモリのビットを 0 にする)
SMB	Set Memory Bit(メモリのビットを 1 にする)
STP	Stop the Clock(停止)
STZ	Store Zero(メモリに 0 を書く)
TRB	Test and Reset Bits(ビットを調べて 0 にする)
TSB	Test and Set Bits(ビットを調べて 1 にする)
WAI	Wait for Interrupt(割り込みを待つ)
INC A	INC と DEC の新しいモード
DEC A	INC と DEC の新しいモード

8.2.2 アドレッシングモードの索引

モード	書き方の例	節
インプライド(Implied)	`INX`	インプライド(Implied)
アキュムレータ(Accumulator)	`ASL A`	アキュムレータ(Accumulator)
イミディエイト(Immediate)	`LDA #$10`	4.3 イミディエイト(Immediate)
ゼロページ(Zero Page)	`LDA $10`	ゼロページ(Zero Page)
ゼロページ X(Zero Page,X)	`LDA $10,X`	ゼロページ X(Zero Page,X)
ゼロページ Y(Zero Page,Y)	`LDX $10,Y`	ゼロページ Y(Zero Page,Y)
アブソリュート(Absolute)	`LDA $0400`	アブソリュート(Absolute)
アブソリュート X(Absolute,X)	`LDA $0410,X`	アブソリュート X(Absolute,X)

(c) 2026 masanorisakai / MIT License

モード	書き方の例	節
アブソリュート Y(Absolute,Y)	`LDA $0420,Y`	アブソリュート Y(Absolute,Y)
インダイレクト(Indirect)	`JMP ($0430)`	4.6 インダイレクト(Indirect)
インデックスト・インダイレクト(Indexed Indirect)	`LDA ($20,X)`	4.7 インデックスト・インダイレクト(Indexed Indirect)
インダイレクト・インデックスト(Indirect Indexed)	`LDA ($30),Y`	4.8 インダイレクト・インデックスト(Indirect Indexed)
リラティブ(Relative)	`BNE label`	4.9 リラティブ(Relative)

8.2.3 フラグとレジスタの索引

項目	節
N(ネガティブ)	6.1.1 N と Z(結果のフラグ)
Z(ゼロ)	6.1.1 N と Z(結果のフラグ)
C(キャリー)	6.1.2 C(キャリー)
V(オーバーフロー)	6.1.3 V(オーバーフロー)
D(10進モード)	6.1.4 D(10進モード)
I(割り込み禁止)	6.1.5 I(割り込み禁止)
B(ブレーク)とビット 5	6.1.6 B とビット 5
フラグの全体(P レジスタ)	3.2 レジスタ、6.1 フラグ(P レジスタ)
A(アキュムレータ)	3.2 レジスタ
X、Y(インデックスレジスタ)	3.2 レジスタ
SP(スタックポインタ)	6.2.1 場所と SP、3.2 レジスタ
PC(プログラムカウンタ)	3.1 CPU とメモリ、3.2 レジスタ
P(ステータス)	6.1 フラグ(P レジスタ)、3.2 レジスタ

8.2.4 モニタのコマンドとルーチンの索引

項目	内容	節
コマンド `D`	メモリダンプ	7.4 モニタのルーチン
コマンド `E`	バイトを書き込む	7.4 モニタのルーチン
コマンド `L`	逆アセンブル	7.4 モニタのルーチン

(c) 2026 masanorisakai / MIT License

項目	内容	節
コマンド `G`	ルーチンを呼ぶ	7.4 モニタのルーチン
コマンド `B`	BASIC を起動	7.4 モニタのルーチン
コマンド `T`	GAME65G を起動	7.4 モニタのルーチン
コマンド `?`	ヘルプ	7.4 モニタのルーチン
ルーチン `$F000` CHRIN	1 文字入力(待つ)	7.4 モニタのルーチン
ルーチン `$F003` CHROUT	1 文字出力	7.4 モニタのルーチン
ルーチン `$F006` ISCNTC	Ctrl-C の確認	7.4 モニタのルーチン
ルーチン `$F009` ウォームスタート	モニタに戻る	7.4 モニタのルーチン
ルーチン `$F00C` GETKEY	待たずに 1 文字入力	7.4 モニタのルーチン

8.2.5 主なアドレスの索引

アドレス	内容	節
`$0000–$00FF`	ゼロページ	7.1.3 RAM の使い分け
`$0006–$000C`	BASIC の USR と変換ルーチンのベクタ	7.1.3 RAM の使い分け
`$00F0–$00FF`	モニタが使うゼロページ	7.1.3 RAM の使い分け
`$0100–$01FF`	スタック	6.2.1 場所と SP
`$0200–$027F`	モニタの行入力バッファ	7.1.3 RAM の使い分け
`$0300–$03F7`	ユーザープログラム用	7.1.3 RAM の使い分け
`$03F8–$03FB`	NMI、IRQ と BRK のハンドラの番地 (RAM)	6.3.6 Glow65 の割り込み
`$0400–$7FFF`	BASIC のプログラムと変数	7.1.3 RAM の使い分け
`$8000–$9FFF`	GAME65G の ROM	7.1.2 Glow65 のメモリマップ
`$A000` `$A001`	UART の状態と制御、データ	7.3 UART
`$A000–$A0FF`	UART(鏡像)	7.1.2 Glow65 のメモリマップ
`$A100–$BFFF`	何もつながっていない	7.1.2 Glow65 のメモリマップ
`$C000–$EFFF`	BASIC の ROM	7.1.2 Glow65 のメモリマップ
`$F000–$FFFF`	モニタの ROM とベクタ	7.1.2 Glow65 のメモリマップ
`$F000–$F00C`	モニタのジャンプテーブル	7.4 モニタのルーチン
`$FFFA–$FFFF`	NMI、リセット、IRQ と BRK のベクタ	3.7 リセットと割り込みのベクタ、6.3.1 4 つの割り込み

(c) 2026 masanorisakai / MIT License

8.2.6 図の一覧

図	タイトル	画像	節
図 1-1	フロントパネル(BASIC の実行中)	frontpanel.png	1.2 6502 の仕様
図 2-1	LDA #$A0 のあと	02-gui-lda-a0.png	2.3 16進数
図 2-2	LDA #$FB のあと	02-gui-lda-fb.png	2.5 負の数と2の補数
図 3-1	リセット直後のパネル	03-reset.png	3.2 レジスタ
図 3-2	STEP 1 回目(SEI のオペコード)	03-cycle-1.png	3.4 フェッチと実行
図 3-3	STEP 3 回目(LDX のオペコード)	03-cycle-3.png	3.4 フェッチと実行
図 3-4	STEP 4 回目(LDX のオペランド)	03-cycle-4.png	3.4 フェッチと実行
図 3-5	STEP 5 回目(LDX の結果)	03-cycle-5.png	3.4 フェッチと実行
図 3-6	INSTR で 3 回目の STEP のあと	03-instr-3.png	3.6 命令の実行例
図 3-7	INSTR で 4 回目の STEP のあと	03-instr-4.png	3.6 命令の実行例
図 4-1	インプライド(`INX`)	04-implied.svg	インプライド(Implied)
図 4-2	アキュムレータ(`ASL A`)	04-accumulator.svg	アキュムレータ(Accumulator)
図 4-3	イミディエイト(`LDA #$10`)	04-immediate.svg	4.3 イミディエイト(Immediate)
図 4-4	ゼロページ(`LDA $10`)	04-zeropage.svg	ゼロページ(Zero Page)
図 4-5	ゼロページ X(`LDA $10,X`)	04-zeropage-x.svg	ゼロページ X(Zero Page,X)
図 4-6	ゼロページ Y(`LDX $10,Y`)	04-zeropage-y.svg	ゼロページ Y(Zero Page,Y)
図 4-7	アブソリュート(`LDA $0400`)	04-absolute.svg	アブソリュート(Absolute)

(c) 2026 masanorisakai / MIT License

図	タイトル	画像	節
図 4-8	アブソリュート X(`LDA $0410,X`)	04-absolute-x.svg	アブソリュート X(Absolute,X)
図 4-9	アブソリュート Y(`LDA $0420,Y`)	04-absolute-y.svg	アブソリュート Y(Absolute,Y)
図 4-10	インダイレクト(`JMP ($0430)`)	04-indirect.svg	4.6 インダイレクト(Indirect)
図 4-11	インデックスト・インダイレクト(`LDA ($20,X)`)	04-indexed-indirect.svg	4.7 インデックスト・インダイレクト(Indexed Indirect)
図 4-12	インダイレクト・インデックスト(`LDA ($30),Y`)	04-indirect-indexed.svg	4.8 インダイレクト・インデックスト(Indirect Indexed)
図 4-13	リラティブ(`BNE`)	04-relative.svg	4.9 リラティブ(Relative)
図 4-14	LDA $0410,X のあと	04-gui-absolute-x.png	4.5 アブソリュート系のモード
図 4-15	LDA $10 のあと	04-gui-zeropage.png	4.5 アブソリュート系のモード
図 4-16	LDA #$10 のあと	04-gui-immediate.png	4.5 アブソリュート系のモード
図 5-1	ASL、LSR、ROL、ROR のビットの動き	05-shift.svg	5.4 論理演算とシフト
図 5-2	BIT が N、V、Z を決める流れ	05-bit.svg	5.4 論理演算とシフト
図 5-3	分岐の飛び先	05-branch.svg	5.5 分岐とジャンプ
図 5-4	JSR と RTS	05-jsr-rts.svg	5.5 分岐とジャンプ
図 5-5	BRK と RTI	05-brk-rti.svg	5.5 分岐とジャンプ
図 5-6	PHA と PLA	05-stack.svg	5.6 スタックとフラグ操作
図 5-7	CLC のあと(V は消灯)	05-gui-adc-before.png	5.3 算術演算と比較
図 5-8	ADC #$50 のあと(N と V が点灯)	05-gui-adc-after.png	5.3 算術演算と比較
図 5-9	BBR のオペコードとビット	05-65c02-bbr.svg	BBR(Branch on Bit Reset)ビットが 0 なら分岐

(c) 2026 masanorisakai / MIT License

図	タイトル	画像	節
図 5-10	ゼロページ・インダイレクト	05-65c02-zp-indirect.svg	ゼロページ・インダイレクト `($nn)`
図 5-11	TSB と TRB	05-65c02-tsb-trb.svg	TSB(Test and Set Bits)ビットを調べて 1 にする
図 6-1	P レジスタの 8 ビット	06-flags.svg	6.1 フラグ(P レジスタ)
図 6-2	CMP のあとのフラグと分岐	06-compare.svg	6.1.8 比較(CMP)のあと
図 6-3	JSR を 2 回重ねたときのスタック	06-stack-nested.svg	6.2.3 JSR を重ねたとき
図 6-4	割り込みの流れ	06-interrupt-flow.svg	6.3.2 受け付けの手順
図 6-5	NMI と IRQ の違い	06-nmi-irq.svg	6.3.3 NMI と IRQ の違い
図 6-6	NMI ボタンを押した直後	06-gui-nmi-pending.png	6.3.7 パネルで割り込みを見る
図 6-7	STEP 1 回目(PC の上位バイトを積む)	06-gui-nmi-push1.png	6.3.7 パネルで割り込みを見る
図 6-8	STEP 3 回目(P を積む)	06-gui-nmi-push3.png	6.3.7 パネルで割り込みを見る
図 6-9	NMI の受け付けの完了	06-gui-nmi-entered.png	6.3.7 パネルで割り込みを見る
図 6-10	ハンドラの PHA のあと	06-gui-nmi-pha.png	6.3.7 パネルで割り込みを見る
図 6-11	RTI のあと	06-gui-nmi-rti.png	6.3.7 パネルで割り込みを見る
図 6-12	IRQ の受け付け直後	06-gui-irq-entered.png	6.3.7 パネルで割り込みを見る
図 6-13	LDA $A001 のあと	06-gui-irq-read.png	6.3.7 パネルで割り込みを見る
図 6-14	IRQ の LED が消える	06-gui-irq-off.png	6.3.7 パネルで割り込みを見る
図 7-1	Glow65 のメモリマップ	07-memory-map.svg	7.1.2 Glow65 のメモリマップ
図 7-2	UART のレジスタ	07-uart.svg	7.3 UART

(c) 2026 masanorisakai / MIT License

図	タイトル	画像	節
図 7-3	ポーリングの流れ	07-polling.svg	7.3.1 ポーリング
図 7-4	受信データがないとき($A000 は $02)	07-gui-status-02.png	7.3.1 ポーリング
図 7-5	h を入力したあと($A000 は $03)	07-gui-status-03.png	7.3.1 ポーリング
図 7-6	$A001 を読んだとき	07-gui-data-read.png	7.3.1 ポーリング
図 7-7	データを読んだあと($A000 は $02)	07-gui-status-after.png	7.3.1 ポーリング

(c) 2026 masanorisakai / MIT License

第 2 部 チュートリアル

短いプログラムを Glow65 に入れて動かし、LED とモニタで結果を確かめます。第 1 部のマニュアルが、その背景にある仕組みを説明します。どちらから読んでもかまいません。サンプルのプログラム(`.s`)は、リポジトリの `tutorial` ディレクトリにあります。

(c) 2026 masanorisakai / MIT License

チュートリアル: はじめてのプログラム(LED カウンタ)

5行の 6502 プログラムを書いて Glow65 に読み込み、LED に 2 進数で数える様子を観察します。以下の内容はすべて Glow65 で実行し、`Tests/Glow65Tests/TutorialTests.swift` のテストで確認しています。

1. プログラム

`counter.s` は、A レジスタに 1 を足し続け、メモリに書き込みます。

```
        lda #$00        ; A = 0
loop:   sta COUNTER     ; A をメモリに書く(COUNTER = $03F0)
        clc             ; 足し算の前にキャリーフラグを消す
        adc #$01        ; A = A + 1
        jmp loop        ; 最初に戻る(ずっと繰り返す)
```

命令	機械語	バスで起きること
`lda #$00`	`A9 00`	読み出し2回: オペコード、値 0
`sta $03F0`	`8D F0 03`	読み出し3回(オペコード、アドレスの下位・上位)のあと、**書き込み**1回
`clc`	`18`	読み出し1回: オペコード
`adc #$01`	`69 01`	読み出し2回
`jmp loop`	`4C 02 03`	読み出し3回。PC が `$0302` に戻る

ループ1周は 11 サイクルです。

2. コンピュータに入れる

11 バイト `A9 00 8D F0 03 18 69 01 4C 02 03` を、RAM の `$0300` に置きます。`$0300–$03F7` は、自作プログラム用に空けてあります(`$03F8–$03FB` は第6章の割り込みベクタです)。(`$0200–$027F` はモニタの行入力バッファです。ここに置くと、起動のために打つコマンドで上書きされます。)

方法 A: バイトを手で入力する。 ツールは要りません。アプリを起動し(`swift run Glow65App`)、POWER を上げ、端末をクリックして、次を入力します。

```
E 0300 A9 00 8D F0 03 18 69 01 4C 02 03
L 0300 0308
```

`E` はバイトを書き込みます。`L` は、それを命令として一覧にします(逆アセンブル)。打ち込んだバイトが、意図したプログラムになっているかを確認できます。次のように表示されれば成功です。

(c) 2026 masanorisakai / MIT License

```
0300  A9 00     LDA   #$00
0302  8D F0 03  STA   $03F0
0305  18        CLC
0306  69 01     ADC   #$01
0308  4C 02 03  JMP   $0302
```

(`D 0300 030F` なら、同じメモリを生のバイトで表示します。)

方法 B: アセンブルしてファイルを読み込む。 cc65(`brew install cc65`)が必要です。結果の `counter.bin` はリポジトリに入っているので、最初のコマンドは省けます。

```
Tools/assemble.sh tutorial/counter.s        # tutorial/counter.bin(11 バイト)を作る
swift run Glow65App --load tutorial/counter.bin@0300
```

`--load` は、POWER を入れるたびに、ファイルを RAM に読み込みます。起動後は **File → Load Program…**(⌘O)でも読み込めます。`counter.bin` を選び、アドレスは `0300` のままにします。電源を入れると RAM が消去されるので、先に POWER を入れてください。

3. 実行する

モニタで次を入力します。

```
G 0300
```

プロンプトは戻ってきません。プログラムは戻らないので、CPU は今、あなたのループを実行しています。(モニタに戻るには **RESET** を押します。RAM は保たれます。)

1MHz では、このループは1秒間に約 90,000 回まわるので、数える様子は追えません。A の LED がでたらめに見えるのは、パネルがレジスタを1秒間に約 60 回しか読み取らないためです。

4. 遅くして観察する

1. **SPEED** を **10 Hz** にします。ループ1周は1秒強になり、バスアクセスが順に表示されます。
2. アドレスバスを見ます。`$0300`、`$0301`、`$0302` と進むので、PC の LED とアドレスの下位の LED が、プログラムの進行に合わせて変わります。
3. **INSTRUCTION AT PC** を見ます。実行している命令が、`0302 8D F0 03 STA $03F0` のように出るので、LED とプログラムを対応づけられます。
4. **SYNC** を見ます。命令の最初のアクセス(オペコードの読み出し)で光ります。
5. **WRITE** を見ます。ループ1周に1回、`sta` のときだけ光ります。そのときアドレスバスは `$03F0`、データバスは A の値を示します。

(c) 2026 masanorisakai / MIT License

6. **RUN/STOP** を STOP にし、**STEP MODE** を CYCLE にして、**STEP** を繰り返し押します。1回押すごとに、バスアクセスが1つ進みます。

最初の1周を、アクセスごとに並べます。

#	アドレス	データ	読み/書き	SYNC	命令
1	`$0300`	`A9`	読み	○	`lda #$00`
2	`$0301`	`00`	読み		
3	`$0302`	`8D`	読み	○	`sta $03F0`
4	`$0303`	`F0`	読み		
5	`$0304`	`03`	読み		
6	`$03F0`	`00`	**書き**		
7	`$0305`	`18`	読み	○	`clc`
8	`$0306`	`69`	読み	○	`adc #$01`
9	`$0307`	`01`	読み		
10	`$0308`	`4C`	読み	○	`jmp $0302`
11	`$0309`	`02`	読み		
12	`$030A`	`03`	読み		

この表示について、2点を知っておいてください。

- 命令を再生している間、A、X、Y、PC、フラグの LED は、命令を実行する**前**の値を示します。命令が終わると変わります。
- 実際の 6502 では `clc` は 2 サイクルかかり、次のバイトをもう1回(ダミーで)読みます。Glow65 はダミーリードを再現しないので、`clc` のアクセスは1回だけ表示されます。サイクルのカウンタは 2 進みます。

5. 結果を見る

RESET を押してループを止め、モニタに戻って、次を入力します。

```
D 03F0 03FF
```

行の先頭のバイトが、プログラムが `$03F0` に最後に書いた値です。1行は 16 バイトです。途中にある 0 以外の4バイト(`$03F8–$03FB`、`24 F4 24 F4`)は、モニタの割り込みベクタで、第6章で説明します。

(c) 2026 masanorisakai / MIT License

6. 試してみる

RESET を押し、`E` でバイトを変えて、`L 0300 0308` で結果を確認し、`G 0300` でもう一度実行します。バイトのアドレスは次のとおりです。

```
0300  A9 00        lda #$00
0302  8D F0 03     sta $03F0
0305  18           clc
0306  69 01        adc #$01
0308  4C 02 03     jmp $0302
```

1. **2ずつ増やす。** `E 0307 02` で `adc #$01` が `adc #$02` になります。A の LED は、いちばん下のビットを飛ばします。
2. **別の場所に書く。** `E 0303 F1` で、`sta` の書き込み先が `$03F1` になります。`D 03F0 03FF` で確認します。
3. **減らしていく。** `E 0305 38 E9 01` で、`clc` / `adc #$01` が `sec` / `sbc #$01` に変わります。長さは同じで、向きが逆になります。`L 0300 0308` を見ると、`$0305` と `$0306` が `SEC` と `SBC #$01` になっています。

慣れたら、`counter.s` を書き換え、`Tools/assemble.sh` をもう一度実行して、新しいファイルを読み込みます。プログラムは、`$0300–$03F7` の 248 バイトに収める必要があります(`ROM/user.cfg` を参照)。

(c) 2026 masanorisakai / MIT License

チュートリアル 2: 足し算とキャリーフラグ

6502 は、1回に 8 ビットずつ足し算をします。255 より大きい数を足すには、下位バイトを足し、次に上位バイトを足して、その間で**キャリー**(繰り上がり)を受け渡します。この章では 16 ビットの足し算を書き、キャリーフラグが下位と上位の間を移る様子を観察します。以下の内容はすべて Glow65 で実行し、`Tests/Glow65Tests/TutorialChapterTests.swift` で確認しています。

1. プログラム

`add.s` は `SUM = A + B` を計算します。数は2バイトで、**下位バイトが先**です。

数	下位バイト	上位バイト
A	`$03F0`	`$03F1`
B	`$03F2`	`$03F3`
SUM	`$03F4`	`$03F5`

```
        clc             ; 下位バイトには、繰り上がりを入れない
        lda A_LO        ; まず下位バイト
        adc B_LO        ; A = A_LO + B_LO
        sta SUM_LO
        lda A_HI        ; 次に上位バイト
        adc B_HI        ; A = A_HI + B_HI + 下位バイトからの繰り上がり
        sta SUM_HI
        rts             ; モニタに戻る
```

`adc` は「キャリー付きの足し算」で、メモリのバイトと**キャリーフラグ**の両方を足します。`clc` で先にキャリーを消すので、下位バイトの足し算は繰り上がりなしで始まります。下位バイトの足し算が 255 を超えると、CPU がキャリーフラグを立て、2回目の `adc` がそれを上位バイトに足します。

プログラムは `rts` で終わります。モニタの `G` はプログラムをサブルーチンとして呼ぶので、`rts` で `*` のプロンプトに戻ります。

2. 読み込む

第1章と同じ方法で `add.bin` を読み込む(`swift run Glow65App --load tutorial/add.bin@0300`、または **File → Load Program…**)か、20 バイトを入力します。

```
E 0300 18 AD F0 03 6D F2 03 8D F4 03 AD F1 03 6D F3 03 8D F5 03 60
L 0300 0313
```

(c) 2026 masanorisakai / MIT License

`L` は次のように表示します。

```
0300  18        CLC
0301  AD F0 03  LDA   $03F0
0304  6D F2 03  ADC   $03F2
0307  8D F4 03  STA   $03F4
030A  AD F1 03  LDA   $03F1
030D  6D F3 03  ADC   $03F3
0310  8D F5 03  STA   $03F5
0313  60        RTS
```

`$0313` より先まで表示すると、`BRK` の行が出ます。何も置いていないメモリのゼロが、そう見えているだけです。`L` はコードとデータを区別できません。

3. 実行する

A と B に数を入れ、プログラムを実行して、結果を見ます。

```
E 03F0 05 00 07 00
G 0300
D 03F0 03F5
```

A は `$0005`、B は `$0007` です。ダンプの先頭6バイトは `05 00 07 00 0C 00` で、SUM は `$000C`(12)です。`*` のプロンプトは自動的に戻ります。

次は、キャリーが必要な足し算です。`$01FF + $0001` を計算します。

```
E 03F0 FF 01 01 00
G 0300
D 03F0 03F5
```

先頭6バイトは `FF 01 01 00 00 02` で、SUM は `$0200` です。下位バイトの `$FF + $01` は1バイトに収まらないので、和の下位バイトは `$00` になり、繰り上がりが次に渡されます。上位バイトの足し算は `$01 + $00 + 1 = $02` になります。

4. キャリーを観察する

プログラムは数マイクロ秒で終わります。観察するために、最後の `rts` を `jmp $0300` に替えて、ループにします。

(c) 2026 masanorisakai / MIT License

```
E 03F0 FF 01 01 00
E 0313 4C 00 03
G 0300
```

プログラムは繰り返し続けるので、プロンプトは戻りません。そのあと、次のようにします。

1. **SPEED** を **10 Hz** にし、**RUN/STOP** を STOP にします。
2. **STEP MODE** を INSTR にして、**INSTRUCTION AT PC** が `0300 18 CLC` になるまで、**STEP** を押します。(CPU はループの途中で止まっているので、最大8回押すことになります。)
3. そのまま STEP を押し続けます。押すごとに、**A** の LED と、STATUS P の行の **C** フラグ(いちばん右の LED)を見ます。

回	押す前の PC の命令	押したあとの A	押したあとの C	起きたこと
1	`0300 CLC`	変わらない	0	キャリーが消える
2	`0301 LDA $03F0`	`$FF`	0	A = A の下位バイト
3	`0304 ADC $03F2`	`$00`	**1**	`$FF + $01 = $100`。下位 8 ビットは 0、キャリーは 1。Z フラグも点灯する
4	`0307 STA $03F4`	`$00`	1	和の下位バイトを保存
5	`030A LDA $03F1`	`$01`	1	A = A の上位バイト。`lda` は C を変えない
6	`030D ADC $03F3`	`$02`	0	`$01 + $00 +` キャリー(1) `= $02`。新しい繰り上がりはなし
7	`0310 STA $03F5`	`$02`	0	和の上位バイトを保存
8	`0313 JMP $0300`	`$02`	0	先頭に戻る

キャリーは3回目に点灯し、6回目で使い切られます。複数バイトの計算は、この仕組みで成り立っています。

5. 試してみる

先に RESET を押し、`E 0313 60` で `rts` を元に戻します(ループ版はモニタに戻りません)。`$03F0–$03F5` の数は残っています。

1. **あふれ。** `E 03F0 FF FF 01 00` のあと `G 0300`、`D 03F0 03F5`。SUM は `$0000` になります。上位バイトから出た繰り上がりはどこにも保存されないので、16 ビットの和は一周して戻ります。

(c) 2026 masanorisakai / MIT License

2. **引き算。** 引き算も同じやり方で、キャリーフラグは「借りなし」を表します。最初の `sbc` の前に `sec` を置きます。`E 0300 38`(`sec`)、`E 0304 ED F2 03`(`sbc $03F2`)、`E 030D ED F3 03`(`sbc $03F3`)でオペコードを替えます。A = `$0200`、B = `$0001`(`E 03F0 00 02 01 00`)なら、SUM は `$01FF` になります。
3. **24 ビットにする。** 各数に3バイト目を足し、プログラムに `lda` / `adc` / `sta` を1組足します。順番は同じです。下位バイトが先で、キャリーを渡していきます。

(c) 2026 masanorisakai / MIT License

チュートリアル 3: UART と話す

ここまでのプログラムは、メモリを書き換えるだけでした。この章では、UART に直接書き込んで、端末に文字を出します。以下の内容はすべて Glow65 で実行し、`Tests/Glow65Tests/TutorialChapterTests.swift` で確認しています。

1. UART は 2 つのアドレス

バスのアドレスは、いつも RAM につながっているわけではありません。Glow65 にはシリアルポート(UART。MC6850 を参考にしたもの)があり、2つのアドレスにあります。そこを読み書きすると、RAM ではなく、デバイスと話すことになります。

アドレス	読む	書く
`$A000`	状態: bit0 = 受信した文字がある、**bit1 = 送信できる**	制御
`$A001`	受信した文字	**この文字を送る**

アプリの端末は、そのポートの向こう側にあります。文字を出すには、状態の bit1 が立つまで待ってから、`$A001` に書きます。(Glow65 の UART は、いつでも送れる状態なので、待ちはすぐ終わります。実際の UART はビットをゆっくり送るので、待つことに意味があります。ここでも、待ちは書いておきます。)

2. プログラム

`hello.s` は「HELLO, GLOW65」と改行を送ります。

```
        ldx #$00            ; X = メッセージの添字
next:   lda message,x       ; 次の文字を取り出す
        beq done            ; 0 のバイトはメッセージの終わり
        tay                 ; UART を調べる間、文字を Y に預けておく
wait:   lda UART_STATUS
        and #$02            ; 送信レジスタは空いているか
        beq wait            ; まだなら、もう一度見る
        tya                 ; 空いた: 文字を A に戻す
        sta UART_DATA       ; 送る
        inx
        jmp next
done:   rts
message:
        .byte "HELLO, GLOW65", $0D, $0A, $00
```

(c) 2026 masanorisakai / MIT License

- `lda message,x` は、`message + X` のバイトを読みます。X を増やしていくと、文章を順に辿れます。
- 状態を A に読み込むと、文字が消えてしまいます。そこで `tay` で文字を Y に預け、`tya` で戻しています。
- `and #$02` で bit1 だけを残します。結果が 0 なら、UART はまだ準備できていないので、`beq wait` でやり直します。
- `$0D, $0A` は復帰と改行で、次のプロンプトが新しい行から始まります。最後の `$00` がメッセージの終わりです。

3. 読み込んで実行する

`hello.bin` を読み込む(`swift run Glow65App --load tutorial/hello.bin@0300`、または **File → Load Program…**)か、コードとメッセージを別々に入力します(合わせて 40 バイト)。

```
E 0300 A2 00 BD 18 03 F0 10 A8 AD 00 A0 29 02 F0 F9 98 8D 01 A0 E8 4C 02 03 60
E 0318 48 45 4C 4C 4F 2C 20 47 4C 4F 57 36 35 0D 0A 00
L 0300 0317
```

`L` は次のように表示します。

```
0300  A2 00     LDX   #$00
0302  BD 18 03  LDA   $0318,X
0305  F0 10     BEQ   $0317
0307  A8        TAY
0308  AD 00 A0  LDA   $A000
030B  29 02     AND   #$02
030D  F0 F9     BEQ   $0308
030F  98        TYA
0310  8D 01 A0  STA   $A001
0313  E8        INX
0314  4C 02 03  JMP   $0302
0317  60        RTS
```

メッセージは `$0318` から始まります。`$0317` より先まで表示すると、文字が意味のない命令(`PHA` など)として出ます。`L` はコードとデータを区別できません。

実行します。

```
G 0300
```

(c) 2026 masanorisakai / MIT License

```
HELLO, GLOW65
*
```

文字はあなたのプログラムが出したもので、`rts` でモニタに戻ります。

4. パネルで見る

全速でも、文字ごとに **WRITE** と **TX** の LED が光るのが見えます。1文字を詳しく見るには、プログラムをループにします(`E 0317 4C 00 03` で `rts` を `jmp $0300` に替える)。`G 0300` で実行し、そのあと **SPEED** を **10 Hz**、**RUN/STOP** を STOP にします。遅くする前に、ループがメッセージを何度も出すので、端末は最初、行でいっぱいになります。次に **STEP MODE** を CYCLE にし、INSTRUCTION AT PC が `STA $A001` のときに **WRITE** の LED が点灯するところまで、**STEP** を押します。

ループは、適当な場所で止めたので、最初に見える文字は、メッセージのどの文字かわかりません。WRITE が点灯するまで STEP を押し続け、次の行を読んでください。

LED の行	プログラムが文字を書き込むときに見えるもの
ADDRESS BUS	いつも `$A001`: **A15**、**A13**、**A0** が点灯
DATA BUS	その文字の ASCII コード。`H` は `$48` (**D6** と **D3** が点灯)、続く `E`、`L`、`L`、`O` は `$45`、`$4C`、`$4C`、`$4F`
WRITE	点灯

書き込みの前には、必ず `$A000` の読み出しがあります。UART に「送れますか」と聞いているところです。

5. 近道: モニタのルーチンを使う

文字を待って送るルーチンは、モニタにあります。`$F003` の `CHROUT` です(アドレスは固定なので、自作プログラムから呼べます)。`hello_monitor.s` は、hello.s から待ちのループを除いたものです。

```
        ldx #$00
next:   lda message,x
        beq done
        jsr $F003           ; モニタが待って、A を送る
        inx
        jmp next
done:   rts
```

31 バイトです。入力するなら、メッセージを `$030F` に置きます。

(c) 2026 masanorisakai / MIT License

```
E 0300 A2 00 BD 0F 03 F0 07 20 03 F0 E8 4C 02 03 60
E 030F 48 45 4C 4C 4F 2C 20 47 4C 4F 57 36 35 0D 0A 00
G 0300
```

同じ文字が出ます。BASIC の出力も、この `CHROUT` を通っています。

6. 試してみる

1. **自分のメッセージ。** RESET を押し、`E 0318 47 4F 4F 44 0D 0A 00` (GOOD、改行、終わりの印)を入力して、`G 0300` を実行します。`0D 0A` を省くと、次のプロンプトが文字のすぐ後ろに出ます。メッセージは短くしてください。`$03F0` にぶつかってはいけません。
2. **別の ASCII コード。** `E 0318 41 42 43 0D 0A 00` は `ABC` を出します。文字は1つずつコードが増えるので、各バイトに 3 を足せば `DEF` になります。
3. **受信。** 状態の bit0 は文字が届いたことを示し、`$A001` を読むとその文字が返ります。bit0 を待ち、文字を読み、そのまま送り返すプログラムがエコーです。同じ部品で、書いてみてください。

(c) 2026 masanorisakai / MIT License

チュートリアル 4: キーボードを読む、エコーのプログラム

第3章では、UART から文字を送り出しました。この章では、UART から文字を読み込みます。プログラムはキーを待ち、読み取って、そのまま送り返します。打った文字が、画面に出るようになります。以下の内容はすべて Glow65 で実行し、`Tests/Glow65Tests/TutorialChapterTests.swift` で確認しています。

1. 画面にエコーが必要な理由

アプリの端末は、キーを打っても、自分では表示しません。キーは UART に届き、端末が表示するのは、UART から送り出された文字だけです。モニタで文字を打つと見えるのは、**モニタ**が、キーを1つずつ読んで、送り返しているからです。

打った文字を画面に出したいプログラムは、同じことをする必要があります。読んで、送り返す。これがエコーです。

受信は、送信と同じ2つのレジスタを使います。

アドレス	読む	書く
`$A000`	状態: **bit0 = 受信した文字がある**、bit1 = 送信できる	制御
`$A001`	**受信した文字**(読むと、UART から取り除かれる)	文字を送る

プログラムには、キーがいつ押されるかがわかりません。そこで**ポーリング**をします。bit0 が立つまで、状態を何度も読み直します。

2. プログラム

`echo.s`:

(c) 2026 masanorisakai / MIT License

```
loop:   lda UART_STATUS
        and #$01            ; 文字が届いたか
        beq loop            ; まだ: もう一度見る
        lda UART_DATA       ; 届いた: 読む
        cmp #$0D            ; Enter か
        beq done            ; そうなら終わる
        tay                 ; 文字を Y に預ける
send:   lda UART_STATUS
        and #$02            ; 送信できるか
        beq send            ; まだ: もう一度見る
        tya
        sta UART_DATA       ; 送り返す
        jmp loop
done:   lda #$0D            ; 改行を送る
        jsr CHROUT
        lda #$0A
        jsr CHROUT
        rts
```

- 最初の3行が、待ちのループです。`and #$01` は状態の bit0 だけを残します。0 の間は、`beq loop` で、また最初に戻ります。
- `cmp #$0D` は、A と Enter のコードを比べます。等しければ Z フラグが立ち、`beq done` が飛びます。この出口がないと、プログラムはモニタに戻りません。
- 真ん中は、第3章の送信のコードです。
- 最後は、モニタの `CHROUT`(`$F003`)で、復帰と改行を送ります。次のプロンプトが、新しい行から始まります。

3. 読み込んで実行する

`echo.bin` を読み込む(`swift run Glow65App --load tutorial/echo.bin@0300`、または **File → Load Program…**)か、40 バイトを入力します。

```
E 0300 AD 00 A0 29 01 F0 F9 AD 01 A0 C9 0D F0 0F A8 AD 00 A0 29 02 F0 F9 98 8D 01 A0 4C
00 03 A9 0D 20 03 F0 A9 0A 20 03 F0 60
L 0300 0327
```

`L` は次のように表示します。

(c) 2026 masanorisakai / MIT License

```
0300  AD 00 A0  LDA   $A000
0303  29 01     AND   #$01
0305  F0 F9     BEQ   $0300
0307  AD 01 A0  LDA   $A001
030A  C9 0D     CMP   #$0D
030C  F0 0F     BEQ   $031D
030E  A8        TAY
030F  AD 00 A0  LDA   $A000
0312  29 02     AND   #$02
0314  F0 F9     BEQ   $030F
0316  98        TYA
0317  8D 01 A0  STA   $A001
031A  4C 00 03  JMP   $0300
031D  A9 0D     LDA   #$0D
031F  20 03 F0  JSR   $F003
0322  A9 0A     LDA   #$0A
0324  20 03 F0  JSR   $F003
0327  60        RTS
```

実行して、文字を打ちます。

```
G 0300
hello
```

打った文字は、モニタではなく、あなたのプログラムが送り返しています。**Enter** を押すと、プログラムが改行を送って戻り、`*` のプロンプトが帰ってきます。

```
hello
*
```

4. パネルで見る

1. `G 0300` を、全速で実行します。プログラムはキーを待っているので、プロンプトは戻りません。
2. **SPEED** を **10 Hz** にして、端末で1文字打ちます。まだ何も表示されません。CPU が遅くて、UART を見に行っていないためです。文字は、UART の中で待っています。
3. **INSTRUCTION AT PC** を見ます。`LDA $A000`、`AND #$01`、`BEQ $0300` を繰り返しています。ポーリングのループです。`$A000` を読むたびに、アドレスバスは `$A000`(**A15** と **A13** が点灯)になり、**READ** が点灯します。
4. 次にループを回ったとき、状態の bit0 が立っているので、`beq` は飛びません。プログラムが `$A001` を読みます。アドレスバスは `$A001`(**A15**、**A13**、**A0**)、データバスは、その文字の ASCII コード(`h`

(c) 2026 masanorisakai / MIT License

は `$68` で、**D6**、**D5**、**D3**)になり、**RX** の LED が点灯します。
5. 数命令あと、プログラムが同じコードを `$A001` に書きます。今度は **WRITE** と **TX** が点灯し、端末に文字が出ます。

Enter を押す前に、**SPEED** を 1 MHz に戻してください。モニタも、また速くなります。

5. 試してみる

先に RESET を押します。RAM に読み込んだプログラムは、残っています。

1. **別のキーで終わる。** `E 030B 2E` で、`cmp #$0D` が `cmp #$2E` に変わり、Enter の代わりに `.`(ASCII の `$2E`)で終わります。このとき Enter は、ほかのキーと同じようにエコーされ、行の先頭に戻るだけで、改行はされません。

2. **大文字にする。** `echo_upper.s`(50 バイト、`echo_upper.bin`)は、`a`〜`z` を `A`〜`Z` にして返します。キーを読んでから送るまでの間に、次を足しています。

   ```
   cmp #$61            ; 'a'
   bcc send_it         ; 'a' より小さい: そのまま
   cmp #$7B            ; 'z' の次
   bcs send_it         ; 'z' 以上: そのまま
   and #$DF            ; bit5 を消す
   ```

   ASCII では、`a`〜`z` が `$61`〜`$7A`、`A`〜`Z` が `$41`〜`$5A` です。違いは bit5(`$20`)だけなので、`and #$DF` で、小文字が大文字になります。`cmp` は、A がオペランド以上のときにキャリーフラグを立てます。`bcc` と `bcs` は、それを調べています。`echo_upper.bin` を読み込んで実行し、`Hello, World 123` と打ってみてください。

3. **打ったキーの数を数える。** 空いているメモリ(例: `$03F0`)に数を置き、文字を読むたびに 1 を足します。Enter を押したあと、`D 03F0 03FF` で数を見ます。

(c) 2026 masanorisakai / MIT License

チュートリアル 5: BASIC から機械語を呼ぶ(USR)

BASIC は、書くのは楽で、動きは遅い言語です。機械語は、その逆です。BASIC の `USR` 関数を使うと、両方のいいところが使えます。プログラムは BASIC で書き、必要なところだけ、機械語のルーチンを呼びます。この章のルーチンは、数を2倍にします。以下の内容はすべて Glow65 で実行し、`Tests/Glow65Tests/TutorialUSRTests.swift` で確認しています。

1. USR の仕組み

`USR(X)` は、次の3つのことをします。

1. BASIC が `X` を計算し、**FAC**(浮動小数点のアキュムレータ)に置きます。
2. BASIC が、ゼロページの **USR ベクタ**に飛びます。10 番地(`$0A`)にある `JMP` 命令です。続く 11 番地と 12 番地の2バイトが、飛び先のアドレスです。
3. 機械語のルーチンが `rts` で終わると、BASIC は、そのとき FAC にある値を、`USR(X)` の値として受け取ります。

ベクタは、自分の目で見られます。BASIC を起動した直後に、次を実行します。

```
PRINT PEEK(10);PEEK(11);PEEK(12)
 76  241  207
```

76 は `$4C` で、`JMP` のオペコードです。241 と 207 は、`$CFF1` の下位と上位で、BASIC の中のエラー処理です。そのため、この時点で `PRINT USR(21)` を実行すると、`?ILLEGAL QUANTITY ERROR` になります。自分のルーチンを使うには、**そのアドレスを 11 番地(下位)と 12 番地(上位)に POKE** します。

FAC は浮動小数点の数なので、機械語では扱いにくいものです。BASIC は、ゼロページのベクタ経由で、2つの補助ルーチンを用意しています。

ベクタ	指す先	働き
`$06–$07`	AYINT	FAC → 16 ビットの符号付き整数。上位バイトが `FACMO`(`$B4`)、下位バイトが `FACLO`(`$B5`)
`$08–$09`	GIVAYF	16 ビットの符号付き整数(A = 上位バイト、Y = 下位バイト)→ FAC

2. プログラム

`usr_double.s`:

(c) 2026 masanorisakai / MIT License

```
        jsr to_integer  ; 引数が整数になって、FACMO:FACLO に入る
        asl FACLO       ; 下位バイトを左にずらす。bit7 がキャリーに入る
        rol FACMO       ; キャリーを上位バイトに回し入れる: 16 ビットの X * 2
        lda FACMO       ; A = 上位バイト
        ldy FACLO       ; Y = 下位バイト
        jsr to_fac      ; 整数を FAC に戻す
        rts

to_integer:
        jmp (ADRAYI)
to_fac: jmp (ADRGAY)
```

- `jsr` はベクタ経由では飛べませんが、`jmp (ベクタ)` は飛べます。そこで、`jsr` で小さな中継(スタブ)を呼び、スタブが `jmp (ADRAYI)` をします。BASIC のルーチンが `rts` で終わると、あなたの `jsr` の次の命令に戻ります。
- `asl FACLO` と `rol FACMO` は、16 ビットの左シフトで、数を2倍にします。第2章のキャリーと同じ考え方です。

3. 読み込む

ルーチンは、これまでと同じ `$0300` に置きます。BASIC は、自分のプログラムと変数を `$0400` から先に置くので、`$0300–$03F7` には触れません。先にファイルを読み込み、そのあと BASIC を起動します。

```
swift run Glow65App --load tutorial/usr_double.bin@0300
```

POWER を入れ、モニタで `B` と入力して、BASIC を起動します。2つの質問は、Return で進みます。(先にモニタで 21 バイトを入力する方法もあります。`E 0300 20 0F 03 06 B5 26 B4 A5 B4 A4 B5 20 12 03 60 6C 06 00 6C 08 00`、確認に `L 0300 0313`、そのあと `B` です。)

`L 0300 0313` は次のように表示します。

```
0300  20 0F 03  JSR   $030F
0303  06 B5     ASL   $B5
0305  26 B4     ROL   $B4
0307  A5 B4     LDA   $B4
0309  A4 B5     LDY   $B5
030B  20 12 03  JSR   $0312
030E  60        RTS
030F  6C 06 00  JMP   ($0006)
0312  6C 08 00  JMP   ($0008)
```

(c) 2026 masanorisakai / MIT License

4. 呼ぶ

ルーチンの場所を BASIC に教えます。`$0300` は 768 なので、下位バイトが 0、上位バイトが 3 です。

```
POKE 11,0:POKE 12,3
PRINT USR(21)
 42
PRINT USR(1000)
 2000
PRINT USR(-5)
-10
```

プログラムの中でも使えます。

```
FOR I=1 TO 5:PRINT USR(I);:NEXT
 2  4  6  8  10
```

BASIC が、自分のループから、あなたのルーチンを呼んでいます。なお、BASIC は起動のたびに、ベクタをエラー処理に戻します。BASIC を起動し直したとき(RESET を押して `B` を入力したときも)は、もう一度 11 番地と 12 番地に POKE してください。

5. ルーチンの限界

整数は、-32768 から 32767 までの 16 ビットの符号付きで、`asl` と `rol` は、ビットをずらすだけです。

```
PRINT USR(16383)
 32766
PRINT USR(16384)
-32768
PRINT USR(20000)
-25536
```

`16384 × 2 = 32768` は収まらず、最上位ビットが符号になって、結果が負になります。機械語のルーチンは、命令に書いたとおりに動くだけで、範囲を確かめてはくれません。

6. 機械語から文字を出す

`usr_hello.s`(37 バイト)は、第3章と同じように、モニタの `CHROUT`(`$F003`)でメッセージを送ります。引数は使わず、FAC にも触れません。同じ方法で読み込み(`--load tutorial/usr_hello.bin@0300`、または `E 0300 A2 00 BD 0F 03 F0 07 20 03 F0 E8 4C 02 03 60` と、メッセージの `E 030F 48 45 4C 4C 4F 20 46 52 4F 4D 20 41 53 53 45 4D 42 4C 59 0D 0A 00`)、次を実行します。

(c) 2026 masanorisakai / MIT License

```
POKE 11,0:POKE 12,3
A=USR(0)
HELLO FROM ASSEMBLY
10 FOR I=1 TO 3:A=USR(0):NEXT
RUN
HELLO FROM ASSEMBLY
HELLO FROM ASSEMBLY
HELLO FROM ASSEMBLY
```

`A=USR(0)` は、結果が要らないルーチンを呼ぶときの、ふつうの書き方です。

7. 試してみる

先に、`usr_double` を読み込み直します。

1. **2倍ではなく、半分にする。** `$0303–$0306` の4バイト(`asl $B5` / `rol $B4`)を、`lsr $B4` / `ror $B5`(`46 B4 66 B5`)に替えます。BASIC の `POKE` なら、771 から 774 番地です。

   ```
   POKE 771,70:POKE 772,180:POKE 773,102:POKE 774,181
   PRINT USR(42)
    21
   ```

   ルーチンは RAM にあるので、BASIC から書き換えられます。`lsr` は上位バイトを右にずらし、bit0 をキャリーに入れます。`ror` はそのキャリーを、下位バイトの最上位ビットに回し入れます。

2. **限界を探す。** `PRINT USR(32767)` は `–2` になります(2倍した 65534 が、一周して戻ります)。`PRINT USR(40000)` は `?ILLEGAL QUANTITY ERROR` です。引数そのものが -32768 から 32767 の外なので、最初の補助ルーチン(FAC → 整数)が、ルーチンが数を使う前に、エラーにします。

3. **100 を足す。** シフトを足し算に替えます。`clc`、`lda FACLO`、`adc #100`、`sta FACLO` と書き、キャリーを `FACMO` に足します(`inc FACMO` を `bcc` で飛び越す)。ルーチンが長くなるので、新しい `.s` ファイルに書き、`Tools/assemble.sh` でアセンブルして、新しい `.bin` を読み込みます。

8. 覚えておくこと

- ルーチンは、`$0300–$03F7` を自由に使えます。FAC と、BASIC のほかのゼロページは、BASIC が期待するとおりにだけ使ってください。BASIC は、ゼロページのほとんどを使っているので、ほかのゼロページのアドレスは避けます。
- `USR` で呼ばれるルーチンは、`rts` で終わる必要があります。
- 機械語から、モニタのルーチンも呼べます(`$F003` の `CHROUT`、`$F00C` の `GETKEY`)。

(c) 2026 masanorisakai / MIT License

チュートリアル 6: 割り込み

ここまでのプログラムは、自分から確かめにいっていました。UART の状態をループで読み続け、キーが来るのを待っていました。**割り込み**は、この関係を逆にします。ハードウェアが信号を出すと、CPU は実行中の作業を中断して短いルーチン(**ハンドラ**)を実行し、そのあと元の場所にぴったり戻ります。この章では、6502 の2本の割り込み線 **NMI** と **IRQ** を使い、CPU がハンドラに入って出る様子をパネルで観察します。以下の内容はすべて Glow65 で実行し、`Tests/Glow65Tests/TutorialInterruptTests.swift` で確認しています。

1. 割り込みで CPU がすること

CPU は、命令を1つ実行する前に、2つの入力を調べます。

入力	出すもの	プログラムで止められるか
NMI(マスク不可)	パネルの **NMI** ボタン	止められません。I フラグが立っていても受け付けます。1回押すと1回だけ(エッジで)起きます
IRQ	UART(受信文字があり、割り込みを許可しているとき)	止められます。**I** フラグが 1 の間は無視します。`sei` で I を立て、`cli` で下ろします。IRQ は**レベル**で働きます。線が上がっている間、CPU は何度でも戻ってきます

割り込みを受け付けると、CPU は次のことをします。

1. **PC** の上位バイト、PC の下位バイト、**P** レジスタの順に、スタックに積みます(`$01xx` への3回の書き込み)。
2. **I** フラグを立てます。ハンドラが、IRQ でさらに中断されないようにするためです。
3. ベクタからハンドラのアドレスを読みます。NMI は `$FFFA/$FFFB`、IRQ は `$FFFE/$FFFF` です。
4. そこへジャンプします。

ハンドラは `rti` で終わります。`rti` は P、続いて PC を取り出すので、プログラムはフラグも含めて元の状態で再開します。`rts` と違って、`rti` はアドレスに 1 を足しません。積まれた PC が、すでに次の命令を指しているからです。

CPU が保存するのは PC と P だけです。ハンドラで A、X、Y を使うなら、自分で保存して戻します(`pha` … `pla`)。そうしないと、主プログラムのレジスタが壊れます。

2. このボードのベクタ

`$FFFA` と `$FFFE` のベクタは ROM にあり、書き換えられません。そこでモニタは、2つの小さなジャンプ命令を置き、RAM にある2つのアドレスを経由して飛ぶようにしています。

(c) 2026 masanorisakai / MIT License

RAM のアドレス	内容	用途
`$03F8/$03F9`	NMI ハンドラのアドレス(下位、上位)	NMI
`$03FA/$03FB`	IRQ ハンドラのアドレス(下位、上位)	IRQ と `BRK`

リセット後は、どちらも `rti` だけのルーチンを指しています。予期しない割り込みが来ても、何も起きません。自分のハンドラを使うには、そのアドレスをここに書き込みます。(自作プログラム用のメモリは `$0300-$03F7` になりました。)

3. NMI: ボタンを押した回数を数える

`tutorial/nmi_count.s` は、ハンドラを登録して、`jmp loop` で空回りします。NMI のたびにハンドラが実行され、`COUNT`(`$03F0`)に 1 を足して `!` を出力します。

```
start:  lda #<handler           ; NMI ベクタをハンドラに向ける
        sta NMIVEC
        lda #>handler
        sta NMIVEC+1
        lda #$00
        sta COUNT
loop:   jmp loop                ; 主プログラムは何もしない

handler:
        pha                     ; CPU が保存したのは PC と P だけ。A は自分で保存する
        inc COUNT
wait:   lda UART_STATUS
        and #$02                ; 送信可?
        beq wait
        lda #'!'
        sta UART_DATA
        pla
        rti                     ; P と PC を戻す(I フラグも元に戻る)
```

実行します。

```
swift run Glow65App --load tutorial/nmi_count.bin@0300
```

POWER を入れて、`G 0300` と入力します。CPU は空回りしているので、端末にプロンプトは出ません。パネルの **NMI** ボタンを押します。押すたびに `!` が1つ出ます。止めるには RESET を押します(RESET は RAM を保持します)。そのあと、回数を確認します。

(c) 2026 masanorisakai / MIT License

```
D 03F0 03F0
```

最初の1バイトが、押した回数です。

4. パネルで割り込みを観察する

SPEED を 1Hz か 10Hz、STEP MODE を CYCLE にします。または RUN/STOP で CPU を止めて、STEP を使います。プログラムが `$030F` の `jmp loop` で空回りしている間に **NMI** を押し、STEP を進めると、割り込みの受け付けがバスアクセスとして見えます。

#	アドレス	データ	R/W	内容
1	`$01FD`	`03`	書き込み	PC の上位バイトを積む
2	`$01FC`	`0F`	書き込み	PC の下位バイトを積む
3	`$01FB`	`26`	書き込み	P を積む(スタック上では bit5 が常に 1)
4	`$FFFA`	`1E`	読み出し	NMI ベクタの下位バイト
5	`$FFFB`	`F4`	読み出し	NMI ベクタの上位バイト

この値は `nmi_count` を実行したときのものです。スタックポインタは `$FD`(モニタの `G` コマンドが、戻りアドレスをすでに積んでいます)、PC は `$030F`(`jmp loop`)でした。5つ目のあと、PC は `$F41E` です。ここはモニタの `jmp (NMIVEC)` で、次の数ステップで `$03F8/$03F9` を読み、CPU は `$0312` にある自分の `handler` に着きます。最初のアクセスには SYNC が付きません。割り込みの受け付けでは、オペコードをフェッチしないからです。実機の 6502 はここに 7 サイクルかけ、余分なダミーリードが2回あります。Glow65 はこれを再現しないので、アクセスは5つに見えます。

`rti` のあと、P と PC の LED は元の値に戻り、スタックポインタも `$FD` に戻ります。

5. IRQ: 割り込みでエコーする

第4章のプログラムは、1文字ごとにループで待っていました。IRQ を使うと、主プログラムは別の仕事をしていて、文字が届いたときに UART が割り込みます。`tutorial/irq_echo.s` は、16 ビットのカウンタ(`$03F0` が下位、`$03F1` が上位)を数え続け、その間にハンドラが打った文字をエコーします。

UART の制御レジスタ(`$A000`)に、新しいビットがあります。

ビット	意味
制御レジスタの bit7	1 で、受信した文字が待っている間、IRQ を出す
状態レジスタの bit7	1 は、この UART が割り込みを要求している

準備は次の順に行います。

(c) 2026 masanorisakai / MIT License

```
        sei                     ; 準備の間は割り込みを受けない
        ...                     ; IRQVEC($03FA)にハンドラのアドレスを書く
        lda #$80                ; UART 制御の bit7: 文字が届いたら IRQ を出す
        sta UART_STATUS         ; ($A000 は、読むと状態、書くと制御)
        cli                     ; CPU が IRQ を受け付けるようにする
```

ハンドラは、文字を読んでエコーし、Enter を終了の合図として扱います。

```
handler:
        pha
        lda UART_DATA           ; データレジスタを読むと、IRQ の要求が下がる
        cmp #$0D
        beq enter
        pha
wait:   lda UART_STATUS
        and #$02
        beq wait
        pla
        sta UART_DATA           ; エコー
        pla
        rti
enter:  sta DONE                ; A = $0D(0 ではない): 主プログラムに終了を知らせる
        pla
        rti
```

`swift run Glow65App --load tutorial/irq_echo.bin@0300` で起動し、`G 0300` を実行します。打った文字はエコーされます。Enter を押すと、モニタのプロンプトが戻ります。カウンタを見ます。

```
D 03F0 03F2
```

`$03F0/$03F1` は、文字を打っている間に主プログラムが数えた値です。`$03F2` は `0D` で、ハンドラが立てたフラグです。

`lda UART_DATA` の行が重要です。IRQ はレベルで働きます。UART は、未読の文字がある間、線を上げ続けます。`$A001` を読まずに戻るハンドラは、すぐにまた割り込まれ、それが延々と続きます。

6. BRK も割り込み

`brk`(オペコード `$00`)は、PC に 2 を足した値と P を積み、I を立て、**IRQ** ベクタ経由でジャンプします。ハードウェアの IRQ と同じ経路なので、`$03FA/$03FB` が両方に使われます。`brk` の次の1バイトは、戻ると

(c) 2026 masanorisakai / MIT License

きに飛ばされます。これは自由に使える「署名」バイトで、ハンドラが読めば、どの `brk` から来たかがわかります。(積まれた P の B フラグで、`brk` から来たのか IRQ から来たのかを区別できます。)

7. 試してみよう

1. **SP を見る。** STEP で進めながら NMI を押します。3回積んだあと、SP の LED は `$FD` から `$FA` になり、`rti` のあとで `$FD` に戻ります。
2. **数え方を変える。** `nmi_count.s` で、`!` の代わりに回数を文字で出力します。COUNT に `#'0'` を足して出力します(9回まで)。
3. **読み忘れる。** `irq_echo.s` のハンドラの最初の命令を `rti` にして、`$A001` を読まないようにします。1文字打つと、主プログラムの数え上げが止まり、エコーも出ません。なぜでしょうか。(CPU を遅くして PC を見てください。割り込みの受け付け、ハンドラ、`rti` を、ひたすら繰り返しています。)回復するには RESET を押します。
4. **BRK を使う。** `E 0300 A9 01 00 60 8D F0 03 60` と入力して、`G 0300` を実行します。`$0302` の `brk` は、既定の `rti` ハンドラを通って、`$0303` の `60` を飛ばし、`$0304` から再開します。`D 03F0 03F0` で `01` が見えます。
5. **止める。** `sei` は IRQ を止めますが、NMI は止めません。`nmi_count.s` の `jmp loop` の前に `sei` を入れて、それでも NMI を押してみます。

8. 覚えておくこと

- NMI はマスクできません。IRQ は I フラグでマスクされます。
- CPU が保存するのは PC と P だけです。A、X、Y を使うなら、ハンドラで保存します。
- ハンドラは `rts` ではなく `rti` で終わります。
- レベルで働く IRQ は、要因の側で解除します(ここでは `$A001` を読む)。
- モニタのウォームスタートは UART の割り込みを切ります。割り込みを許可したまま `rts` でモニタに戻っても安全です。

(c) 2026 masanorisakai / MIT License

チュートリアル 7: 65C02 に切り替える

65C02 は 6502 の後継です。6502 のプログラムをそのまま動かせて、プログラムを短くできる命令が加わっています。Glow65 は既定では 6502 として動き、指定すると 65C02 として動きます。この章では、小さなプログラムを CPU ごとに1つずつ、合わせて2通り書いてサイズを比べ、65C02 用のプログラムを両方のマシンで動かして、未定義命令がどう動くかを見ます。以下の内容はすべて Glow65 で実行し、`Tests/Glow65Tests/TutorialChapter65C02Tests.swift` で確認しています。

1. 65C02 で加わったもの

この章では、新しい命令のうち4つを使います。

命令	バイト数	動作	6502 でのやり方
`stz addr`	2〜3	メモリに 0 を書く	先に `lda #$00`、次に `sta addr`
`inc a`	1	A に 1 を足す	`clc` と `adc #$01`(3 バイト)
`bra label`	2	必ず分岐する	`jmp label`(3 バイト)
`phx` / `plx`	1	X をスタックに積む/取り出す(Y は `phy` / `ply`)	`txa`、`pha`、`pla`、`tax`(A を経由)

65C02 には、このほかにも新しいアドレッシングモードと命令(`wai`、`stp`、ビット命令など)があります。詳しくはマニュアルの 4.11 65C02 で加わるモードと 5.7 65C02 を見てください。Glow65 での実装は実装仕様書にあります。

2. 同じプログラムを2通り

どちらのプログラムも、2つのことをします。`$03D0-$03DF` の16バイトを 0 で埋め、そのあと16ビットの値を 0 から `$0300` まで数えて `COUNT`(`$03F0` が下位、`$03F1` が上位)に書きます。数えるときは、上位バイトを A に、下位バイトをメモリに持ちます。

`clear_6502.s` が 6502 用です。

(c) 2026 masanorisakai / MIT License

```
        ldx #$0F        ; X = 15: index of the last byte of the area
        lda #$00        ; a 6502 cannot store a constant: A must hold the 0
clr:    sta BUF,x       ; clear one byte
        dex
        bpl clr         ; repeat until X wraps from 0 to $FF
        sta COUNT       ; A is still 0
loop:   cmp #$03        ; A = high byte of the count: finished?
        beq done
        inc COUNT       ; low byte + 1
        bne loop        ; no wrap: next
        clc             ; the low byte wrapped: high byte + 1
        adc #$01
        jmp loop        ; the 6502 has no unconditional short branch
done:   sta COUNT+1
        rts
```

`clear_65c02.s` が 65C02 用です。

```
        ldx #$0F        ; X = 15: index of the last byte of the area
clr:    stz BUF,x       ; STZ stores a zero without using A
        dex
        bpl clr         ; repeat until X wraps from 0 to $FF
        stz COUNT       ; the low byte of the count
        lda #$00        ; A = high byte of the count
loop:   cmp #$03        ; finished?
        beq done
        inc COUNT       ; low byte + 1
        bne loop        ; no wrap: next
        inc a           ; the low byte wrapped: INC A adds 1 to A
        bra loop        ; BRA: unconditional branch, 2 bytes
done:   sta COUNT+1
        rts
```

65C02 版は、A に 0 を読み込む代わりに `stz`、`clc` と `adc #$01` の代わりに `inc a`、`jmp` の代わりに `bra` を使っています。途中のループの形は同じです。

3. アセンブルする

これまでの章と同じく、`Tools/assemble.sh` には `cc65` パッケージが必要です。65C02 のソースは `.setcpu "65C02"` で始まります。これで `ca65` が新しい命令を受け付けます。書かないと `stz` はエラーに

(c) 2026 masanorisakai / MIT License

なります。6502 のソースには `.setcpu "6502"` と書いてあります。

```
Tools/assemble.sh tutorial/clear_6502.s
Tools/assemble.sh tutorial/clear_65c02.s
```

```
tutorial/clear_6502.bin: 32 bytes (load at $0300)
tutorial/clear_65c02.bin: 29 bytes (load at $0300)
```

65C02 版は3バイト短くなります。`inc a` で2バイト、`bra` で1バイトです。このプログラムでは `stz` は短くなる要因になっていません。65C02 版も上位バイトのために `lda #$00` が必要で、6502 版は1回の `lda #$00` で両方をまかなえるからです。`.bin` は `tutorial/` にコミットしてあるので、この手順は飛ばせます。

4. 65C02 で動かす

Glow65 を 65C02 として起動し、65C02 用のバイナリを読み込みます。`--cpu 65c02` は、どちらのプログラムにも使えます。

```
swift run glow65-cli --cpu 65c02 --load tutorial/clear_65c02.bin@0300
```

または

```
swift run Glow65App --cpu 65c02 --load tutorial/clear_65c02.bin@0300
```

アプリでは、POWER を上げて、ターミナルをクリックします。消えていく様子が分かるように、最初に領域を `$AA` で埋めます。

```
E 03D0 AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
```

プログラムを一覧します。モニタの `L` コマンドは実行時に CPU の種類を判定するので、65C02 では新しい命令を名前で表示します。

```
L 0300 031C
```

(c) 2026 masanorisakai / MIT License

```
0300  A2 0F     LDX   #$0F
0302  9E D0 03  STZ   $03D0,X
0305  CA        DEX
0306  10 FA     BPL   $0302
0308  9C F0 03  STZ   $03F0
030B  A9 00     LDA   #$00
030D  C9 03     CMP   #$03
030F  F0 08     BEQ   $0319
0311  EE F0 03  INC   $03F0
0314  D0 F7     BNE   $030D
0316  1A        INC   A
0317  80 F4     BRA   $030D
0319  8D F1 03  STA   $03F1
031C  60        RTS
```

実行して、領域とカウンタをダンプします。

```
G 0300
D 03D0 03F1
```

```
03D0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
03E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
03F0: 00 03 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

`$03D0` からの16バイトは 0 に戻り、`$03F0/$03F1` は `00 03` です。下位バイトが先なので、`$0300` を表します。`$03F8–$03FB` は、第6章の割り込みベクタです。

5. 6502 で動かす

次に、同じバイナリを既定の CPU で起動します。`--cpu` は付けません(`--cpu 6502` と書いても同じです)。

```
swift run glow65-cli --load tutorial/clear_65c02.bin@0300
```

もう一度、領域を `$AA` で埋めて、プログラムを一覧します。

```
E 03D0 AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
L 0300 0318
```

(c) 2026 masanorisakai / MIT License

```
0300  A2 0F     LDX   #$0F
0302  9E        .BYTE $9E
0303  D0 03     BNE   $0308
0305  CA        DEX
0306  10 FA     BPL   $0302
0308  9C        .BYTE $9C
0309  F0 03     BEQ   $030E
030B  A9 00     LDA   #$00
030D  C9 03     CMP   #$03
030F  F0 08     BEQ   $0319
0311  EE F0 03  INC   $03F0
0314  D0 F7     BNE   $030D
0316  1A        .BYTE $1A
0317  80        .BYTE $80
0318  F4        .BYTE $F4
```

バイトは同じですが、6502 は `$9E`、`$9C`、`$1A`、`$80` を知らないので、`L` は `.BYTE` と表示します。Glow65 の 6502 は、このような未定義のオペコードを1バイトの NOP(2サイクル)として実行します。そのあとに続くバイト(`stz` や `bra` のオペランドだったもの)は、それぞれ独立した命令として読まれます。`D0 03` は `BNE`、`F0 03` は `BEQ` になります。実行します。

```
G 0300
D 03D0 03F1
```

```
03D0: AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
03E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
03F0: 00 00 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

何も消えておらず、領域は `$AA` のままです。カウンタは `00 03` ではなく `00 00` です。ここでは `inc a` が NOP なので、A は 3 になりません。下位バイトが256通りを1周すると、A が 0 のままループを抜けます。プログラムはクラッシュせず、エラーも出さず、最後まで実行して、間違った答えでモニタに戻りました。CPU は警告してくれません。65C02 用のプログラムを 6502 のマシンに読み込むと、こうなります。

逆の向きは問題ありません。6502 用のプログラムは 65C02 にもある命令だけを使っているので、`clear_6502.bin` は、どちらのマシンでも同じダンプになります。

```
03D0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
03E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
03F0: 00 03 00 00 00 00 00 00 24 F4 24 F4 00 00 00 00
```

(c) 2026 masanorisakai / MIT License

参考までに、`clear_6502.bin` に対する `L 0300 031F` は 6502 の命令だけを表示し、どちらの CPU でも同じ行を出力します。

```
0300  A2 0F     LDX   #$0F
0302  A9 00     LDA   #$00
0304  9D D0 03  STA   $03D0,X
0307  CA        DEX
0308  10 FA     BPL   $0304
030A  8D F0 03  STA   $03F0
030D  C9 03     CMP   #$03
030F  F0 0B     BEQ   $031C
0311  EE F0 03  INC   $03F0
0314  D0 F7     BNE   $030D
0316  18        CLC
0317  69 01     ADC   #$01
0319  4C 0D 03  JMP   $030D
031C  8D F1 03  STA   $03F1
031F  60        RTS
```

6. パネルの CPU スイッチ

アプリでは、パネルの **CPU** スイッチで 6502 と 65C02 を切り替えます。操作できるのは POWER が OFF のときだけで、起動時の CPU は `--cpu` オプションで選びます。CPU を変えるには、POWER を OFF にしてスイッチを動かし、もう一度 POWER を ON にします。電源を入れると RAM が消えるので、プログラムは POWER を ON にしたあとで読み込みます。`--load` は毎回そうしてくれます。**File → Load Program…** を使うか、バイトを入力しても構いません。

7. 試してみよう

1. **上限を変える。** 65C02 のマシンで `E 030E 01` と入力して `cmp` のオペランドを変え、もう一度 `G 0300` を実行します。カウンタは `$0100` で止まります。`D 03F0 03F1` が出力する行は、`00 01` で始まります。
2. **X を積む。** `E 0300 A2 07 DA A2 00 FA 8E F0 03 60` と入力します。プログラムは `ldx #$07`、`phx`、`ldx #$00`、`plx`、`stx $03F0`、`rts` です。`G 0300` で実行して、`D 03F0 03F0` を見ます。65C02 では先頭のバイトが `07` です。`plx` が 7 を戻してくれました。6502 では `00` です。`phx` と `plx` が何もしないからです。
3. **自分で書く。** `dec a` と `bra` を使って、A で 10 から数え下ろし、最後に `stz` で 0 を書きます。

(c) 2026 masanorisakai / MIT License

8. 覚えておくこと

- 65C02 は 6502 のプログラムをすべて実行でき、`stz`、`inc a`、`bra`、`phx`、`plx` などの命令が加わっています。
- どちらの CPU 向けかは、`.setcpu "6502"` か `.setcpu "65C02"` で `ca65` に伝えます。
- 6502 では、新しいオペコードは未定義です。Glow65 はそれを NOP として実行するので、65C02 用のプログラムはエラーなしで間違った結果を出します。
- `L` が `STZ`、`BRA` などの新しい命令を名前で表示するのは、65C02 のマシンだけです。6502 では `.BYTE` と表示します。
- CPU を変えられるのは、POWER が OFF のときだけです。

(c) 2026 masanorisakai / MIT License